

Documento maestro de investigación

COBRIA

Diseño y evaluación de contratos reconstruibles
para sistemas NoSQL documentales

Billy Jak Cordero Porras

Investigador independiente
Coto Brus, Costa Rica

Edición de investigación · 2026

Declaración de alcance y originalidad

Este manuscrito propone, formaliza y evalúa una integración arquitectónica denominada COBRIA. No se atribuye como invención propia ninguno de los patrones consolidados que la componen, entre ellos Repositorio, Mapeador de datos, Capa de servicios, casos de uso, vistas materializadas, separación de modelos de lectura y escritura o generación dirigida por metadatos. La contribución buscada reside en su articulación explícita alrededor de entidades canónicas, proyecciones reconstruibles, aislamiento multiinquilino y preparación trazable para minería de datos e inteligencia artificial, así como en su evaluación mediante una implementación neutral y dos perfiles de caso anonimizados.

Los casos se describen de forma neutral como *Sistema A* y *Sistema B*. Los experimentos emplearon datos sintéticos o semirrealistas y configuraciones reproducibles. No se publicarán nombres comerciales, credenciales, datos personales ni detalles que permitan identificar organizaciones o personas.

Resumen

Los sistemas NoSQL permiten construir aplicaciones flexibles y distribuidas, pero trasladan al diseño de la aplicación decisiones complejas sobre desnormalización, índices, consistencia, aislamiento entre organizaciones y evolución del esquema. Estas dificultades aumentan cuando los mismos datos deben alimentar consultas operacionales, procesamiento de eventos, minería de datos e inteligencia artificial. Este trabajo presenta COBRIA (*Capa Orientada a Bases Reconstruibles para Inteligencia Analítica*), una arquitectura dirigida por metadatos que distingue entidades canónicas de representaciones físicas y proyecciones derivadas. La propuesta organiza el acceso mediante repositorios, servicios y casos de uso; clasifica las proyecciones según sus requisitos de consistencia; y exige que todo dato derivado declare su origen, versión y mecanismo de reconstrucción. La investigación siguió ciencia del diseño y examinó la propuesta en tres aplicaciones documentales anonimizadas. La evaluación comprende inspección de contratos, pruebas NoSQL, reconstrucción de proyecciones, aislamiento por ámbito, evolución de esquemas y recuperación autorizada para analítica e inteligencia artificial. El repositorio conserva 270 corridas históricas distribuidas sobre MongoDB, CouchDB y OpenSearch, además de una repetición preliminar del arnés corregido en MongoDB y CouchDB. Las corridas históricas no se presentan como certificación vigente porque una auditoría posterior fortaleció el oráculo. Las mediciones SQL permanecen segregadas y no sostienen afirmaciones de rendimiento NoSQL. Permanecen pendientes la repetición distribuida confirmatoria, la evaluación humana de transferencia y la revisión independiente.

Palabras clave: NoSQL; datos canónicos; proyecciones reconstruibles; multiinquilinato; vistas materializadas; minería de datos; inteligencia artificial; arquitectura dirigida por metadatos.

Índice general

Declaración de alcance y originalidad	I
Resumen	II
I Fundamentos de la investigación	1
1 Introducción	2
1.1 Contexto	2
1.2 Problema de investigación	3
1.3 Motivación	4
1.3.1 Motivación técnica	4
1.3.2 Motivación de mantenibilidad	5
1.3.3 Motivación analítica	5
1.3.4 Motivación social y regional	5
1.4 La propuesta COBRIA en síntesis	5
1.5 Vacío de investigación	6
1.6 Síntesis ampliada de trabajos relacionados	7
1.7 Objetivos	8
1.7.1 Objetivo general	8
1.7.2 Objetivos específicos	8
1.8 Preguntas de investigación	10
1.9 Hipótesis	10
1.10 Contribuciones esperadas	12
1.10.1 Contribución conceptual	12
1.10.2 Contribución formal	12
1.10.3 Contribución arquitectónica	13
1.10.4 Contribución algorítmica	13
1.10.5 Contribución empírica	13

1.10.6	Contribución práctica	13
1.11	Casos de estudio y neutralidad del análisis	13
1.12	Alcance	14
1.13	Delimitaciones	15
1.14	Unidad de análisis y niveles de observación	16
1.15	Consideraciones éticas iniciales	16
1.16	Organización del documento maestro	17
1.17	Comparación sistemática con arquitecturas relacionadas	17
1.18	Síntesis de la Parte I	18
II	Diseño y formalización de COBRIA	20
2	Metodología para el diseño de COBRIA	21
2.1	Enfoque de ciencia del diseño	21
2.2	Proceso de abstracción desde los casos	22
2.3	Criterios de diseño	23
2.4	Evidencia prevista	24
3	Arquitectura COBRIA	25
3.1	Definición	25
3.2	Capas y dirección de dependencias	26
3.3	Módulos internos	26
3.3.1	COBRIA Núcleo	26
3.3.2	COBRIA Metadatos	26
3.3.3	COBRIA Proyecciones	26
3.3.4	COBRIA Analítica	27
3.3.5	COBRIA Inteligencia	27
3.3.6	COBRIA Gobierno	27
3.4	COBRIA frente a patrones relacionados	27
4	Modelo formal de datos canónicos	28
4.1	Conjuntos fundamentales	28
4.2	Atributos y ausencia de valor	28
4.3	Ámbito y multiinquilinato	29
4.4	Ciclo de vida	29
4.5	Revisión y concurrencia	29

4.6	Invariantes canónicas	30
5	Representación física y persistencia	31
5.1	Mapeo semántico–físico	31
5.2	Repositorio y Mapeador de datos	32
5.3	Mapa de identidad y caché	32
5.4	Pasarela de almacenamiento	32
5.5	Registro de rutas semánticas	33
5.6	Capa de servicios y casos de uso	33
5.7	Contexto autorizado como dato derivado confiable	34
6	Teoría de proyecciones reconstruibles	35
6.1	Definición	35
6.2	Taxonomía	36
6.3	Propiedades	36
6.3.1	Determinismo	36
6.3.2	Idempotencia	36
6.3.3	Reconstructibilidad	37
6.3.4	No autoridad	37
6.4	Grafo de dependencias	37
6.5	Contrato de proyección	37
7	Algoritmos, consistencia y recuperación	39
7.1	Creación de una entidad	39
7.2	Actualización optimista	39
7.3	Reconstrucción total	40
7.4	Reconstrucción incremental	40
7.5	Verificación de integridad	41
7.6	Consistencia por clase	42
7.7	Estado materializado: completitud, frescura y procedencia	42
7.8	Arquitectura ejecutable y funciones de aptitud	43
7.9	Cola de salida, reintentos y duplicados	43
7.10	Recuperación y observabilidad	44
7.11	Antipatrones	44
7.12	Criterio mínimo de adopción	45
7.13	Síntesis de la Parte II	45

III Rendimiento, complejidad y optimización	46
8 Modelo de rendimiento y notación	47
8.1 Variables del modelo	47
8.2 Frontera de medición	48
8.3 Métricas principales	48
9 Complejidad computacional de las operaciones	50
9.1 Acceso canónico	50
9.2 Creación y actualización	50
9.3 Reconstrucción total e incremental	51
9.4 Verificación de proyecciones	51
10 Amplificación y modelo de costos	53
10.1 Amplificación de lectura	53
10.2 Amplificación de escritura	53
10.3 Amplificación de almacenamiento	53
10.4 Costo total por ventana	54
10.5 Punto de equilibrio de una proyección	54
10.6 Presupuesto de proyecciones	54
11 Estrategias de optimización	56
11.1 Optimizar desde la consulta	56
11.2 Lotes y paralelismo controlado	56
11.3 Caché con ámbito y revisión	57
11.4 Reconstrucción eficiente	57
11.5 Optimización adaptativa	57
12 Protocolo experimental de rendimiento	59
12.1 Objetivo y comparaciones	59
12.2 Factores y niveles	60
12.3 Datos y cargas reproducibles	60
12.4 Instrumentación y control	61
12.5 Análisis estadístico	61
12.6 Criterios de aceptación	61
12.7 Plantilla de resultados	62
13 Rendimiento para minería de datos e inteligencia artificial	63

13.1	Preparación analítica como proyección	63
13.2	Corrección temporal y prevención de fuga	63
13.3	Características, embeddings y multimodalidad	64
13.4	RAG y agentes	64
13.5	Optimización de cadenas de procesamiento de IA	64
13.6	Métricas analíticas y de IA	65
14	Amenazas a la validez y síntesis de la Parte III	66
14.1	Validez interna	66
14.2	Validez de constructo	66
14.3	Validez externa	66
14.4	Validez de conclusión	67
14.5	Síntesis de la Parte III	67
IV	Analítica, minería de datos e inteligencia artificial	68
15	Extensión analítica de COBRIA	69
15.1	De la operación al conocimiento	69
15.2	Capacidades analíticas	70
15.3	Componentes de la extensión	70
15.4	Principios de adaptación	71
16	Contratos de conjuntos de datos y linaje	72
16.1	Definición formal	72
16.2	Contrato mínimo	72
16.3	Grafo de linaje	73
16.4	Algoritmo de materialización	74
16.5	Calidad y observabilidad	74
17	Almacén de características, series temporales y minería de datos	75
17.1	Características como proyecciones	75
17.2	Equivalencia fuera de línea—en línea	75
17.3	Corrección punto-en-tiempo	76
17.4	Eventos tardíos y marcas de avance temporal	76
17.5	Minería descriptiva y descubrimiento	77
17.6	Catálogo analítico	77

18 Multimodalidad, embeddings y búsqueda semántica	78
18.1 Objeto multimedia canónico	78
18.2 Pipeline multimodal	78
18.3 Contrato de embedding	79
18.4 Fragmentación	79
18.5 Recuperación híbrida	79
18.6 Derecho a retirar y reconstrucción	80
19 Recuperación aumentada y agentes gobernados	81
19.1 RAG como composición de proyecciones	81
19.2 Flujo seguro de recuperación	81
19.3 Contrato de herramienta para agentes	82
19.4 Memoria de agentes	83
19.5 Evidencia y verificabilidad	83
20 Gobierno, seguridad y gestión del riesgo	84
20.1 Clasificación de artefactos	84
20.2 Registro y documentación de modelos	84
20.3 Separación de funciones	85
20.4 Amenazas específicas	85
20.5 Controles por frontera	85
20.6 Ciclo de vida	86
20.7 Monitoreo y respuesta	86
21 Evaluación de adaptabilidad analítica e inteligente	87
21.1 Preguntas de evaluación	87
21.2 Escenarios anonimizados	87
21.3 Experimentos propuestos	88
21.4 Métricas de linaje y reproducibilidad	88
21.5 Métricas de recuperación	89
21.6 Métricas de agentes	89
21.7 Criterios de adaptabilidad	89
21.8 Resultados negativos previstos	89
21.9 Síntesis de la Parte IV	90

V	Implementación de referencia y casos de estudio	91
22	Método de análisis de las implementaciones	92
22.1	Preguntas del estudio de implementación	92
22.2	Fuentes de evidencia	93
22.3	Procedimiento de inventario	93
22.4	Neutralidad y ética	93
23	Implementación neutral de referencia	95
23.1	Objetivo	95
23.2	Comportamientos comprobados de la aplicación pedagógica	95
23.3	Estructura propuesta	95
23.4	Entidad canónica	96
23.5	Mapeador y Repositorio	97
23.6	Dater como variante de acceso a datos	98
23.7	Contrato de proyección ejecutable	98
23.8	Caso de uso y transacción lógica	99
23.9	Pruebas mínimas	99
24	Caso de estudio A: sistema orientado a eventos y contenido	100
24.1	Caracterización estructural	100
24.2	Objetos y serialización	101
24.3	Daters e índices derivados	101
24.4	Servicios, casos de uso e IA	101
24.5	Fortalezas observadas	102
24.6	Deuda y oportunidades	102
24.7	Evolución longitudinal posterior del Sistema A	102
24.8	Implicación para la validez del caso	104
25	Caso de estudio B: sistema empresarial multiinquilino	105
25.1	Caracterización estructural	105
25.2	Modelo base y mapeo físico	106
25.3	Dater base	106
25.4	Metadatos, relaciones y catálogos	106
25.5	Servicios y operaciones críticas	107
25.6	Fortalezas observadas	107
25.7	Deuda y oportunidades	108

26 Comparación entre casos y derivación de COBRIA	109
26.1 Similitudes estructurales	109
26.2 Diferencias que prueban adaptabilidad	110
26.3 Elementos heredados y contribución	110
26.4 Patrones de diseño identificados	111
26.5 Modelo de madurez	112
27 Estrategia de adopción y migración	113
27.1 Adopción incremental	113
27.2 estrangulador para datos	113
27.3 Priorización por riesgo y valor	114
27.4 Generación dirigida por metadatos	114
27.5 Compatibilidad y versionado	114
27.6 Presupuesto de deuda y retiro de puentes	114
27.7 Criterio de finalización	115
28 Reproducibilidad, limitaciones y síntesis de la Parte V	116
28.1 Paquete reproducible previsto	116
28.2 Limitaciones del análisis estático	116
28.3 Riesgo de sesgo del investigador	117
28.4 Validez de la generalización	117
28.5 Síntesis de la Parte V	117
VI Evaluación experimental, análisis y validez	118
29 Diseño de evaluación exclusivamente NoSQL	119
29.1 Delimitación tecnológica	119
29.2 Casos documentales anonimizados	119
30 Hipótesis, variables y protocolo	121
30.1 Hipótesis activas	121
30.2 Variables y unidades	121
31 Experimentos NoSQL	123
31.1 P1: lectura selectiva documental	123
31.2 P2: amplificación de escritura	123
31.3 R1: reconstrucción total e incremental	123

31.4	S1: aislamiento multiinquilino	123
31.5	A1: analítica e inteligencia artificial	124
32	Evidencia y resultados responsables	125
32.1	Evidencia disponible	125
32.2	Resultados COBRIA Core 1.0	125
33	Amenazas a la validez y síntesis	127
VII	Síntesis, adopción y conclusiones	128
34	Síntesis integral de COBRIA	129
34.1	Respuesta a la investigación	129
35	Guía de adopción	130
36	Contribuciones y límites	131
37	Conclusión	132
VIII	Manual de réplica y operación NoSQL	133
38	Propósito del manual de réplica	134
38.1	Resultado esperado	134
38.2	Principios de neutralidad	134
39	Selección y descripción de motores documentales	135
39.1	Criterios de inclusión	135
39.2	Ficha del motor	135
40	Modelo documental neutral	137
40.1	Documento canónico	137
40.2	Proyección de cobertura	137
40.3	Catálogos y evolución	137
41	Generación de cargas semirrealistas	138
41.1	Perfiles	138
41.2	Anomalías controladas	138

42 Protocolo P1: lectura documental	140
42.1 Configuraciones	140
42.2 Ejecución	140
42.3 Interpretación	140
43 Protocolo P2: escritura y convergencia	141
43.1 Camino de actualización	141
43.2 Amplificación	141
43.3 Frontera de decisión	141
44 Protocolos de reconstrucción y publicación	142
44.1 Reconstrucción total	142
44.2 Reconstrucción incremental	142
44.3 Interrupción segura	142
45 Aislamiento, autorización y recuperación	143
45.1 Matriz adversarial	143
45.2 Pruebas negativas	143
46 Productos para minería de datos e inteligencia artificial	144
46.1 Conjunto reproducible	144
46.2 Recuperación con evidencia	144
46.3 Herramientas de agentes	144
47 Observabilidad y costos	145
47.1 Indicadores	145
47.2 Presupuesto de proyecciones	145
47.3 Criterio de retiro	145
48 Análisis estadístico y resultados negativos	146
48.1 Unidad inferencial	146
48.2 Multiplicidad y sensibilidad	146
48.3 Publicación de derrotas	146
49 Paquete abierto y revisión independiente	147
49.1 Contenido del depósito	147
49.2 Revisión técnica	147
49.3 Criterio de cierre	147

50 Cuadernos de campo reproducibles	148
50.1 Bitácora de preparación	148
50.2 Bitácora de ejecución	148
50.3 Bitácora de cierre	148
51 Migraciones y evolución del contrato	150
51.1 Cambio aditivo	150
51.2 Cambio incompatible	150
51.3 Reversión	150
52 Rúbricas de conformidad COBRIA	151
52.1 Rúbrica del documento canónico	151
52.2 Rúbrica del derivado	151
52.3 Rúbrica del uso analítico	151
53 Escenarios de aceptación integral	152
53.1 Escenario A: lectura y actualización concurrentes	152
53.2 Escenario B: reconstrucción bajo cambio de esquema	152
53.3 Escenario C: recuperación con inteligencia artificial	152
53.4 Escenario D: degradación y recuperación	153
IX Anexos técnicos y reproducibilidad	154
54 Glosario operacional	155
55 Plantillas de contratos	157
55.1 Contrato de entidad	157
55.2 Contrato de Mapeador	157
55.3 Contrato de proyección	158
55.4 Contrato de conjunto de datos	158
55.5 Contrato de característica	159
55.6 Contrato de herramienta de agente	159
56 Listas de comprobación de ingeniería y gobierno	161
56.1 Lista de comprobación de entidad canónica	161
56.2 Lista de comprobación de proyección	161
56.3 Lista de comprobación multiinquilino	162

56.4	Lista de comprobación de IA y agentes	162
57	Diccionario de métricas	164
58	Esquema del paquete reproducible	166
58.1	Estructura de directorios	166
58.2	Manifiesto de corrida	167
58.3	Formato de evento de medición	167
58.4	Reglas de publicación	168
58.5	Lista de comprobación de réplica	168
59	Plantillas de registro y publicación	170
59.1	Ficha de experimento	170
59.2	Ficha de resultado	171
59.3	Ficha de incidente experimental	171
59.4	Declaración de disponibilidad	171
59.5	Declaración de autoría	171
60	Matriz de trazabilidad del manuscrito	172
60.1	Trazabilidad de partes	173
60.2	Criterio de completitud para una versión publicable	173
60.3	Cierre de los anexos	173
	Referencias	174

Índice de figuras

1.1	Vista conceptual inicial de COBRIA. Las proyecciones optimizan consumidores específicos, pero su autoridad procede de las entidades canónicas.	6
3.1	Capas y preocupaciones transversales de COBRIA.	26
6.1	Ejemplo de grafo acíclico de reconstrucción.	37
11.1	Ciclo gobernado de optimización adaptativa.	58

13.1	Flujo gobernado desde datos canónicos hasta decisiones asistidas por IA.	65
15.1	Componentes de la extensión analítica e inteligente de COBRIA.	71
17.1	Contratos sintéticos de recuperación en 30 repeticiones. No constituyen evaluación humana de un modelo generativo.	76
18.1	Pipeline multimodal reconstruible.	78
24.1	Flujo de datos observado y abstraído en el Sistema A.	101
25.1	Abstracción del acceso común observado en el Sistema B.	106
27.1	Migración incremental mediante una fachada compatible.	114

Índice de cuadros

1.1	Preguntas de investigación de COBRIA.	10
1.2	Hipótesis y evidencia prevista.	11
1.3	Caracterización inicial y anónima de los casos.	14
1.4	Comparación entre COBRIA y enfoques relacionados.	18
2.1	Normalización conceptual de componentes observados.	23
5.1	Ejemplo de mapeo semántico–físico versionado.	31
6.1	Taxonomía de proyecciones COBRIA.	36
7.1	Mecanismos de consistencia recomendados.	42
8.1	Símbolos del modelo de rendimiento.	48
9.1	Complejidad esperada de operaciones COBRIA.	52
12.1	Factores propuestos para los experimentos.	60
12.2	Matriz mínima de resultados por escenario.	62
15.1	Adaptación de COBRIA a modalidades analíticas.	70

16.1	Contenido mínimo de un contrato de conjunto de datos.	72
19.1	Niveles de autonomía propuestos.	82
20.1	Gobierno según tipo de artefacto.	84
21.1	Experimentos para analítica e inteligencia artificial.	88
22.1	Niveles de evidencia utilizados.	93
23.1	Correspondencia del Dater con patrones consolidados.	98
24.1	Inventario neutral del Sistema A.	100
24.2	Evidencia longitudinal neutral del Sistema A.	103
25.1	Inventario neutral del Sistema B.	105
26.1	Comparación neutral de los casos.	109
26.2	Patrones observados y papel dentro de COBRIA.	111
26.3	Niveles de madurez COBRIA para una entidad.	112
28.1	Contenido del paquete reproducible.	116
29.1	Perfiles NoSQL utilizados en la evaluación.	120
39.1	Campos mínimos de la ficha del motor NoSQL.	136
41.1	Anomalías que debe producir el generador.	138
54.1	Glosario COBRIA.	155
56.1	Verificación de una entidad canónica.	161
56.2	Verificación de una proyección reconstruible.	162
56.3	Verificación de aislamiento por ámbito.	162
56.4	Verificación de componentes inteligentes.	163
57.1	Definición de métricas del estudio.	164
59.1	Plantilla de ficha experimental.	170
59.2	Plantilla de resultado.	171
60.1	Trazabilidad entre preguntas, artefactos y evaluación.	172
60.2	Función de cada parte del documento maestro.	173

Parte I

Fundamentos de la investigación

Introducción

1.1. Contexto

Los sistemas de información contemporáneos deben atender simultáneamente necesidades que antes solían resolverse en plataformas separadas. Una misma aplicación puede registrar operaciones de negocio, reaccionar a eventos en tiempo real, conservar un historial auditable, aislar información de varias organizaciones y producir datos para analítica o inteligencia artificial. Esta convergencia ha aumentado el valor de las bases de datos NoSQL, cuya flexibilidad permite representar estructuras heterogéneas y distribuir grandes volúmenes de información. Sistemas influyentes como Bigtable demostraron que un modelo distribuido orientado a columnas puede soportar cargas muy diferentes cuando el diseño físico se alinea con los patrones de acceso (Chang et al., 2006). Dynamo, por su parte, expuso de forma explícita los compromisos entre disponibilidad, particionamiento, replicación y consistencia en un almacenamiento distribuido orientado a clave (DeCandia et al., 2007).

Esa flexibilidad no elimina la necesidad de modelar; la desplaza. En sistemas NoSQL, la aplicación debe decidir qué datos se almacenan juntos, cuáles se duplican, qué índices se mantienen y cómo se reparan las representaciones derivadas. El esquema puede ser flexible en la base, pero sigue existiendo en el código, en las rutas, en las reglas de acceso y en las expectativas de cada consumidor.

La dificultad aumenta en aplicaciones *multiinquilino*, es decir, aquellas en las que una misma plataforma sirve a varias organizaciones o contextos lógicos. La consolidación puede reducir costos, pero introduce requisitos estrictos de aislamiento, configuración, rendimiento y recuperación. La literatura ha mostrado que no existe un único diseño físico que sea óptimo para todas las organizaciones y cargas; las alternativas intercambian escalabilidad, espacio, flexibilidad y complejidad operativa (Aulbach et al., 2009; Rico et al., 2016; Yaish et al., 2014).

Al mismo tiempo, los datos operacionales han dejado de ser únicamente un registro del

pasado. Se utilizan para detectar anomalías, descubrir secuencias, construir modelos predictivos, recuperar evidencias y asistir decisiones humanas. Los sistemas de aprendizaje automático necesitan características consistentes entre entrenamiento e inferencia, versiones reproducibles y trazabilidad hacia las fuentes. Los almacenes de características surgieron precisamente para organizar este flujo y reducir la fragmentación entre preparación, entrenamiento y servicio de modelos (Li et al., 2023; Orr et al., 2021). Sin embargo, una capa analítica no puede corregir por sí sola un origen operacional ambiguo o lleno de duplicaciones sin propietario.

Este trabajo parte de una observación práctica: los sistemas NoSQL de alcance amplio necesitan distinguir con claridad qué representación conserva el estado autorizado del negocio y cuáles representaciones existen para acelerar consultas, integrar módulos o alimentar procesos analíticos. A partir de esa separación se formula COBRIA, una arquitectura que organiza entidades canónicas, mapeos físicos, repositorios, servicios, casos de uso y proyecciones reconstruibles dentro de una misma disciplina de diseño.

1.2. Problema de investigación

En un modelo desnormalizado, una modificación lógica puede afectar varias ubicaciones. Por ejemplo, actualizar el estado de una entidad puede requerir modificar su registro principal, retirar una referencia de un índice, añadirla a otro y actualizar un resumen. Si estas ubicaciones se tratan como fuentes equivalentes, el sistema pierde una base clara para decidir cuál valor es correcto cuando aparece una discrepancia. Si, por el contrario, se reconoce una fuente canónica y las demás se definen como funciones derivadas, la discrepancia puede detectarse y repararse.

La vista materializada es un antecedente directo de esta idea: una representación derivada puede mejorar el acceso, pero debe mantenerse frente a cambios en los datos base. La investigación clásica sobre mantenimiento incremental estudió cómo calcular los cambios de una vista sin reconstruirla completamente (Gupta & Mumick, 1995; Gupta et al., 1993). En aplicaciones NoSQL, el mismo problema reaparece en índices manuales, resúmenes, relaciones inversas, contadores, colas de salida y conjuntos de datos analíticos, aunque no siempre se describa con esa terminología.

Se identifican seis manifestaciones principales del problema:

- P1. Ambigüedad de autoridad.** Varias copias contienen atributos similares, pero no se declara cuál es la fuente autorizada.
- P2. Consultas no acotadas.** Ante la ausencia de índices apropiados, el cliente recupera colecciones grandes y filtra localmente, aumentando transferencia, memoria, latencia y exposición innecesaria.

- P3. Derivaciones frágiles.** Los índices se actualizan mediante lógica dispersa y carecen de versión, propietario, verificador o procedimiento de reconstrucción.
- P4. Acoplamiento semántico–físico.** Nombres, claves compactas, rutas y reglas se duplican en múltiples capas; una evolución del esquema puede dejar consumidores incompatibles.
- P5. Aislamiento incompleto.** El organización se aplica como filtro de interfaz, pero no como dimensión obligatoria del modelo, la consulta y la autorización.
- P6. Preparación analítica no reproducible.** Los conjuntos de datos y características se crean con exportaciones o scripts independientes, sin linaje suficiente hacia los datos operacionales y sin corrección temporal verificable.

Estas manifestaciones están relacionadas. Una consulta amplia suele motivar la creación de un índice; el índice introduce duplicación; la duplicación exige mantenimiento; el mantenimiento necesita autoridad y trazabilidad; y la trazabilidad se vuelve todavía más importante cuando el resultado alimenta un modelo de IA. El problema de investigación se formula, por tanto, de la siguiente manera:

¿Cómo diseñar y evaluar una arquitectura NoSQL multiinquilino que preserve una fuente canónica comprensible, permita optimizar consultas mediante proyecciones derivadas, pueda reconstruir esas proyecciones de forma verificable y ofrezca datos trazables para minería de datos e inteligencia artificial?

1.3. Motivación

1.3.1. Motivación técnica

El rendimiento de una aplicación conectada a una base remota no depende solamente del tiempo de CPU. Importan también el número de solicitudes, los bytes transferidos, la cantidad de registros procesados por el cliente y la variabilidad de la red. Una proyección pequeña puede reducir el conjunto candidato de N entidades a K resultados relevantes, donde $K \ll N$. No obstante, esta mejora de lectura incrementa el trabajo de escritura y almacenamiento. COBRIA busca hacer visible ese intercambio, en vez de tratar cada índice como una optimización gratuita.

1.3.2. Motivación de mantenibilidad

En sistemas extensos, una entidad puede estar representada por un modelo de dominio, un serializador, un repositorio, un servicio, reglas de validación, índices, catálogos y documentación. Cuando esos artefactos evolucionan de manera independiente, aparecen errores difíciles de detectar. Un enfoque dirigido por metadatos puede reducir la duplicación de decisiones, siempre que no convierta los archivos generados en una falsa medida de funcionalidad. COBRIA propone evaluar la generación por su capacidad para preservar contratos y reducir defectos, no por la cantidad de código producido.

1.3.3. Motivación analítica

La minería de datos requiere transformar eventos y entidades en series, agregaciones, secuencias, grafos o vectores de características. Cuando esas transformaciones se consideran proyecciones versionadas, pueden reconstruirse desde datos canónicos, compararse entre versiones y auditarse. Este enfoque conecta la arquitectura operacional con prácticas de Almacén de características y MLOps, pero evita asumir que cualquier proyección operacional es automáticamente adecuada para entrenar modelos.

1.3.4. Motivación social y regional

El nombre COBRIA incorpora de manera simbólica la identidad de Coto Brus, Costa Rica: **CO-BR-IA**. La referencia territorial no cambia el carácter general del artefacto; expresa que la investigación tecnológica puede originarse fuera de los centros tradicionales y producir conocimiento susceptible de evaluación y reutilización internacional. El estudio evitará, sin embargo, convertir esa identidad en una afirmación técnica o en evidencia de validez.

1.4. La propuesta COBRIA en síntesis

COBRIA significa *Capa Orientada a Bases Reconstruibles para Inteligencia Analítica*. La palabra *objeto* se utiliza en un sentido amplio: una entidad semántica identificable, no necesariamente una clase ligada a un lenguaje de programación específico.

La arquitectura propone cinco movimientos principales:

1. representar el estado autorizado mediante entidades canónicas;
2. separar nombres semánticos de la representación física almacenada;

3. encapsular persistencia, reglas y operaciones mediante repositorios, servicios y casos de uso;
4. definir índices, resúmenes, relaciones y conjuntos de datos como proyecciones con contrato de reconstrucción;
5. conservar el linaje necesario para que minería de datos e IA utilicen representaciones versionadas, autorizadas y temporalmente correctas.

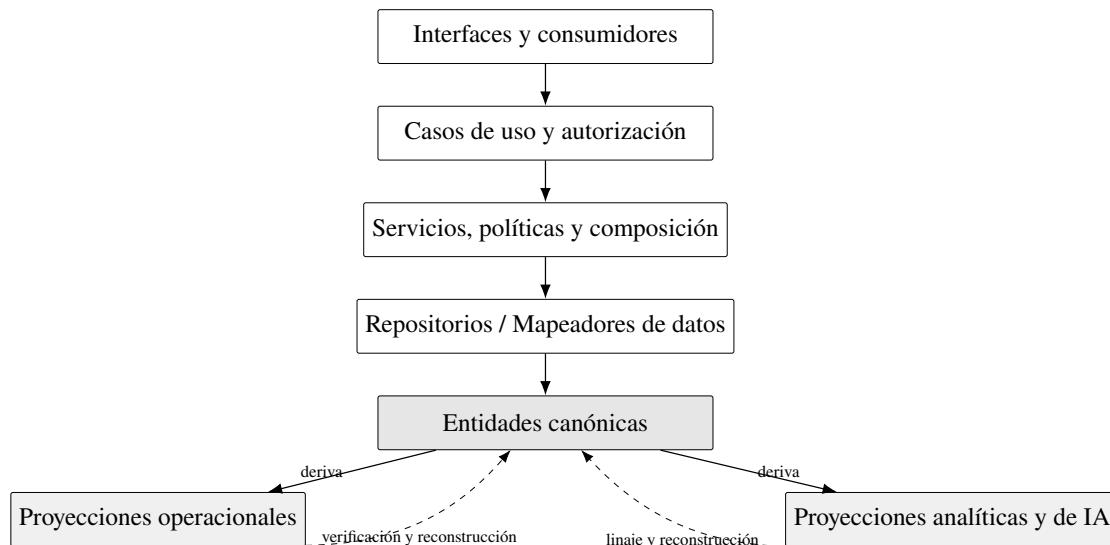


Figura 1.1: Vista conceptual inicial de COBRIA. Las proyecciones optimizan consumidores específicos, pero su autoridad procede de las entidades canónicas.

La flecha de reconstrucción en la figura 1.1 no implica que una proyección sobrescriba libremente la entidad canónica. Indica que la proyección puede compararse con el resultado esperado de su función de derivación y regenerarse cuando sea necesario. Las correcciones del estado de negocio deben realizarse por medio de un caso de uso autorizado.

1.5. Vacío de investigación

COBRIA se apoya en conceptos conocidos. Repositorio y Mapeador de datos separan el dominio de la persistencia; las vistas materializadas aceleran consultas; CQRS distingue modelos de lectura y escritura; las arquitecturas multiinquilino estudian la consolidación y el aislamiento; los Almacenes de características organizan características para aprendizaje automático; y la ciencia del diseño ofrece un método para construir y evaluar artefactos (Hevner et al., 2004; Peffers et al., 2007).

El vacío que motiva este trabajo no es la inexistencia de esas ideas, sino la falta de una articulación y evaluación conjunta para aplicaciones NoSQL basadas en estructuras JSON que:

- trate la reconstruibilidad como contrato de toda proyección, no solo como una tarea de mantenimiento ocasional;
- conecte el modelo semántico y la representación física compacta mediante un mapeo versionado y reversible;
- incorpore el organización en identidad, rutas, consultas, autorización y pruebas;
- incluya proyecciones operacionales y analíticas bajo una taxonomía común;
- mida simultáneamente beneficios de lectura y ampliaciones de escritura y almacenamiento;
- preserve linaje desde la entidad operacional hasta características, predicciones y retroalimentación humana;
- sea contrastada en más de un tipo de sistema y exponga también los casos en que su complejidad no se justifica.

Por tanto, la originalidad pretendida es *composicional* y *evaluativa*. El estudio no afirmará que COBRIA sustituye todas las alternativas documentales ni que su complejidad sea preferible para cualquier problema.

1.6. Síntesis ampliada de trabajos relacionados

La procedencia de datos antecede a COBRIA: la literatura distingue el origen de un resultado y las partes de la fuente responsables de él (Buneman et al., 2001). COBRIA adopta esa preocupación, pero la lleva al contrato operacional de cada proyección: fuente, versión, transformación, alcance y procedimiento verificable de reconstrucción. Por ello, el linaje no se presenta como una invención, sino como una condición para que una copia optimizada pueda auditarse y regenerarse.

La evolución de esquemas NoSQL también es un problema conocido. La ausencia de un esquema global puede trasladar heterogeneidad y migraciones al código de aplicación, y se han propuesto lenguajes declarativos para administrar esos cambios sin modificar el motor (Scherzinger et al., 2013). COBRIA coincide en explicitar versiones, pero añade la separación entre semántica canónica, mapeo físico y derivados reconstruibles.

En analítica, la arquitectura lakehouse busca reunir almacenamiento abierto, gobierno, inteligencia de negocios y aprendizaje automático (Armbrust et al., 2021). Su escala es la plataforma de datos; COBRIA opera en otra capa: contratos de dominio y proyecciones que pueden alimentar una plataforma analítica sin convertirla en autoridad del estado transaccional. Ambas ideas pueden coexistir.

Finalmente, los sistemas de datos autoajustables estudian el uso del historial de carga para planificar cambios físicos (Pavlo et al., 2017). Esto delimita una línea futura para COBRIA: recomendar o retirar proyecciones con un presupuesto observado. La presente evaluación no implementa un optimizador autónomo; mide primero los beneficios y costos que una política de ese tipo necesitaría comparar.

En conjunto, la revisión sitúa la novedad posible en la integración y en sus contratos, no en renombrar vistas materializadas, procedencia, CQRS, repositorios o evolución de esquema. La contribución es refutable porque puede fallar si la reconstrucción no es exacta, si el aislamiento se rompe o si la amplificación supera el ahorro de lectura.

1.7. Objetivos

1.7.1. Objetivo general

Diseñar, formalizar y evaluar COBRIA como una arquitectura dirigida por metadatos, basada en entidades canónicas y proyecciones reconstruibles, para determinar su efecto sobre la eficiencia, integridad, mantenibilidad, recuperabilidad, aislamiento multiinquilino y preparación para minería de datos e inteligencia artificial en sistemas NoSQL.

1.7.2. Objetivos específicos

- OE1.** Definir los conceptos, componentes, responsabilidades y reglas de dependencia de COBRIA con terminología independiente de lenguajes y proveedores.
- OE2.** Formalizar entidades canónicas, mapeos semántico–físicos, proyecciones, contratos de reconstrucción, alcance multiinquilino y linaje analítico.
- OE3.** Clasificar las proyecciones en críticas, operacionales, analíticas y efímeras, y asociar a cada clase requisitos de consistencia y recuperación.
- OE4.** Analizar la complejidad temporal y espacial de lectura, escritura, hidratación, mantenimiento y reconstrucción.

- OE5.** Cuantificar latencia, transferencia de datos y ampliación de lectura, escritura y almacenamiento frente a alternativas sin proyecciones explícitas.
- OE6.** Evaluar la exactitud y el costo de reconstrucciones totales e incrementales, incluida la detección de referencias huérfanas y proyecciones faltantes.
- OE7.** Medir el efecto de metadatos compartidos sobre la evolución del esquema, la consistencia entre artefactos y el esfuerzo de mantenimiento.
- OE8.** Evaluar el aislamiento multiinquilino mediante rutas acotadas, reglas de acceso y pruebas negativas con datos sintéticos.
- OE9.** Analizar la capacidad de COBRIA para producir conjuntos de datos, series temporales y características versionadas con linaje y corrección temporal.
- OE10.** Examinar la integración segura de modelos y agentes de IA mediante casos de uso autorizados, evidencia, consentimiento, auditoría y retroalimentación humana.
- OE11.** Validar la aplicabilidad del artefacto en dos casos de estudio anónimos de diferente escala y naturaleza operacional.
- OE12.** Identificar límites, antipatronos y condiciones bajo las cuales COBRIA resultaría innecesaria o excesivamente compleja.

1.8. Preguntas de investigación

Cuadro 1.1: Preguntas de investigación de COBRIA.

Código	Pregunta
RQ1	¿Cómo afecta COBRIA la complejidad y el comportamiento observado de las operaciones de lectura, escritura y reconstrucción?
RQ2	¿En qué medida las proyecciones reducen la latencia, los registros procesados y los bytes transferidos en consultas frecuentes?
RQ3	¿Qué amplificación de escritura y almacenamiento introduce el mantenimiento de proyecciones y cuándo se justifica ese costo?
RQ4	¿Con qué exactitud, tiempo y consumo de recursos pueden reconstruirse las proyecciones a partir de entidades canónicas?
RQ5	¿Cómo influye un mapeo semántico–físico dirigido por metadatos en la evolución del esquema y en la consistencia entre modelos, validadores, reglas e índices?
RQ6	¿En qué medida el ámbito obligatorio mejora el aislamiento y limita las consultas en un sistema multiinquilino?
RQ7	¿Puede la misma arquitectura adaptarse a un sistema orientado a eventos y multimedia y a una plataforma empresarial con un dominio mucho más extenso?
RQ8	¿En qué medida las proyecciones analíticas versionadas mejoran la reproducibilidad, el linaje y la corrección temporal de conjuntos de datos y características?
RQ9	¿Qué mecanismos arquitectónicos son necesarios para que modelos y agentes de IA utilicen datos autorizados, auditables y acompañados de evidencia?
RQ10	¿Cuáles son los límites prácticos y las señales de sobrearquitectura de COBRIA?

Las preguntas combinan propiedades técnicas y de ingeniería. RQ1–RQ4 se responderán principalmente mediante mediciones; RQ5 y RQ6 combinarán análisis de artefactos con pruebas; RQ7 utilizará comparación entre casos; RQ8 y RQ9 abordarán preparación y gobernanza de IA; y RQ10 sintetizará resultados negativos y costos de adopción.

1.9. Hipótesis

Cuadro 1.2: Hipótesis y evidencia prevista.

Código	Hipótesis	Indicadores principales
H1	Las consultas apoyadas en proyecciones procesan una cantidad de datos más cercana al número de resultados que al tamaño total de la colección.	Registros leídos, bytes, latencia p50/p95/p99 y amplificación de lectura.
H2	La reducción del costo de lectura se obtiene a cambio de mayor amplificación de escritura y almacenamiento.	Escrituras físicas por operación lógica, tamaño de canónicos y derivados, duración de escritura.
H3	Un contrato explícito permite reconstruir proyecciones deterministas con alta exactitud a partir del estado canónico.	Cobertura de proyección, referencias huérfanas, igualdad esperada–observada y tiempo de reconstrucción.
H4	La reconstrucción incremental resulta más eficiente que la total cuando el número de entidades modificadas es pequeño respecto al conjunto canónico.	Tiempo, lecturas, escrituras y memoria para ΔN frente a N .
H5	La definición compartida de metadatos reduce discrepancias entre modelo lógico, representación física, validación, reglas e índices.	Defectos detectados, artefactos modificados manualmente, tiempo de evolución y cobertura de contrato.
H6	La incorporación obligatoria del organización en rutas, consultas y autorización reduce la superficie de exposición cruzada.	Pruebas negativas, consultas fuera de ámbito, registros innecesarios recuperados y violaciones detectadas.

Código	Hipótesis	Indicadores principales
H7	Las proyecciones analíticas versionadas mejoran la reproducibilidad y el linaje de conjuntos de datos frente a exportaciones ad hoc.	Tiempo de preparación, repetibilidad, porcentaje de campos con procedencia y diferencias entre ejecuciones.
H8	COBRIA puede aplicarse a los dos casos anónimos sin cambiar sus principios centrales, aunque requiera proyecciones y políticas diferentes.	Correspondencia de componentes, adaptaciones necesarias y resultados por caso.
H9	El beneficio neto disminuye cuando una entidad tiene pocos registros, pocas consultas o demasiadas proyecciones respecto a su valor funcional.	Función de costo, presupuesto de proyección y análisis de sensibilidad.

Las hipótesis son deliberadamente refutables. H2 y H9 reconocen costos y evitan evaluar la arquitectura únicamente con métricas favorables. La investigación considerará valioso identificar escenarios donde COBRIA no produzca una mejora neta.

1.10. Contribuciones esperadas

1.10.1. Contribución conceptual

Una definición neutral de COBRIA y un vocabulario que distinga entidad canónica, representación física, proyección, índice, resumen, conjunto de datos y artefacto efímero. Esta distinción busca facilitar la discusión entre desarrollo operacional, ingeniería de datos y aprendizaje automático.

1.10.2. Contribución formal

Un modelo de entidades, ámbitos, mapeos y proyecciones; propiedades de autoridad, reconstruibilidad, idempotencia y linaje; y condiciones bajo las cuales una proyección puede considerarse verificable.

1.10.3. Contribución arquitectónica

Una integración reproducible de modelos de dominio, Repositorio/Mapeador de datos, Servicio Capa, casos de uso, metadatos, proyecciones reconstruibles y componentes de analítica e IA. El aporte arquitectónico incluye reglas de dependencia y una clasificación de proyecciones por consistencia.

1.10.4. Contribución algorítmica

Pseudocódigo y posteriormente implementaciones de referencia para sincronización, reconstrucción total e incremental, detección de inconsistencias, migración, generación de conjuntos de datos y verificación de linaje.

1.10.5. Contribución empírica

Un conjunto de experimentos sobre dos casos anónimos que mida tanto beneficios como costos. Se publicarán configuraciones y datos sintéticos suficientes para permitir la replicación sin revelar información privada.

1.10.6. Contribución práctica

Una guía de adopción, un modelo de madurez, un presupuesto de proyecciones y criterios para decidir cuándo utilizar COBRIA, cuándo simplificarla y cuándo preferir otra organización documental.

1.11. Casos de estudio y neutralidad del análisis

La investigación se apoya en dos implementaciones existentes que comparten una familia de decisiones arquitectónicas, pero difieren en dominio, escala y comportamiento. Para evitar que el estudio se convierta en documentación de productos particulares, se adoptan descripciones neutrales.

Cuadro 1.3: Caracterización inicial y anónima de los casos.

Dimensión	Sistema A	Sistema B
Naturaleza	Orientado a eventos, relaciones contextuales y artefactos multimodales.	Plataforma empresarial multiinquilino con dominio extenso y operaciones transaccionales.
Escala estructural	Decenas de entidades y servicios especializados.	Cientos de entidades, catálogos, schemas y proyecciones.
Datos destacados	Eventos temporales, secuencias, texto, métricas y multimedia.	Entidades operacionales, financieras, configurables, de auditoría e integración.
Procesamiento	Servicios servicios internos, colas, procesos de trabajo y generación de artefactos.	Operaciones autorizadas, índices, procesos de trabajo, políticas y generación por metadatos.
Valor para el estudio	Adaptabilidad multimodal, temporal, analítica y de IA.	Escalabilidad estructural, multiinquilinato, gobierno del esquema y reconstrucción.

Los conteos exactos se fijarán en el protocolo experimental mediante scripts reproducibles y una fecha de corte. No se utilizará el tamaño del código como sustituto de calidad o completitud. Cuando una capacidad solo esté modelada o documentada, pero no verificada mediante pruebas, se declarará como tal.

1.12. Alcance

El estudio incluye:

- sistemas NoSQL cuyo modelo de aplicación pueda representarse mediante documentos, árboles JSON o estructuras clave–valor jerárquicas;
- entidades canónicas y representaciones derivadas para consulta;
- repositorios, servicios y casos de uso como fronteras de responsabilidad;
- aislamiento lógico multiinquilino;
- mantenimiento y reconstrucción de índices y proyecciones;

- metadatos para modelos, esquemas, reglas, validación y documentación;
- complejidad computacional y mediciones de rendimiento;
- proyecciones analíticas, series temporales, características y linaje;
- integración gobernada de modelos y agentes de IA;
- dos casos de estudio anonimizados y una implementación neutral de referencia.

1.13. Delimitaciones

El estudio no pretende:

- demostrar que un motor NoSQL específico es superior a todos los demás;
- reemplazar los patrones Repositorio, CQRS, vistas materializadas, Almacén de características o arquitectura por capas;
- evaluar todos los motores NoSQL disponibles;
- probar que una cantidad elevada de entidades o archivos implica mayor madurez;
- presentar una implementación de IA como evidencia suficiente de exactitud, equidad o seguridad;
- entrenar modelos con información personal o sensible;
- revelar nombres, reglas comerciales, credenciales o datos de los sistemas utilizados como casos;
- afirmar causalidad más allá de lo que permita el diseño experimental.

Los experimentos iniciales se realizarán en entornos controlados y con datos sintéticos. Las diferencias entre emuladores y producción se tratarán como amenaza a la validez externa. Las integraciones externas no se considerarán funcionales únicamente porque existan modelos o contratos en el código.

1.14. Unidad de análisis y niveles de observación

La unidad principal de análisis es una *operación de datos COBRIA*: una acción que lee o modifica una entidad canónica y, cuando corresponde, consulta o mantiene sus proyecciones. El estudio observará cuatro niveles:

1. **Nivel de entidad:** identidad, campos, versión, organización e invariantes.
2. **Nivel de operación:** lecturas, escrituras, transacciones, reintentos y efectos sobre proyecciones.
3. **Nivel de sistema:** latencia, transferencia, almacenamiento, concurrencia, aislamiento y recuperación.
4. **Nivel analítico:** conjunto de datos, característica, modelo, predicción, evidencia y retroalimentación.

Esta separación evita mezclar conclusiones. Por ejemplo, que un acceso a caché tenga costo promedio constante no implica que la operación distribuida completa sea instantánea; y que un conjunto de datos sea reconstruible no implica que sea adecuado o ético para entrenar un modelo.

1.15. Consideraciones éticas iniciales

La anonimización de los sistemas no es suficiente si los datos conservan combinaciones que permitan reidentificar personas. Por ello, los experimentos utilizarán datos sintéticos y no exportarán información de salud, identidad, pagos, conversaciones, ubicaciones precisas ni contenido multimedia privado. Las pruebas de aislamiento se realizarán únicamente en entornos autorizados.

En los componentes de IA se aplicarán, desde el diseño, los principios de trazabilidad, supervisión humana, control de acceso y evaluación de riesgo. Marcos oficiales como el marco de gestión de riesgos de inteligencia artificial de NIST enfatizan que la gestión del riesgo debe acompañar el ciclo de vida completo del sistema, no limitarse al modelo entrenado (National Institute of Standards and Technology, 2023). COBRIA no considerará una respuesta generada como una fuente canónica de negocio mientras no pase por el caso de uso y las confirmaciones requeridas.

1.16. Organización del documento maestro

Después de esta introducción, el documento se organizará en siete bloques:

1. **Fundamentos y literatura:** conceptos necesarios y comparación con trabajos relacionados.
2. **Diseño de COBRIA:** componentes, modelo formal, algoritmos, consistencia y reconstrucción.
3. **Rendimiento:** complejidad, amplificación, modelo de costos y optimización.
4. **Analítica e IA:** minería, Almacén de características, multimodalidad, RAG, agentes y gobernanza.
5. **Implementación y casos:** implementación neutral y caracterización anónima de los dos sistemas.
6. **Evaluación:** protocolo, resultados, análisis estadístico, discusión y amenazas a la validez.
7. **Síntesis:** guía de adopción, conclusiones y líneas futuras.

La estructura seguirá la lógica de la ciencia del diseño: el problema conduce a un artefacto; el artefacto se formaliza e implementa; y su utilidad se juzga mediante evaluación explícita, no por afirmación del autor (Hevner et al., 2004; Peffers et al., 2007).

1.17. Comparación sistemática con arquitecturas relacionadas

La revisión ampliada distingue coincidencia de mecanismos y coincidencia de objetivos. CQRS separa modelos de lectura y escritura, pero no obliga por sí mismo a declarar linaje, ámbito o reconstrucción. Las vistas materializadas anticipan consultas y pueden regenerarse desde sus fuentes, aunque su gobierno suele quedar dentro del motor (Microsoft, 2025a, 2025b). El abastecimiento por eventos conserva una secuencia autoritativa capaz de rehidratar estado y proyecciones, pero COBRIA permite que la autoridad sea una entidad actual versionada y no exige un registro completo de eventos (Fowler, 2005).

Los almacenes de características gobiernan definiciones para aprendizaje automático; los sistemas de linaje explican procedencia; y los entornos tipo *lakehouse* unifican operación analítica. COBRIA conecta esos mecanismos con la frontera de casos de uso, sin pretender

sustituírlos (Armbrust et al., 2021; Buneman et al., 2001; Li et al., 2023; Orr et al., 2021). COBRIA trata las semánticas particulares de persistencia, aislamiento e índices como capacidades explícitas de cada adaptador NoSQL, no como supuestos universales.

Cuadro 1.4: Comparación entre COBRIA y enfoques relacionados.

Enfoque	Autoridad	Reconstrucción	Gobierno y diferencia principal
CQRS	Parcial	Parcial	Separa lectura y escritura; ámbito y linaje no son inherentes.
Vista materializada	Externa	Sí	Optimiza consultas; el ámbito depende del motor y del diseño.
Abastecimiento por eventos	Eventos	Sí	El registro de eventos es autoridad y aporta historia.
Almacén de características	Fuentes analíticas	Parcial	Gobierna variables de modelos y su procedencia.
Arquitectura tipo <i>lakehouse</i>	Archivos o tablas	Parcial	Unifica almacenamiento y procesamiento analítico.
COBRIA	Declarada	Obligatoria	Exige ámbito, linaje y gobierno desde operación hasta IA.

La matriz no establece superioridad. Muestra que la contribución propuesta reside en hacer simultáneamente verificables autoridad, ámbito, reconstrucción, costo y continuidad hacia inteligencia artificial. Si una solución existente satisface esos contratos con menor complejidad, COBRIA recomienda conservar la solución más sencilla.

1.18. Síntesis de la Parte I

Esta primera parte estableció el problema que COBRIA pretende investigar. Los sistemas NoSQL multiinquilino necesitan optimizar consultas sin perder autoridad, consistencia ni capacidad de recuperación. Las necesidades analíticas añaden el requisito de conservar linaje, versiones y corrección temporal desde la operación hasta la característica y la predicción. COBRIA se propone como una integración de ideas existentes alrededor de dos conceptos organizadores: *datos canónicos* y *proyecciones reconstruibles*.

La investigación no presupone que dicha integración sea siempre beneficiosa. Las preguntas e hipótesis obligan a medir reducción de lectura junto con amplificación de escritura y almacenamiento, exactitud de reconstrucción junto con su costo, y reproducibilidad analítica

junto con complejidad de adopción. Esta posición crítica será la base del marco teórico, el modelo formal y la evaluación posterior.

Parte II

Diseño y formalización de COBRIA

Metodología para el diseño de COBRIA

2.1. Enfoque de ciencia del diseño

COBRIA se estudia como un artefacto de ingeniería y no únicamente como una descripción de prácticas observadas. La ciencia del diseño resulta apropiada cuando la investigación busca construir y evaluar un artefacto destinado a resolver un problema relevante. Ese artefacto puede adoptar la forma de constructos, modelos, métodos o instanciaciones (Hevner et al., 2004). COBRIA comprende las cuatro formas: propone un vocabulario, formaliza relaciones, prescribe un método de diseño y se instancia en sistemas ejecutables.

El proceso sigue seis actividades adaptadas de la metodología de Peffers et al. (2007):

1. identificar el problema de autoridad, duplicación, consulta y trazabilidad;
2. definir objetivos verificables para una solución;
3. diseñar y desarrollar los componentes de COBRIA;
4. demostrar su aplicación en dos casos anonimizados;
5. evaluar beneficios, costos y fallos mediante experimentos;
6. comunicar resultados, límites y material de replicación.

El desarrollo no es estrictamente lineal. Los hallazgos de evaluación pueden exigir reformular un contrato, retirar una proyección o reducir una abstracción. Este ciclo evita presentar como principio general una decisión que solo funciona en un caso.

2.2. Proceso de abstracción desde los casos

Los dos sistemas de estudio contienen implementaciones con diferentes grados de uniformidad. El Sistema A combina componentes heredados y especializados; el Sistema B utiliza metadatos y generación con mayor regularidad. COBRIA no copia literalmente ninguna de las dos estructuras. Su derivación sigue cuatro pasos:

1. **Inventario:** identificar entidades, accesos, servicios, operaciones, índices, catálogos y procesos asíncronos.
2. **Normalización conceptual:** traducir nombres internos a términos reconocibles, como Entidad, Repositorio, Mapeador de datos, Capa de servicios y Proyección.
3. **Comparación:** separar mecanismos comunes de decisiones exclusivas de un dominio o de deuda histórica.
4. **Generalización cautelosa:** conservar únicamente propiedades que puedan definirse, implementarse y posteriormente evaluarse.

La correspondencia terminológica inicial se muestra en la cuadro 2.1. El término interno *Dater*, útil para describir los casos, se sustituye en la propuesta por Repositorio/Mapeador de datos, una terminología más próxima a la literatura de arquitectura empresarial (Fowler, 2002).

Cuadro 2.1: Normalización conceptual de componentes observados.

Nombre observado	Término COBRIA	Responsabilidad principal
Objeto	Canónico Entidad	Representar identidad, estado semántico, versión y ciclo de vida.
Dater	Repositorio/Mapeador de datos	Traducir representaciones y encapsular persistencia y consulta.
Servicio	Dominio/Aplicación Servicio	Aplicar políticas, componer datos y coordinar capacidades.
CasoDeUso	Caso de uso de aplicación	Ejecutar una intención autorizada y completa.
Servicio indexador	Gestor de proyecciones	Mantener, verificar y reconstruir proyecciones.
thingsData	Registro de catálogos	Administrar vocabularios finitos y datos de referencia.
DataConnect	Pasarela de almacenamiento	Adaptar el motor físico y sus primitivas.

2.3. Criterios de diseño

Cada componente propuesto debe satisfacer cinco criterios:

Comprensibilidad

Su propósito debe poder explicarse sin conocer una ruta física.

Verificabilidad

Deben existir invariantes o resultados observables.

Reemplazabilidad

El dominio no debe depender innecesariamente de un proveedor.

Proporcionalidad

La complejidad añadida debe responder a una necesidad medible.

Trazabilidad

Debe conocerse el origen y la versión de toda derivación relevante.

Estos criterios impiden equiparar uniformidad con calidad. Una clase generada puede ser estructuralmente correcta y, aun así, carecer de comportamiento funcional. Del mismo modo, un componente especializado puede ser adecuado aunque no herede de una base genérica.

2.4. Evidencia prevista

La evaluación combinará cuatro clases de evidencia:

- **estructural:** dependencias, contratos, metadatos y cobertura;
- **funcional:** resultados de operaciones y pruebas de invariantes;
- **experimental:** latencia, bytes, escrituras, almacenamiento y tiempo de reconstrucción;
- **analítica:** linaje, repetibilidad, corrección temporal y calidad de los datos derivados.

La presencia de archivos o documentación se considerará evidencia estructural, no prueba de funcionamiento. Una capacidad solo se declarará validada si existe una ejecución observable bajo condiciones documentadas.

Arquitectura COBRIA

3.1. Definición

COBRIA es una arquitectura dirigida por metadatos que representa el estado autorizado mediante entidades canónicas, separa el modelo semántico de su representación física y produce proyecciones reconstruibles para operación, integración, minería de datos e inteligencia artificial dentro de un ámbito explícito y auditable.

La definición contiene cinco compromisos. Primero, existe una fuente de autoridad identificable. Segundo, almacenar no equivale a modelar: la representación física puede cambiar sin alterar el significado. Tercero, los datos derivados poseen contrato y no son copias accidentales. Cuarto, la pertenencia a un organización o contexto forma parte del diseño. Quinto, analítica e IA consumen proyecciones trazables en lugar de exportaciones opacas.

3.2. Capas y dirección de dependencias

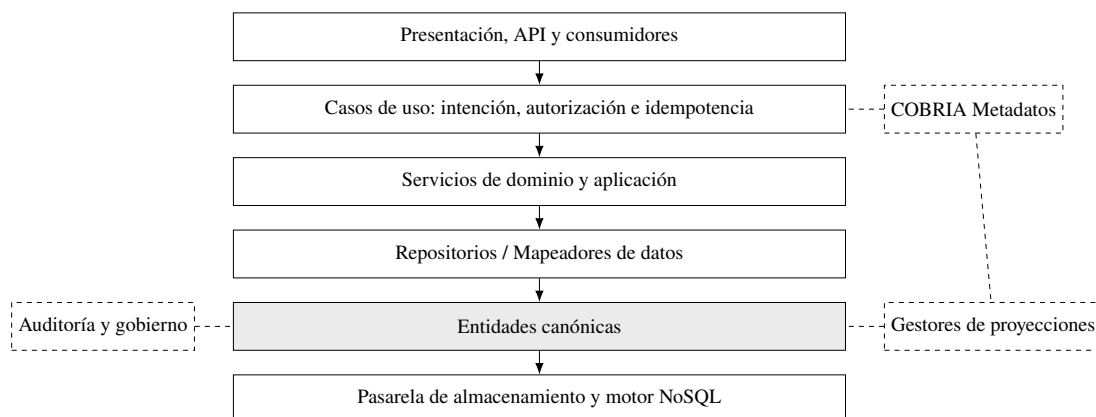


Figura 3.1: Capas y preocupaciones transversales de COBRIA.

La dirección descendente de la figura 3.1 representa conocimiento de abstracciones, no necesariamente el orden temporal de cada llamada. Un escucha del almacenamiento puede iniciar una notificación ascendente, pero su dato debe cruzar el Mapeador de datos antes de presentarse como entidad. La interfaz no debe construir rutas físicas ni actualizar proyecciones por cuenta propia.

3.3. Módulos internos

3.3.1. COBRIA Núcleo

Incluye entidades, repositorios, servicios, casos de uso, catálogos y contexto. Es el subconjunto mínimo. Un sistema pequeño puede adoptar Núcleo sin Analítica o Inteligencia.

3.3.2. COBRIA Metadatos

Contiene descripciones de campos, tipos, mapeos, ámbito, relaciones, sensibilidad, índices y ciclo de vida. Los metadatos pueden alimentar validadores y generadores, pero no sustituyen pruebas funcionales.

3.3.3. COBRIA Proyecciones

Define funciones de derivación, destinos, niveles de consistencia, verificadores y procedimientos de reconstrucción. La proyección puede residir en el mismo motor o en uno

diferente.

3.3.4. COBRIA Analítica

Transforma datos autorizados en eventos, agregaciones, series, grafos, conjuntos de datos y características versionadas. Su diseño debe preservar linaje y tiempo de observación.

3.3.5. COBRIA Inteligencia

Administra versiones de modelos, inferencias, evidencia, retroalimentación, consentimiento y agentes. Ninguna predicción adquiere por sí sola autoridad sobre el estado canónico.

3.3.6. COBRIA Gobierno

Reúne auditoría, retención, protección de datos, observabilidad, reconciliación y políticas de acceso. La gobernanza no se considera un módulo opcional cuando se manejan datos sensibles o decisiones asistidas por IA.

3.4. COBRIA frente a patrones relacionados

COBRIA puede usar separación de lectura y escritura, pero no exige dos bases ni mensajería. Por ello no es sinónimo de CQRS. CQRS permite optimizar modelos de lectura y escritura de forma independiente, aunque añade sincronización y consistencia eventual (Microsoft, 2025a). COBRIA adopta esa separación cuando existe una razón medible y añade el contrato de reconstrucción y el linaje analítico.

Tampoco exige abastecimiento por eventos. La fuente canónica puede ser el estado actual versionado, un historial de eventos o una combinación. En un sistema basado en estado, la auditoría no convierte automáticamente cada cambio en un evento capaz de reconstruir todo el agregado.

Las proyecciones COBRIA incluyen vistas materializadas, pero su alcance es mayor: pueden ser índices booleanos, relaciones inversas, resúmenes, artefactos multimedia o conjuntos de datos. Comparten la propiedad esencial de ser desechables y regenerables desde una fuente (Gupta & Mumick, 1995; Microsoft, 2025b).

Modelo formal de datos canónicos

4.1. Conjuntos fundamentales

Sea $\mathcal{E}$ el conjunto de entidades canónicas, $\mathcal{P}$ el conjunto de proyecciones, $\mathcal{S}$ el conjunto de ámbitos y $\mathcal{M}$ el conjunto de versiones de metadatos. Una entidad se define como:

$$E = (I, A, S, V, L, R), \quad (4.1)$$

donde I es la identidad, A los atributos semánticos, $S \in \mathcal{S}$ el ámbito, V la versión del esquema, L el estado de ciclo de vida y R la revisión de concurrencia.

La identidad global lógica puede expresarse como:

$$I_g = (\tau, S, I_l), \quad (4.2)$$

donde τ es el tipo de entidad e I_l el identificador local. Dos entidades con el mismo identificador local pero ámbitos distintos no son la misma entidad.

4.2. Atributos y ausencia de valor

Cada atributo $a_j \in A$ declara tipo, nulabilidad, valor inicial, sensibilidad y reglas de validación. COBRIA distingue entre ausencia, valor nulo y valor vacío cuando el dominio lo requiere. Convertir indiscriminadamente los tres estados puede destruir información necesaria para migraciones o analítica.

Los valores monetarios deben conservar moneda y unidad menor; las fechas de negocio no deben confundirse con instantes; y los identificadores externos deben diferenciarse de las claves internas. Estas decisiones pertenecen al modelo semántico, no al nombre corto utilizado en almacenamiento.

4.3. Ámbito y multiinquilinato

Un ámbito puede contener varios niveles:

$$S = (\text{organizacion}, \text{contexto}_1, \dots, \text{contexto}_q). \quad (4.3)$$

Para toda operación autorizada o realizada por un actor u sobre una entidad E , se exige:

$$\text{autorizado}(u, o, E) \Rightarrow \text{ambito}(E) \in \text{ambitos}(u, o). \quad (4.4)$$

La condición de la ecuación (4.4) debe verificarse en una frontera confiable. Ocultar una entidad en la interfaz no constituye aislamiento. La ruta física puede incluir el ámbito para acotar consultas, pero la seguridad no debe depender únicamente de que el cliente construya correctamente esa ruta.

4.4. Ciclo de vida

El estado L se modela como:

$$L \in \{\text{active}, \text{archived}, \text{deleted}\}, \quad (4.5)$$

con extensiones específicas del dominio. *Eliminado* puede representar una marca lógica cuando existen obligaciones de auditoría. Una política de retención define si y cuándo procede la eliminación física. Las proyecciones deben excluir o representar entidades archivadas de acuerdo con su contrato, no según una convención implícita.

4.5. Revisión y concurrencia

La revisión $R \in \mathbb{N}_0$ aumenta con cada modificación aceptada. Una actualización optimista exige:

$$R_{\text{request}} = R_{\text{stored}}. \quad (4.6)$$

Si la igualdad no se cumple, el caso de uso rechaza, reintenta desde un estado nuevo o ejecuta una reconciliación explícita. El control de revisión no reemplaza transacciones atómicas cuando una invariante involucra varias ubicaciones.

4.6. Invariantes canónicas

Se proponen cinco invariantes generales:

$$\forall E \in \mathcal{E} : I(E) \neq \emptyset, \quad (4.7)$$

$$\forall E \in \mathcal{E} : S(E) \neq \emptyset \quad \text{si el tipo es ámbitod}, \quad (4.8)$$

$$\forall E_1, E_2 : I_g(E_1) = I_g(E_2) \Rightarrow E_1 = E_2, \quad (4.9)$$

$$\forall E : \text{deserialize}(\text{serialize}(E, M), M) = E, \quad (4.10)$$

$$\forall p \in \mathcal{P} : \text{source}(p) \subseteq \mathcal{E} \cup \mathcal{P}. \quad (4.11)$$

La cuarta igualdad se entiende sobre campos persistidos y admite normalizaciones declaradas. Un timestamp generado por servidor, por ejemplo, debe modelarse como tal en vez de ocultarse como una diferencia accidental.

Representación física y persistencia

5.1. Mapeo semántico–físico

Sea M_v un mapeo versionado y F una función de serialización:

$$F_{M_v} : \mathcal{E} \rightarrow \mathcal{R}, \quad (5.1)$$

donde $\mathcal{R}$ es el conjunto de representaciones físicas. La operación inversa G_{M_v} debe satisfacer:

$$G_{M_v}(F_{M_v}(E)) \equiv E. \quad (5.2)$$

La equivalencia de la ecuación (5.2) incluye identidad y semántica, no propiedades transitorias de memoria. Un ejemplo neutral es:

Cuadro 5.1: Ejemplo de mapeo semántico–físico versionado.

Campo semántico	Clave física	Regla
organizaciónId	a	cadena no vacía
contextId	b	cadena no vacía
status	c	código de catálogo versionado
createdAt	d	instante UTC
revision	e	entero no negativo

Las claves compactas pueden reducir repetición, pero disminuyen legibilidad y elevan el costo de diagnóstico. COBRIA no las prescribe; exige medir su beneficio y conservar un diccionario versionado. Una clave física nunca debe reutilizarse con otro significado dentro de la misma versión.

5.2. Repositorio y Mapeador de datos

El Repositorio ofrece una colección conceptual de entidades; el Mapeador de datos traduce entre objeto y almacenamiento sin introducir dependencias físicas en el dominio (Fowler, 2002). En una implementación compacta ambas responsabilidades pueden residir en una clase, siempre que su interfaz permanezca explícita.

El contrato mínimo es:

```
get(id, ámbito) -> Entidad | null
list(query, ámbito, page) -> Page<Entidad>
save(entity, expectedRevision) -> Entidad
remove(id, ámbito, policy) -> Result
subscribe(id, ámbito, observer) -> Unsubscribe
```

El Repositorio no decide por sí solo una transición de negocio. Puede validar forma y ámbito, pero una operación como aprobar, cancelar o transferir pertenece al caso de uso.

5.3. Mapa de identidad y caché

Una caché $C : I_g \rightarrow E$ evita cargar repetidamente la misma identidad durante una unidad de trabajo. Su acceso promedio puede aproximarse a $O(1)$ mediante tabla hash, pero introduce expiración, invalidación y riesgo de servir datos obsoletos. COBRIA exige declarar el alcance de la caché:

- por operación, con vida corta y consistencia sencilla;
- por sesión, con invalidación mediante eventos o TTL;
- compartida, con políticas explícitas de coherencia y aislamiento.

Una caché nunca debe omitir la dimensión de organización en su clave.

5.4. Pasarela de almacenamiento

El Pasarela adapta primitivas como lectura, transacción, actualización multipath, escucha, paginación y generación de identidad. Esta frontera permite contrastar el modelo en emuladores y sustituir el motor, pero no promete portabilidad automática: las garantías varían entre almacenes.

Sistemas como Dynamo priorizan disponibilidad y resolución de versiones (DeCandia et al., 2007), mientras Spanner demuestra un modelo distribuido con transacciones externamente consistentes bajo otras decisiones de infraestructura (Corbett et al., 2012).

5.5. Registro de rutas semánticas

Cuando el almacenamiento utiliza nombres físicos compactos, el desacoplamiento no se logra solo con un Mapeador. También se necesita un registro central que traduzca nombres semánticos a nodos y construya rutas con ámbito. Sea $\rho_v : S \times K^* \rightarrow Path$ la función versionada que recibe un símbolo semántico S y segmentos de clave K . Los servicios de aplicación dependen de ρ_v , no de literales físicos dispersos.

Este registro aporta cuatro propiedades comprobables:

1. una ruta física cambia en un punto controlado;
2. una búsqueda estática puede prohibir literales fuera de infraestructura;
3. las pruebas verifican normalización y presencia obligatoria de ámbito;
4. telemetría, reglas y migraciones comparten el mismo vocabulario.

El registro no convierte automáticamente un esquema compacto en modelo canónico. Su función es establecer una frontera semántico-física y hacer localizable la deuda.

5.6. Capa de servicios y casos de uso

El Capa de servicios establece capacidades y coordina la respuesta de la aplicación (Fowler, 2002). COBRIA distingue:

- **servicio de dominio:** regla que no pertenece naturalmente a una sola entidad;
- **servicio de aplicación:** composición técnica de repositorios, políticas y proveedores;
- **caso de uso:** intención completa iniciada por un actor o proceso.

Un caso de uso recibe comando, actor, ámbito, clave de idempotencia y revisión esperada. Produce un resultado funcional, una auditoría y los cambios derivados requeridos. La interfaz no debe convertir un cambio de estado complejo en un CRUD genérico.

5.7. Contexto autorizado como dato derivado confiable

El ámbito solicitado por cabeceras, query string o cuerpo identifica una intención, no demuestra autorización. COBRIA distingue $S_{solicitado}$ de $S_{autorizado}$. El segundo debe derivarse en servidor a partir de identidad, declaraciones y relación comprobada con el recurso:

$$S_{autorizado} = \text{autorizar}(\text{actor}, \text{recurso}, \text{politica}).$$

Servicios, auditoría, facturación, FinOps y proyecciones posteriores consumen únicamente el contexto autorizado. Recalcularlo de manera distinta en cada capa crea divergencia; aceptarlo del cliente permite atribución falsa. Por ello, el contexto validado es una proyección efímera de seguridad: tiene fuente, política, alcance y vida limitada, pero no reemplaza la autorización de cada operación sensible.

Teoría de proyecciones reconstruibles

6.1. Definición

Una proyección P_i se define mediante una función versionada:

$$P_i = g_{i,v}(D_i), \quad (6.1)$$

donde D_i es un conjunto de entidades canónicas o proyecciones previamente autorizadas. Para evitar ciclos no controlados, el grafo de dependencias de reconstrucción debe ser acíclico o declarar un punto fijo y un algoritmo de convergencia.

Una copia es proyección COBRIA solo si declara:

1. fuentes y ámbito;
2. función y versión;
3. destino y clave;
4. propietario de escritura;
5. nivel de consistencia;
6. procedimiento de reconstrucción;
7. verificador de integridad;
8. política de privacidad y retención.

6.2. Taxonomía

Cuadro 6.1: Taxonomía de proyecciones COBRIA.

Clase	Consistencia esperada	Ejemplos y tratamiento
Crítica	Atómica con la operación o verificada antes de confirmar	Reserva de recurso, saldo operativo, unicidad. Si no puede mantenerse, falla el caso de uso.
Operacional	Eventual acotada	Índices de consulta, resúmenes de lista, relaciones inversas. Debe existir reconciliación.
Analítica	Eventual y versionada	Series, características, conjuntos de datos, agregados históricos. Prioriza linaje y corrección temporal.
Efímera	Sin garantía durable	Caché, vista previa o cálculo temporal. Puede descartarse sin reconstrucción persistente.

Clasificar no convierte una proyección en correcta. La clase define qué retrasos y fallos son aceptables y qué pruebas se requieren.

6.3. Propiedades

6.3.1. Determinismo

Con fuentes y versión iguales, una proyección determinista produce el mismo resultado:

$$D_i^{(1)} = D_i^{(2)} \Rightarrow g_{i,v}(D_i^{(1)}) = g_{i,v}(D_i^{(2)}). \quad (6.2)$$

Si utiliza tiempo actual, aleatoriedad o un proveedor externo, esos valores deben capturarse como entradas versionadas.

6.3.2. Idempotencia

Aplicar dos veces el mismo cambio no debe duplicar resultados:

$$apply(apply(P, c), c) = apply(P, c). \quad (6.3)$$

6.3.3. Reconstructibilidad

Si P_i se pierde, debe poder calcularse P'_i tal que:

$$\text{verify}(P'_i, g_{i,v}(D_i)) = \text{true}. \quad (6.4)$$

6.3.4. No autoridad

Una proyección no se edita para corregir negocio. La corrección se ejecuta sobre el canónico mediante un caso de uso y luego se propaga.

6.4. Grafo de dependencias

Sea $G = (V, E_d)$ un grafo donde $V = \mathcal{E} \cup \mathcal{P}$ y una arista (x, p) indica que p depende de x . El orden de reconstrucción se obtiene mediante orden topológico. Si se detecta un ciclo no declarado, el plan se rechaza.

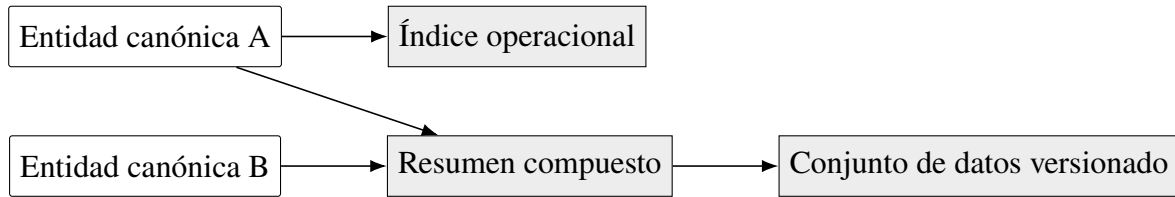


Figura 6.1: Ejemplo de grafo acíclico de reconstrucción.

6.5. Contrato de proyección

Una representación neutral puede adoptar la siguiente forma:

```

ProjectionContract {
  name, version, class,
  sources[], ámbito,
  destination, keyFunction,
  build, verify,
  consistency, responsable,
  privacy, retention
}
  
```

El contrato es declarativo respecto al propósito, aunque ‘build’ y ‘verify’ pueden ser funciones. Debe poder responder quién escribe la proyección y cómo se repara. Si esas preguntas no tienen respuesta, la duplicación permanece fuera de gobierno.

Algoritmos, consistencia y recuperación

7.1. Creación de una entidad

```
algorithm Create(command, actor, idempotencyKey):
  require authorize(actor, command.ámbito, command.action)
  if resultExists(idempotencyKey): return previousResult
  metadata <- registry.forType(command.type)
  entity <- metadata.construct(command.carga útil)
  validate(entity)
  critical <- projections.criticalFor(entity.type)
  atomicWrite(entity, buildAll(critical, entity))
  enqueue(projections.nonCriticalFor(entity.type), entity.revision)
  audit(actor, command, entity)
  return remember(idempotencyKey, entity)
```

Las proyecciones críticas forman parte de la confirmación. Las demás se envían a un proceso idempotente. Si el motor no ofrece una escritura atómica suficiente, el diseño debe reducir la frontera crítica o implementar una reserva y compensación explícitas.

7.2. Actualización optimista

```
algorithm Update(command, actor, expectedRevision):
  current <- repository.get(command.id, command.ámbito)
  require current.revision = expectedRevision
  next <- useCase.transition(current, command)
  next.revision <- current.revision + 1
```

```
validateTransition(current, next)
atomicWrite(next, criticalDelta(current, next))
enqueue(nonCriticalDelta(current, next))
return next
```

El delta debe eliminar claves antiguas además de añadir nuevas. Cambiar el estado o el ámbito sin retirar índices previos produce referencias fantasma.

7.3. Reconstrucción total

```
algorithm RebuildAll(contract, ámbito):
  acquireLease(contract.name, ámbito)
  targetVersion <- createShadowDestination()
  for page in sourcePages(contract.sources, ámbito):
    records <- contract.build(page)
    writeIdempotently(targetVersion, records)
    checkpoint(page.cursor)
  report <- contract.verify(targetVersion)
  require report.errors = 0
  atomicCutover(targetVersion)
  releaseLease()
  return report
```

El destino sombra evita dejar una proyección parcialmente reconstruida como si estuviera completa. Para volúmenes pequeños puede reconstruirse en sitio, pero debe exponerse el estado ‘rebuilding’ y tolerar reanudación.

7.4. Reconstrucción incremental

```
algorithm RebuildChanged(contract, marca de corte):
  changes <- canonicalChangesAfter(marca de corte)
  for change in ordered(changes):
    require notProcessed(change.id, contract.version)
    delta <- contract.buildDelta(change.before, change.after)
    applyIdempotently(delta)
    markProcessed(change.id, contract.version)
```

```
return contract.verifyAffected(changes)
```

La reconstrucción incremental necesita un registro confiable de cambios o una marca de actualización consultable. Si no existe, escanear el conjunto canónico puede ser la única forma correcta de detectar diferencias.

7.5. Verificación de integridad

Se definen cuatro clases de anomalía:

$$orphan = \{p \in P : sourceId(p) \notin I(\mathcal{E})\}, \quad (7.1)$$

$$missing = \{e \in E : expected(e) \setminus actual(e) \neq \emptyset\}, \quad (7.2)$$

$$stale = \{p \in P : sourceRevision(p) < revision(source(p))\}, \quad (7.3)$$

$$divergent = \{p \in P : p \neq g_v(source(p))\}. \quad (7.4)$$

Un reporte debe separar conteos, ejemplos anonimizados y severidad. La reparación automática es apropiada para derivados deterministas; una divergencia canónica requiere revisión de negocio.

7.6. Consistencia por clase

Cuadro 7.1: Mecanismos de consistencia recomendados.

Necesidad	Mecanismo	Consideración
Invariante local	Transacción o compare-and-set	Rechazar revisiones obsoletas.
Varias rutas del mismo motor	Actualización multipath atómica	Validar límites reales del proveedor.
Derivado operacional	Evento/cola de salida y proceso de trabajo idempotente	Medir retraso y reconciliar.
Integración externa	Saga o compensación	No asumir transacción distribuida.
Conjunto de datos analítico	Lote versionado con marca de corte	Preservar tiempo y linaje.

La consistencia fuerte global no debe afirmarse por analogía. Las garantías dependen del motor, partición y frontera transaccional. COBRIA documenta esas garantías en el contrato del caso de uso.

7.7. Estado materializado: completitud, frescura y procedencia

Una vista no debe considerarse válida por el solo hecho de existir. El contrato mínimo incluye metadatos de control:

```
_meta:
  complete: boolean
  updatedAt: timestamp
  schemaVersion: integer
  sourceRevision|marca de corte: value
  count|checksum: optional
```

La lectura acepta la vista únicamente si está completa, dentro de su presupuesto de frescura y en una versión compatible. Una mutación puede actualizarla, invalidarla o marcarla incompleta. Si la proyección es mejor esfuerzo, su fallo no revierte la escritura canónica; la lectura cae a la fuente autorizada y programa reparación.

Cuando varias solicitudes encuentran una vista fría, la reconstrucción debe *coalescerse*: una promesa, arrendamiento o bloqueo por ámbito evita que todas repitan el mismo escaneo. Esta protección contra *cache stampede* forma parte del contrato de reconstrucción y debe probarse con solicitudes concurrentes.

7.8. Arquitectura ejecutable y funciones de aptitud

Una regla escrita puede degradarse silenciosamente. COBRIA incorpora funciones de aptitud arquitectónica: pruebas o análisis estáticos que convierten una propiedad estructural en una condición de integración continua. Ejemplos:

- presentación no importa infraestructura de datos;
- rutas físicas compactas no aparecen en servicios de aplicación;
- todo modelo de lectura declara metadatos de completitud y frescura;
- una operación sensible usa contexto autorizado por servidor;
- el número de consumidores de una fachada heredado no puede crecer;
- rutas críticas respetan presupuestos de bytes, LCP y bloqueo.

Para una deuda medible d_t , el control automatizado de migración impone $d_{t+1} \leq d_t$, y el objetivo final puede fijarse en cero. Este presupuesto monótono permite migrar sin una reescritura total y evita que una fachada temporal se convierta en arquitectura permanente.

7.9. Cola de salida, reintentos y duplicados

Cuando guardar el canónico y publicar un mensaje no forman una transacción única, existe una ventana de pérdida. El patrón cola de salida conserva la intención de publicación junto al cambio autorizado; un proceso de trabajo la entrega y registra el resultado. La entrega puede ser al menos una vez, por lo que consumidores y Gestores de proyecciones deben ser idempotentes.

Una clave de idempotencia se asocia con actor, ámbito, operación y ventana temporal. No debe reutilizarse entre organizaciones. El resultado previo puede devolverse siempre que los parámetros relevantes coincidan; en caso contrario, la reutilización se rechaza.

7.10. Recuperación y observabilidad

Cada proceso de reconstrucción registra:

- contrato y versión;
- ámbito y rango temporal;
- cursor o checkpoint;
- entidades leídas y registros escritos;
- anomalías antes y después;
- duración y consumo aproximado;
- identidad del operador o proceso de trabajo;
- resultado del cambio de versión.

El objetivo de recuperación (RTO) y el punto de recuperación (RPO) deben definirse por clase de proyección. Una caché puede perderse sin RPO; un saldo derivado utilizado en una operación exige un tratamiento diferente.

7.11. Antipatrones

COBRIA identifica como antipatrones:

1. editar una proyección para corregir la fuente;
2. construir rutas físicas en la presentación;
3. mantener el mismo índice desde múltiples servicios sin propietario;
4. usar un ID sin ámbito como clave de caché multiinquilino;
5. generar una clase por cada estructura sin justificar identidad o ciclo de vida;

6. llamar transacción a una secuencia de escrituras independientes;
7. entrenar con una exportación sin versión, marca de corte ni linaje;
8. permitir que un agente de IA escriba directamente en el almacenamiento;
9. declarar una integración completa por la sola presencia de modelos o contratos;
10. añadir proyecciones sin medir amplificación y frecuencia de consulta;
11. confiar en el ámbito enviado por el cliente para auditoría o autorización;
12. servir una vista existente sin comprobar completitud, versión y frescura;
13. crear una capa de compatibilidad sin inventario ni presupuesto decreciente;
14. documentar límites de capas sin un control automatizado que detecte regresiones.

7.12. Criterio mínimo de adopción

COBRIA Núcleo puede justificarse cuando existe al menos una de estas condiciones: dominio con reglas relevantes, multiinquilinato, duplicación derivada, necesidad de reconstrucción, procesamiento asíncrono o uso analítico trazable. Si el sistema es un CRUD pequeño, con pocos registros y consultas directas, Repositorio más un modelo sencillo puede ser suficiente. La arquitectura se considera exitosa cuando hace explícitos los compromisos, incluso si la decisión resultante es no crear una proyección.

7.13. Síntesis de la Parte II

Esta parte transformó la intuición inicial en un artefacto definido. COBRIA organiza el estado mediante entidades con identidad, ámbito, versión, ciclo de vida y revisión; separa semántica de persistencia mediante un mapeo reversible; y asigna a Repositorio, Servicio y Caso de uso fronteras distintas. Su elemento diferenciador es el contrato de proyección, que convierte índices, resúmenes y conjuntos de datos en derivaciones gobernadas y reconstruibles.

También se establecieron algoritmos de creación, actualización, reconstrucción y verificación, junto con mecanismos de consistencia proporcionales a la criticidad. Estas definiciones preparan la siguiente parte del estudio: analizar complejidad, amplificación y costo, y comprobar mediante experimentos si la disciplina propuesta produce un beneficio neto.

Parte III

Rendimiento, complejidad y optimización

Modelo de rendimiento y notación

La utilidad de una arquitectura de datos no puede deducirse únicamente de la claridad de sus clases o diagramas. Una separación correcta de responsabilidades puede reducir errores y, al mismo tiempo, introducir lecturas, escrituras, almacenamiento y tareas de mantenimiento adicionales. Esta parte establece un modelo para razonar sobre esos compromisos antes de medirlos y un protocolo para comprobarlos sin atribuir a COBRIA resultados que dependan del motor, la red o una configuración particular.

El análisis distingue dos preguntas. La primera es asintótica: cómo crece el trabajo cuando aumenta el volumen de datos. La segunda es empírica: cuánto tarda y cuánto cuesta una operación bajo una carga y una plataforma concretas. La notación $O(\cdot)$ responde a la primera pregunta; no representa milisegundos, precio monetario ni garantía de rendimiento. Dos operaciones de orden $O(1)$ pueden tener costos reales muy distintos si una requiere una lectura local y otra varias comunicaciones de red (Cormen et al., 2022).

8.1. Variables del modelo

Sea N el número de entidades canónicas dentro de un ámbito; R el número de resultados de una consulta; P el número de proyecciones asociadas con una entidad; $P_w \leq P$ el número de proyecciones afectadas por una escritura; ΔN el número de entidades modificadas desde un marca de corte; B_c el tamaño medio de un canónico y B_{p_i} el tamaño medio de su contribución a la proyección i . Se utilizan además:

Cuadro 8.1: Símbolos del modelo de rendimiento.

Símbolo	Significado	Símbolo	Significado
L	latencia observada	Q	operaciones por unidad de tiempo
S	bytes almacenados	T	bytes transferidos
W_f	escrituras físicas	R_f	lecturas físicas
C_x	costo unitario del recurso x	h	proporción de aciertos de caché
f_q	frecuencia de la consulta	f_w	frecuencia de escritura
d	número de dependencias	k	tamaño del lote

Estas variables se calculan por organización y también de manera agregada. El promedio global puede ocultar que una organización pequeño recibe peor servicio o que uno grande domina el consumo. Por ello, cada resultado debe conservar al menos identificador anonimizado de ámbito, tipo de operación, versión del contrato y configuración experimental.

8.2. Frontera de medición

Una medición debe indicar dónde comienza y termina. COBRIA propone tres fronteras:

1. **Algorítmica:** trabajo efectuado por la transformación, sin red ni persistencia.
2. **Componente:** Repositorio, Gestor de proyecciones o Constructor de conjuntos de datos junto con sus dependencias simuladas o reales.
3. **Extremo a extremo:** desde el caso de uso hasta la respuesta observable, incluyendo autorización, serialización, red, almacenamiento y reintentos.

No deben compararse cifras de fronteras distintas como si midieran el mismo fenómeno. La instrumentación se incorpora fuera del dominio siempre que sea posible para evitar que el modelo canónico dependa de un proveedor de telemetría.

8.3. Métricas principales

La latencia se reportará mediante mediana y percentiles p95 y p99, no solo con el promedio. El rendimiento efectivo se expresa como operaciones completadas por segundo bajo una

conurrencia declarada. También se registrarán bytes leídos y escritos, cantidad de operaciones físicas, errores, reintentos, uso de memoria, tiempo de CPU y retraso de actualización de proyecciones. El enfoque sigue la práctica de separar carga de trabajo, métricas y configuración al comparar sistemas de almacenamiento (Cooper et al., 2010; Jain, 1991).

Complejidad computacional de las operaciones

9.1. Acceso canónico

Cuando una clave física se deriva directamente de (S, id) y el motor ofrece acceso indexado por clave, la búsqueda lógica de una entidad se modela como $O(1)$ promedio. Esta expresión presupone una clave conocida; no cubre resolución DNS, autenticación, red, deserialización ni contención. Si se busca por un atributo no indexado, el costo puede crecer hasta $O(N)$ porque deben examinarse los registros del ámbito.

La recuperación de una página de R resultados desde una proyección ordenada puede modelarse como $O(\log N + R)$ cuando existe una estructura de búsqueda balanceada, o como $O(R)$ después de localizar un cursor. El uso de offset puede degradarse al aumentar la profundidad; COBRIA prefiere cursores estables compuestos por orden, ámbito e identidad.

9.2. Creación y actualización

Validar un objeto con m campos tiene costo $O(m)$ si cada regla es local. Las reglas que consultan otras entidades agregan d accesos y deben expresarse como $O(m + d)$ más el costo de esos accesos. Persistir solamente el canónico requiere una escritura lógica. Mantener síncronamente P_w proyecciones agrega, en el caso simple, $O(P_w)$ transformaciones y escrituras:

$$T_{write} = T_{validate} + T_{canonical} + \sum_{i=1}^{P_w} (T_{gi} + T_{persist_i}). \quad (9.1)$$

La suma no implica que todos los términos sean iguales. Una proyección de clave puede ser constante, mientras un agregado que inspecciona una colección puede depender de su

tamaño. La ejecución paralela reduce tiempo de pared, pero no elimina trabajo total ni garantiza atomicidad.

9.3. Reconstrucción total e incremental

Una reconstrucción total que aplica P transformaciones independientes a N entidades tiene costo superior $O(NP)$. Si cada entidad alimenta solamente un subconjunto fijo, el costo efectivo es $O(N\bar{P})$, donde $\bar{P}$ es el promedio de proyecciones aplicables. La memoria puede mantenerse en $O(k)$ mediante procesamiento por lotes, en vez de cargar $O(N)$ objetos.

Una reconstrucción incremental basada en un índice confiable de cambios procesa ΔN entidades:

$$T_{\text{incremental}} \approx O(\Delta N \bar{P}) + T_{\text{discover}} + T_{\text{verify}}. \quad (9.2)$$

Es ventajosa cuando $\Delta N \ll N$ y el costo de descubrir cambios no domina. Si el marca de corte es incompleto, una reconstrucción incremental rápida puede producir un estado incorrecto. La optimización, por tanto, está subordinada a la cobertura del registro de cambios.

9.4. Verificación de proyecciones

Comparar cada canónico con sus derivados cuesta $O(N\bar{P})$. Un muestreo de s entidades reduce el trabajo a $O(s\bar{P})$, pero solo estima la tasa de anomalías. Los resúmenes criptográficos por partición permiten localizar diferencias sin transferir todos los objetos; su construcción inicial sigue siendo lineal y su actualización requiere disciplina de versionado.

Cuadro 9.1: Complejidad esperada de operaciones COBRIA.

Operación	Orden esperado	Condición principal
Buscar canónico por clave	$O(1)$ promedio	Ruta derivable de ámbito e ID.
Escanear por atributo	$O(N)$	No existe proyección o índice.
Consultar proyección	$O(\log N + R)$	Índice ordenado; depende del motor.
Validar entidad	$O(m + d)$	Reglas locales y dependencias explícitas.
Actualizar derivados	$O(P_w)$	Transformaciones acotadas por proyección.
Reconstrucción total	$O(N\bar{P})$	Recorrido completo por lotes.
Reconstrucción incremental	$O(\Delta N\bar{P})$	Registro de cambios completo.
Verificación completa	$O(N\bar{P})$	Comparación fuente–derivado.
Generar conjunto de datos	$O(NF)$	F características por entidad, sin uniones externos.

La tabla representa modelos de crecimiento y no contratos del proveedor. Cada implementación deberá confirmar sus supuestos con planes de consulta, contadores y pruebas de carga.

Amplificación y modelo de costos

10.1. Amplificación de lectura

La amplificación de lectura relaciona trabajo físico y resultado útil. Para una consulta q se define:

$$A_r(q) = \frac{B_{read}(q)}{\max(B_{result}(q), \epsilon)}, \quad (10.1)$$

donde ϵ evita división por cero en consultas sin resultados. También puede calcularse por registros: $A_r^{reg} = R_f / \max(R, 1)$. Una consulta que escanea diez mil registros para devolver diez tiene amplificación aproximada de mil. Una proyección bien diseñada acerca el numerador al volumen útil, aunque añade mantenimiento y almacenamiento.

10.2. Amplificación de escritura

Para una operación lógica o :

$$A_w(o) = \frac{W_f(o)}{W_l(o)}, \quad (10.2)$$

donde normalmente $W_l = 1$. Si se persiste el canónico, una entrada cola de salida y tres proyecciones, $A_w = 5$ sin contar réplicas internas del motor. Deben reportarse por separado la amplificación visible para la aplicación y la amplificación interna cuando el proveedor la exponga.

10.3. Amplificación de almacenamiento

El factor de almacenamiento es:

$$A_s = \frac{S_{canonical} + S_{projections} + S_{metadata} + S_{registros}}{S_{canonical}}. \quad (10.3)$$

Una proyección no se justifica por ser pequeña de forma aislada. Su costo acumulado incluye versiones durante reconstrucciones, índices del motor, historial, respaldos y transferencia. COBRIA asigna a cada proyección un presupuesto y un propietario.

10.4. Costo total por ventana

Para una ventana temporal τ , el costo observable se modela como:

$$C_{total}(\tau) = C_{read}R_f + C_{write}W_f + C_{store}S\tau \quad (10.4)$$

$$+ C_{transfer}T + C_{compute}U + C_{ops}H, \quad (10.5)$$

donde U representa consumo computacional y H esfuerzo operativo estimado. El último término evita declarar óptima una solución barata para el proveedor pero costosa de operar, depurar o reconstruir. Los precios cambian; por ello el experimento publicará cantidades físicas y dejará los precios como parámetros reemplazables.

10.5. Punto de equilibrio de una proyección

Sea C_s el costo de una consulta sin proyección, C_p el costo de consultarla con proyección y C_m el costo de mantenerla por escritura. La proyección produce beneficio en una ventana cuando:

$$f_q(C_s - C_p) > f_w C_m + C_{storage} + C_{rebuild}. \quad (10.6)$$

Esta desigualdad no pretende reducir toda decisión a dinero. Latencia máxima, disponibilidad, aislamiento y corrección pueden actuar como restricciones obligatorias. Sí permite identificar proyecciones con uso insuficiente o mantenimiento excesivo.

10.6. Presupuesto de proyecciones

Cada contrato debe declarar límites de bytes, escrituras por cambio, retraso tolerable, tiempo máximo de reconstrucción y frecuencia mínima que justifica su existencia. Un reporte

periódico compara presupuesto y observación. Si una proyección excede el límite, las opciones son optimizarla, reducir campos, agrupar eventos, cambiar consistencia o retirarla; no se oculta el exceso mediante promedios.

Estrategias de optimización

11.1. Optimizar desde la consulta

La secuencia recomendada parte de una consulta observable: frecuencia, selectividad, orden, tamaño de respuesta y objetivo de latencia. Después se decide si basta una clave canónica, un índice del motor, una proyección específica o una caché. Crear estructuras derivadas antes de conocer la carga aumenta P y puede empeorar H9.

Una proyección debe almacenar solo campos necesarios para filtrar, ordenar y responder. Duplicar el documento completo facilita algunas lecturas, pero aumenta transferencia, amplificación y riesgo de exposición. La paginación por cursor evita recorrer desplazamientos crecientes y conserva una frontera consistente de lectura cuando el orden incluye una clave de desempate.

11.2. Lotes y paralelismo controlado

El tamaño de lote k equilibra memoria, viajes de red y duración de transacciones. Un lote demasiado pequeño multiplica comunicaciones; uno demasiado grande presiona memoria y extiende la recuperación después de un fallo. COBRIA no fija un k universal: lo selecciona experimentalmente y registra la configuración.

El paralelismo se limita por partición, cuota y capacidad de escritura. Aumentar procesos de trabajo puede reducir tiempo hasta alcanzar contención o limitación de capacidad; después, el rendimiento efectivo se estanca y la latencia de cola aumenta. El controlador debe aplicar control de contrapresión, variación aleatoria y reintentos acotados, conservando idempotencia.

11.3. Caché con ámbito y revisión

Con una tasa de aciertos h , el costo medio simplificado de lectura es:

$$E[T] = hT_{hit} + (1 - h)T_{miss}. \quad (11.1)$$

La clave incluye organización, tipo, ID y, cuando corresponda, versión. La caché no sustituye una proyección si se necesita consulta por atributos, ni se considera autoridad. TTL reduce permanencia, pero no garantiza invalidación inmediata. Para datos sensibles se deben medir además exposición, borrado y segregación.

11.4. Reconstrucción eficiente

Una reconstrucción segura y optimizada sigue cinco decisiones:

1. fijar versión y marca de corte de entrada;
2. dividir por ámbito y rango estable;
3. procesar lotes idempotentes con checkpoints;
4. escribir en una versión paralela de la proyección;
5. verificar y efectuar un cambio de versión atómico o reversible.

La versión paralela requiere almacenamiento temporal, pero evita exponer una mezcla de contratos. Para reconstrucciones incrementales se conserva una operación total como mecanismo de recuperación y como prueba periódica de reconstruibilidad.

11.5. Optimización adaptativa

La arquitectura puede recomendar cambios a partir de telemetría, sin permitir que una heurística modifique por sí sola el estado canónico. Un ciclo adaptativo observa frecuencia, amplificación, latencia y retraso; propone crear, rediseñar o retirar una proyección; estima el punto de equilibrio; ejecuta una comparación controlada; y somete el cambio a aprobación. Esto hace posible usar minería de datos para descubrir patrones de acceso y aprendizaje automático para predecir carga, manteniendo una frontera de gobierno.

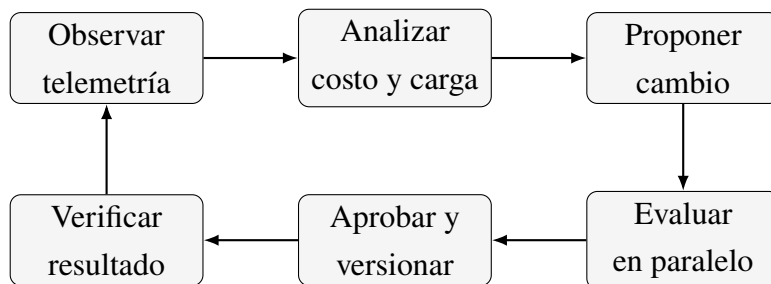


Figura 11.1: Ciclo gobernado de optimización adaptativa.

Protocolo experimental de rendimiento

12.1. Objetivo y comparaciones

El protocolo evaluará si COBRIA reduce el costo de lectura y mejora reconstrucción y trazabilidad a cambio de costos cuantificables. No se presentarán resultados hasta ejecutar el protocolo. Se compararán, como mínimo, estas configuraciones:

1. **B0, acceso directo:** canónicos sin proyección específica;
2. **B1, optimización nativa:** índices disponibles en el motor;
3. **C1, COBRIA síncrona:** canónico y proyecciones críticas en la operación;
4. **C2, COBRIA asíncrona:** canónico, cola de salida y proyecciones actualizadas por procesos de trabajo;
5. **C3, COBRIA incremental:** C2 con registro de cambios y reconstrucción incremental.

No todas las configuraciones aplican a toda operación. La comparación se declara antes de observar resultados y mantiene la misma semántica, conjunto de datos y política de consistencia.

12.2. Factores y niveles

Cuadro 12.1: Factores propuestos para los experimentos.

Factor	Niveles iniciales	Propósito
Volumen N	10^3 , 10^4 , 10^5 y, si es viable, 10^6	Estimar tendencia de crecimiento.
Mezcla lectu- ra/escritura	95/5, 80/20, 50/50	Representar cargas distintas.
Concurrencia	1, 8, 32 y 128 clientes	Detectar saturación y contención.
Número de proyeccio- nes	0, 1, 3, 5 y 10	Medir amplificación marginal.
Distribución	uniforme y sesgada	Evaluar claves calientes.
Tamaño de entidad	pequeño, medio y grande	Medir serialización y transferencia.
Cambios $\Delta N/N$	0.1 %, 1 %, 10 % y 100 %	Comparar reconstrucción incremental y total.

Los valores son puntos iniciales y se ajustarán a la capacidad del entorno antes de registrar la especificación definitiva. El Sistema A aportará cargas temporales, multimodales y de eventos; el Sistema B, operaciones empresariales, catálogos e aislamiento multiinquilino. Los nombres y datos originales no aparecerán en scripts ni resultados.

12.3. Datos y cargas reproducibles

Los generadores usarán semilla publicada, distribuciones declaradas y relaciones válidas. Para evitar que datos triviales favorezcan una estrategia, incluirán cardinalidad, sesgo, valores ausentes, revisiones y tamaños variables. Los artefactos multimedia se representarán mediante metadatos y archivos sintéticos; no se copiará contenido privado.

Cada ejecución incluye calentamiento, ventana de medición y enfriamiento. Se aleatoriza el orden de configuraciones cuando sea posible y se repiten ejecuciones independientes. Se registra hardware, sistema operativo, entorno de ejecución, versión del motor, región, límites, configuración de índices y revisión del código. Un prueba comparativa como YCSB sirve de referencia para estructurar cargas, pero las operaciones de dominio y reconstrucción requieren escenarios COBRIA adicionales (Cooper et al., 2010).

12.4. Instrumentación y control

Cada operación recibe un identificador de correlación. Los contadores se toman en el cliente y, cuando exista acceso, en el almacenamiento. Se sincronizan relojes para medir retraso asíncrono, pero la latencia local utiliza un reloj monótonico. Se evita registrar cargas útiles sensibles. La telemetría debe distinguir éxito, rechazo de negocio, conflicto, timeout y reintento.

Los ensayos controlarán cachés, calentamiento, concurrencia y tareas de fondo. Si un servicio administrado introduce ruido no controlable, se reportará y se aumentarán repeticiones, sin eliminar observaciones solo por ser desfavorables.

12.5. Análisis estadístico

Para cada combinación se reportarán tamaño muestral, mediana, p95, p99, intervalo de confianza y distribución gráfica. Las comparaciones incluirán diferencia absoluta, razón y tamaño de efecto. Antes de usar pruebas paramétricas se examinarán independencia, forma de distribución y varianza. Cuando no se satisfagan supuestos, se preferirán intervalos remuestreo con reemplazo o pruebas no paramétricas. Las múltiples comparaciones se identificarán y corregirán cuando corresponda.

La significancia estadística no reemplaza relevancia práctica. Una mejora pequeña puede carecer de valor si aumenta mucho A_w , A_s o el retraso. El análisis de sensibilidad variará precios, frecuencias y objetivos de latencia en la ecuación 10.5.

12.6. Criterios de aceptación

Las hipótesis se juzgarán con reglas previas:

- H1 se apoya si el crecimiento de lecturas se aproxima a R y la latencia de cola mejora sin cambiar la semántica;
- H2 se cuantifica, no se considera un fallo: toda reducción se acompaña de A_w y A_s ;
- H3 requiere igualdad según el contrato, ausencia de huérfanos y cobertura completa después de reconstruir;
- H4 compara total e incremental para distintos $\Delta N/N$ y busca su punto de cruce;
- H9 se apoya si existen regiones donde el costo marginal supera el beneficio de lectura.

No se modificará un umbral después de conocer resultados sin declarar el cambio como exploratorio.

12.7. Plantilla de resultados

Cuadro 12.2: Matriz mínima de resultados por escenario.

Dimensión	Indicadores	Interpretación requerida
Respuesta	p50, p95, p99, rendimiento efectivo	Saturación y variabilidad.
Lectura	registros, bytes, A_r	Trabajo evitado frente a resultado.
Escritura	operaciones, bytes, A_w	Precio de mantener derivados.
Almacenamiento	bytes y A_s	Incluye versiones e índices.
Reconstrucción	duración, cobertura, anomalías	Exactitud antes que velocidad.
Consistencia	retraso, conflictos, reintentos	Ventana de obsolescencia.
Operación	CPU, memoria, horas de trabajo	Costo total y mantenibilidad.
Equidad de ámbito	percentiles por organización	Evitar ocultar degradación.

Rendimiento para minería de datos e inteligencia artificial

13.1. Preparación analítica como proyección

Un conjunto de datos se trata como una proyección versionada del estado canónico y de eventos permitidos. Su costo de construcción depende de entidades, ventanas temporales y número de características. Para N entidades y F características locales, el caso simple es $O(NF)$. Uniones, agregados temporales, texto o embeddings añaden términos que deben medirse por separado.

Esta estructura favorece minería de datos porque conserva el significado de la entidad, el ámbito, el tiempo del evento, la versión de transformación y el linaje. Permite volver a producir un conjunto de datos, comparar versiones y detectar qué cambios de datos explican una diferencia. No garantiza que las características sean útiles ni que el modelo sea justo.

13.2. Corrección temporal y prevención de fuga

Para una observación con tiempo de corte t , solo se utilizan hechos disponibles en o antes de t . Una característica calculada con datos futuros produce fuga y métricas de entrenamiento engañosas. El Constructor de conjuntos de datos recibe marca de corte, ventana y versión; el resultado registra tiempo de evento, tiempo de ingestión y tiempo de cálculo.

El acceso punto-en-tiempo puede requerir historial canónico, log de eventos o snapshots. Si el sistema conserva únicamente el estado actual, COBRIA debe declarar que ciertas reconstrucciones históricas no son posibles en vez de inferirlas.

13.3. Características, embeddings y multimodalidad

Las características tabulares, series, texto y referencias multimedia pueden compartir un contrato de procedencia. Los archivos grandes permanecen fuera del objeto canónico; este conserva ubicación autorizada, hash, tipo, versión y política. Una extracción de audio, texto o imagen genera un derivado con modelo, parámetros y fecha.

Los embeddings son proyecciones reconstruibles si se conserva la entrada autorizada, el identificador exacto del modelo y la transformación. Cambiar de modelo produce una nueva versión; no se mezclan vectores incompatibles en el mismo índice sin una estrategia explícita. La reconstrucción puede ser costosa, por lo que se presupuestan cómputo, transferencia y ventana de coexistencia.

13.4. RAG y agentes

En recuperación aumentada, el índice vectorial no es autoridad: sirve para localizar candidatos. La respuesta debe enlazar fragmento, documento canónico, versión y permisos. La autorización se aplica antes de incorporar contexto al instrucción. Un agente puede leer proyecciones adaptadas a su tarea, pero cualquier cambio de negocio vuelve a un caso de uso que valida identidad, ámbito, invariantes e idempotencia.

Esta frontera limita el daño de una inferencia incorrecta y permite auditar propuesta, evidencia, decisión humana y efecto final. El rendimiento del modelo se mide separado del rendimiento de datos: recuperación, construcción de contexto, inferencia y persistencia tienen latencias y costos distintos.

13.5. Optimización de cadenas de procesamiento de IA

COBRIA permite reutilizar transformaciones deterministas, materializar características de alto uso y reconstruirlas en lotes. La decisión se rige por el mismo punto de equilibrio de la ecuación 10.6. Para entrenamiento se favorecen lotes grandes y reproducibilidad; para inferencia en línea, baja latencia y frescura. Una sola representación física rara vez optimiza ambos caminos, por lo que los contratos deben declarar equivalencia semántica entre versión fuera de línea y en línea, idea central de los Almacenes de características (Li et al., 2023; Orr et al., 2021).

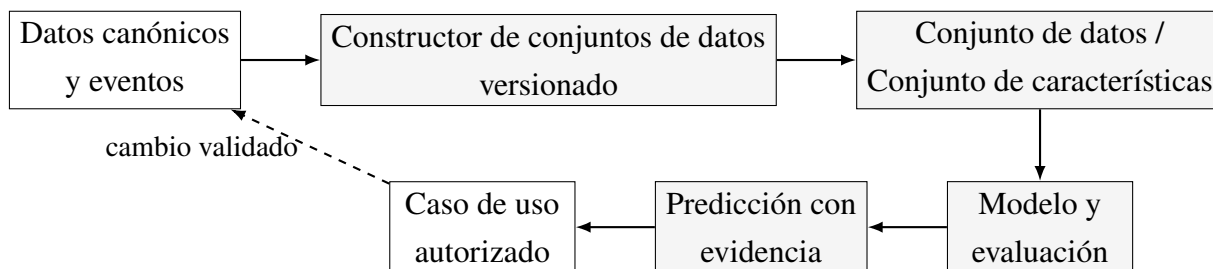


Figura 13.1: Flujo gobernado desde datos canónicos hasta decisiones asistidas por IA.

13.6. Métricas analíticas y de IA

Además de exactitud, precisión, exhaustividad u otras métricas propias del problema, se registrarán cobertura de linaje, reproducibilidad entre ejecuciones, frescura, divergencia en línea–fuera de línea, proporción de campos desconocidos, costo por ejemplo y latencia por etapa. Para RAG se separan recuperación y generación; para agentes se miden también acciones rechazadas, confirmaciones y efectos reversibles. La gestión de riesgo acompaña el ciclo completo y no se reduce a una puntuación del modelo (National Institute of Standards and Technology, 2023).

Amenazas a la validez y síntesis de la Parte III

14.1. Validez interna

Cambios de caché, calentamiento, throttling, tareas de fondo o distinta configuración pueden explicar una diferencia atribuida a COBRIA. Se mitigarán mediante automatización, orden aleatorio, repetición, configuración registrada y comparación sobre la misma semántica. La instrumentación también tiene costo; se medirá una configuración de control cuando su efecto pueda ser relevante.

14.2. Validez de constructo

Latencia no equivale a calidad arquitectónica, cantidad de clases no equivale a mantenibilidad y reconstrucción exitosa no equivale a corrección del negocio. Por ello cada concepto utiliza varios indicadores. La reconstruibilidad exige fuente suficiente, contrato versionado, resultado verificable y ausencia de autoridad en el derivado.

14.3. Validez externa

Dos sistemas anonimizados permiten variación de dominio, pero no representan todos los motores ni escalas. Los emuladores pueden diferir de servicios administrados. Los resultados se generalizarán como condiciones y mecanismos —por ejemplo, relación entre frecuencia de lectura y amplificación—, no como una promesa universal de milisegundos.

14.4. Validez de conclusión

Muestras pequeñas, colas largas y múltiples comparaciones pueden producir conclusiones inestables. Se publicarán distribuciones, intervalos, tamaños de efecto y resultados negativos. Una hipótesis no apoyada no se ocultará; puede delimitar el dominio donde la arquitectura deja de ser conveniente.

14.5. Síntesis de la Parte III

Esta parte convirtió la afirmación general de “optimizar” en magnitudes observables. El beneficio esperado de las proyecciones es reducir el trabajo de lectura desde una dependencia de N hacia una dependencia más cercana a R , a cambio de trabajo de escritura $O(P_w)$, almacenamiento adicional y reconstrucción. Las ecuaciones de amplificación y costo permiten comparar ese intercambio sin confundir complejidad asintótica con tiempo real.

También se definieron estrategias de optimización, un protocolo reproducible y reglas de análisis que impiden inventar resultados. Finalmente, el mismo principio de proyección reconstruible se extendió a conjuntos de datos, características, embeddings, RAG y agentes, incorporando corrección temporal, linaje y autorización. Con ello queda preparada la siguiente parte: desarrollar en profundidad la adaptabilidad de COBRIA a analítica e inteligencia artificial y formalizar sus mecanismos de gobierno.

Parte IV

Analítica, minería de datos e inteligencia artificial

Extensión analítica de COBRIA

La analítica y la inteligencia artificial suelen incorporarse después de que un sistema operacional ha acumulado datos. En ese momento aparecen exportaciones manuales, significados incompatibles, atributos sin procedencia y modelos difíciles de reproducir. COBRIA aborda este problema desde la arquitectura: considera conjuntos de datos, características, embeddings, índices de recuperación y predicciones como proyecciones gobernadas de datos canónicos y eventos autorizados.

La propuesta no convierte el dominio en un sistema de aprendizaje automático. Mantiene una frontera explícita entre hechos de negocio, evidencia derivada e inferencias probabilísticas. Esta frontera permite utilizar la misma disciplina de identidad, ámbito, versionado, reconstrucción y trazabilidad tanto en minería descriptiva como en modelos predictivos, recuperación aumentada y agentes.

15.1. De la operación al conocimiento

Se distinguen cinco niveles semánticos:

1. **Hecho canónico:** estado validado por un caso de uso y sujeto a invariantes del dominio.
2. **Evento:** observación temporal de un cambio o interacción, con tiempo, ámbito y procedencia.
3. **Derivación analítica:** transformación determinista o controlada, como una característica, agregado o fragmento.
4. **Inferencia:** salida probabilística de un modelo, acompañada de versión, evidencia y medidas de incertidumbre disponibles.
5. **Decisión:** acción aceptada por una persona o caso de uso autorizado.

Una inferencia no asciende automáticamente a hecho. Si una clasificación, recomendación o respuesta debe modificar el negocio, se formula como propuesta y atraviesa validación, autorización e idempotencia. Esta regla evita que una salida plausible pero incorrecta sobrescriba la fuente canónica.

15.2. Capacidades analíticas

Cuadro 15.1: Adaptación de COBRIA a modalidades analíticas.

Modalidad	Proyección principal	Requisitos COBRIA
Descriptiva	Agregado o cubo	Definición, ventana, ámbito y reconciliación.
Minería de datos	Conjunto de datos versionado	Semilla, filtros, linaje y calidad.
Series temporales	Secuencia por entidad	Tiempo de evento, ingestión y marca de corte.
Aprendizaje supervisado	Conjunto de características y etiquetas	Corte temporal y prevención de fuga.
Texto y multimedia	Fragmentos y descriptores	Resumen criptográfico, derechos, modelo extractor y versión.
Búsqueda semántica	Índice de embeddings	Modelo compatible, permisos y reconstrucción.
RAG	Corpus recuperable	Evidencia, vigencia y autorización previa.
Agentes	Herramientas tipadas	Capacidades mínimas, aprobación y auditoría.

15.3. Componentes de la extensión

La extensión de inteligencia se compone de un Constructor de conjuntos de datos, un Registro de características, un Almacén de artefactos, un Registro de modelos, un Pasarela de recuperación, un Motor de políticas y un Evidencia Registro. Estos componentes se apoyan en Núcleo, Metadatos, Proyecciones y Gobierno; no acceden a rutas físicas mediante convenciones privadas.

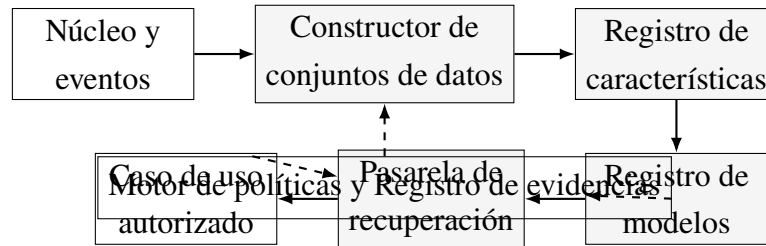


Figura 15.1: Componentes de la extensión analítica e inteligente de COBRIA.

15.4. Principios de adaptación

La extensión conserva ocho principios: autoridad canónica, derivación declarada, reconstruibilidad proporcional, aislamiento por ámbito, corrección temporal, mínimo dato necesario, evidencia enlazable y supervisión proporcional al riesgo. La reconstruibilidad es proporcional porque no toda salida probabilística puede reproducirse bit a bit; en esos casos se exige reproducibilidad del proceso, captura de parámetros y medición de variación.

Contratos de conjuntos de datos y linaje

16.1. Definición formal

Un contrato de conjunto de datos se representa como:

$$\mathcal{D} = \langle id, v, Q, F, W, S, L, P, V, C \rangle, \quad (16.1)$$

donde id es la identidad; v , la versión; Q , la selección de fuentes; F , las transformaciones; W , la política temporal o marca de corte; S , el esquema de salida; L , el conjunto de etiquetas; P , las políticas; V , las validaciones; y C , las condiciones de publicación. Una ejecución produce un manifiesto M con entradas, salidas, resúmenes criptográficos, tiempos, código y entorno.

El conjunto de datos se considera reconstruible si existen fuentes permitidas suficientes, las transformaciones y dependencias están fijadas, el contrato puede ejecutarse y el resultado puede verificarse según una equivalencia declarada. Para datos tabulares deterministas puede exigirse igualdad exacta; para conversiones multimedia o librerías no deterministas se requiere tolerancia documentada.

16.2. Contrato mínimo

Cuadro 16.1: Contenido mínimo de un contrato de conjunto de datos.

Elemento	Pregunta que responde	Evidencia conservada
Propósito	¿Para qué análisis puede utilizarse?	Tarea, población y usos excluidos.

Elemento	Pregunta que responde	Evidencia conservada
Fuentes	¿De dónde procede cada campo?	Tipo canónico, evento, versión y ámbito.
Selección	¿Qué se incluye o excluye?	Predicados, ventanas y razones.
Transformación	¿Cómo se obtiene cada variable?	Función, parámetros y código.
Tiempo	¿Qué era conocido en el corte?	tiempo del evento, tiempo de ingestión y marca de corte.
Esquema	¿Qué significa cada columna?	Tipo, unidad, dominio y nulabilidad.
Calidad	¿Qué condiciones debe cumplir?	Reglas, umbrales y resultados.
Privacidad	¿Qué datos están permitidos?	Base de acceso, minimización y retención.
Publicación	¿Cuándo es utilizable?	Aprobación, hash, ubicación y estado.
Retiro	¿Cuándo deja de utilizarse?	Fecha, motivo y dependencias notificadas.

Este contrato complementa prácticas de documentación como las fichas de datos, que buscan explicitar motivación, composición, proceso de recolección, usos y mantenimiento (Gebu et al., 2021). COBRIA añade correspondencia ejecutable con entidades, proyecciones y ámbitos del sistema.

16.3. Grafo de linaje

El linaje se modela como un grafo dirigido acíclico $G_L = (V_L, E_L)$. Los nodos pueden ser fuentes, ejecuciones, conjuntos de datos, características, modelos, evaluaciones y predicciones. Una arista expresa *derivado de*, *entrenado con*, *evaluado con* o *producido por*. Cada nodo posee identidad y versión; una etiqueta textual sin referencia estable no constituye linaje suficiente.

Para una predicción y , la consulta de procedencia debe poder recorrer:

$y \rightarrow \text{modelo} \rightarrow \text{Conjunto de características} \rightarrow \text{conjunto de datos} \rightarrow \text{fuente canónica o evento.}$
(16.2)

El grafo no almacena necesariamente los datos completos. Puede conservar resúmenes

criptográficos, manifiestos y referencias protegidas. Los permisos se vuelven a comprobar al navegar; poseer el ID de un nodo no concede acceso a su contenido.

16.4. Algoritmo de materialización

```
BuildDataset(contractId, version, ámbito, cutoff):  
  contract <- registry.require(contractId, version)  
  policy.require("conjunto de datos.build", ámbito, contract.purpose)  
  snapshot <- resolveSources(contract.sources, ámbito, cutoff)  
  run <- ledger.start(contract, snapshot, actor, environment)  
  for batch in snapshot.readBatches():  
    rows <- contract.transform(batch)  
    contract.validate(rows)  
    artifact.write(run.stagingVersion, rows)  
    run.checkpoint(batch.cursor, rows.hash)  
  summary <- verify(run.stagingVersion, contract)  
  if not summary.accepted: quarantine(run, summary)  
  else publishAtomically(run, summary)  
  return run.manifiesto
```

La publicación se realiza después de validar, no mientras el conjunto de datos está incompleto. Una ejecución fallida queda en cuarentena con sus métricas y no reemplaza la versión vigente.

16.5. Calidad y observabilidad

Las reglas se agrupan en esquema, completitud, dominio, unicidad, integridad referencial, distribución, tiempo y privacidad. Un cambio estadístico no siempre es un defecto; puede reflejar un cambio real. Por eso las alertas de deriva generan revisión y no corrección automática del canónico.

Almacén de características, series temporales y minería de datos

17.1. Características como proyecciones

Una característica se define mediante:

$$\phi = \langle \text{nombre}, v, \text{entidad}, \text{ambito}, \text{tipo}, \text{unidad}, g, W, \text{frescura}, \text{responsable} \rangle. \quad (17.1)$$

La función g transforma fuentes autorizadas; W define la ventana temporal. La identidad de entidad permite unir valores sin depender de campos ambiguos. El registro describe significado y versión; los valores pueden almacenarse fuera de línea, en línea o en ambos medios.

Los Almacenes de características buscan coordinar cálculo, almacenamiento y servicio de características entre entrenamiento e inferencia (Li et al., 2023; Orr et al., 2021). COBRIA los incorpora como una especialización de sus proyecciones: la definición vive en metadatos, la fuente conserva autoridad y las materializaciones pueden reconstruirse.

17.2. Equivalencia fuera de línea–en línea

Sea $\phi_o(e, t)$ la característica calculada fuera de línea y $\phi_n(e, t)$ la servida en línea. Se exige una relación de equivalencia:

$$\phi_o(e, t) \simeq_{\epsilon} \phi_n(e, t), \quad (17.2)$$

donde $\epsilon = 0$ para valores deterministas exactos y puede ser una tolerancia justificada para cómputo numérico. Las diferencias se investigan por contrato, versión, marca de corte, ventana

y código. Compartir nombre no demuestra equivalencia.

17.3. Corrección punto-en-tiempo

Para una observación de entrenamiento en t_c , COBRIA solo admite una fuente x si su tiempo de disponibilidad satisface $t_a(x) \leq t_c$. El tiempo del evento puede ser anterior y el de ingestión posterior; usar únicamente event time puede introducir información que todavía no estaba disponible.

$$\mathcal{D}(t_c) = \{x \mid \text{ambito}(x) = s \wedge t_a(x) \leq t_c \wedge \text{politica}(x) = \text{permitido}\}. \quad (17.3)$$

Esta condición reduce fuga temporal. Las etiquetas usan una ventana futura separada y no participan en la construcción de características del mismo corte.

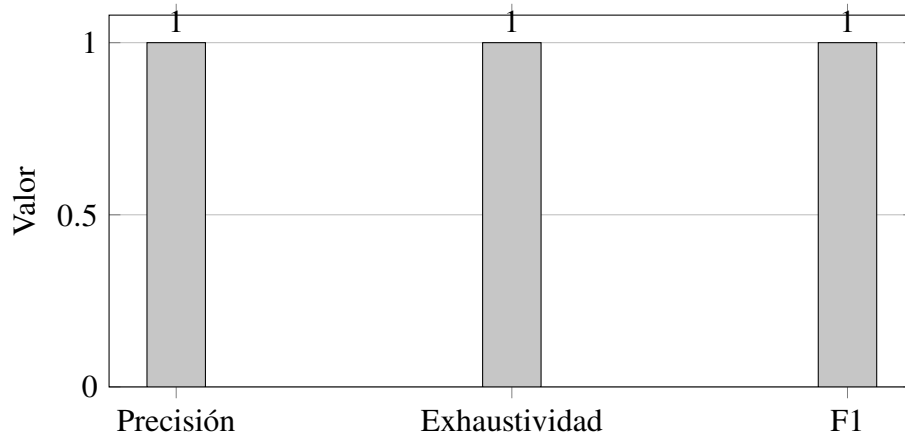


Figura 17.1: Contratos sintéticos de recuperación en 30 repeticiones. No constituyen evaluación humana de un modelo generativo.

17.4. Eventos tardíos y marcas de avance temporal

Los eventos se clasifican como a tiempo, tardíos pero aceptables, o fuera de la ventana de corrección. La política declara cuánto se espera, cuándo se recalcula una proyección y cuándo se publica una corrección. Un marca de corte no significa que nunca aparecerá otro evento; expresa una decisión operacional sobre completitud suficiente.

17.5. Minería descriptiva y descubrimiento

COBRIA facilita segmentación, reglas de asociación, detección de anomalías y análisis de secuencias porque cada observación conserva entidad, ámbito, tiempo y procedencia. El proceso exploratorio puede descubrir nuevas variables o patrones, pero sus resultados se registran como hipótesis analíticas. Una agrupación no redefine automáticamente una categoría del negocio y una anomalía no implica fraude o error.

Para análisis no supervisado se versionan preprocesamiento, escalamiento, imputación, selección de variables, métrica de distancia, algoritmo y semilla. Esto permite separar la inestabilidad del algoritmo de cambios en los datos de entrada.

17.6. Catálogo analítico

El catálogo enlaza entidades, eventos, características, conjuntos de datos, modelos y responsables. Debe responder quién puede usar un artefacto, con qué propósito, de qué versión depende, cuándo fue actualizado, qué calidad alcanzó y qué consumidores se verán afectados por un cambio. No es solo una lista de archivos: refleja relaciones ejecutables del grafo de linaje.

Multimodalidad, embeddings y búsqueda semántica

18.1. Objeto multimedia canónico

El canónico de un recurso multimedia no contiene necesariamente los bytes. Conserva identidad, ámbito, propietario, tipo MIME validado, tamaño, hash, ubicación lógica, tiempos, consentimiento o base de uso, retención y estado. Las miniaturas, transcripciones, descriptores y embeddings son derivados.

Separar contenido y metadatos evita transportar objetos grandes en consultas comunes y permite políticas distintas. Una URL temporal no es identidad permanente; el Artefacto Pasarela resuelve la referencia solo después de autorización.

18.2. Pipeline multimodal

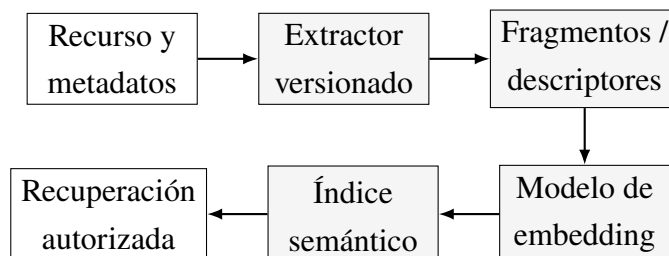


Figura 18.1: Pipeline multimodal reconstruible.

Cada etapa genera un manifiesto. Para una transcripción se conserva idioma, modelo, parámetros y segmentos temporales; para imagen, regiones o descriptores; para video, fotogramas

o intervalos; para series, frecuencia y método de remuestreo. Los errores de extracción no modifican el recurso fuente.

18.3. Contrato de embedding

Un embedding se representa como:

$$z = \langle sourceId, sourceVersion, chunkId, modelId, \\ modelVersion, d, normalization, vector, policy \rangle. \quad (18.1)$$

La dimensión d , normalización y modelo determinan compatibilidad. Dos vectores con la misma dimensión no son necesariamente comparables. Una migración crea un índice nuevo, recalcula y valida recuperación antes del cambio de versión.

Los modelos basados en atención permiten representar dependencias contextuales y son la base de numerosos modelos modernos de lenguaje y modalidades relacionadas (Vaswani et al., 2017). COBRIA no depende de una arquitectura de modelo específica; registra qué modelo produjo cada representación.

18.4. Fragmentación

El contrato de fragmentación declara unidad, solapamiento, límites semánticos, tamaño máximo y tratamiento de tablas o marcas temporales. Un fragmento mantiene referencia a la fuente y desplazamientos suficientes para mostrar evidencia. Si cambia el documento, los fragmentos anteriores quedan asociados a su versión y se generan otros nuevos.

Fragmentos demasiado pequeños pierden contexto; demasiado grandes disminuyen precisión de recuperación y aumentan costo de instrucción. La elección se evalúa por corpus y tarea, no mediante una constante universal.

18.5. Recuperación híbrida

La búsqueda puede combinar filtros estructurados, texto y similitud vectorial. Primero se limita por ámbito, permiso, vigencia y tipo; después se puntúa. Aplicar autorización solo al final permite que un documento prohibido influya en ranking o contexto.

$$score(x, q) = \alpha s_{vector} + \beta s_{text} + \gamma s_{metadata}, \quad (18.2)$$

con pesos versionados. El resultado incluye score por componente, modelo, consulta normalizada y referencias. El score ordena candidatos; no prueba verdad.

18.6. Derecho a retirar y reconstrucción

Cuando un recurso se retira, se invalidan fragmentos, embeddings, cachés y conjuntos de datos que lo contienen según la política. El grafo de linaje permite localizar dependencias. Un modelo ya entrenado plantea un problema diferente: puede requerir evaluación, nuevo entrenamiento o documentación de la limitación, según obligación y riesgo. Borrar una fila del índice no demuestra que toda influencia desapareció.

Recuperación aumentada y agentes gobernados

19.1. RAG como composición de proyecciones

La recuperación aumentada combina una memoria paramétrica con evidencia recuperada para producir respuestas condicionadas por fuentes externas (Lewis et al., 2020). En COBRIA, el corpus, los fragmentos y el índice son proyecciones; la respuesta es una inferencia; las entidades y documentos autorizados conservan autoridad.

Una ejecución se formaliza como:

$$r = \text{Generar}(M_v, p, \text{Recuperar}(I_v, q, \text{ambito}, \text{politica}), \theta), \quad (19.1)$$

donde M_v es el modelo, p la plantilla, I_v el índice, q la consulta y θ los parámetros. El registro conserva todas estas versiones junto con fragmentos recuperados, scores, filtros, tiempos y resultado.

19.2. Flujo seguro de recuperación

1. autenticar actor y resolver ámbito;
2. clasificar propósito y sensibilidad de la solicitud;
3. construir filtros obligatorios antes de recuperar;
4. recuperar candidatos y verificar vigencia;
5. limitar y ordenar evidencia;

6. generar con plantilla y modelo versionados;
7. verificar formato, citas y políticas de salida;
8. registrar evidencia y devolver respuesta con incertidumbre apropiada.

No se envía al modelo más contexto del necesario. Las instrucciones presentes dentro de documentos recuperados se tratan como contenido, no como autoridad para cambiar políticas o ejecutar herramientas.

19.3. Contrato de herramienta para agentes

Una herramienta se declara como:

$$\mathcal{T} = \langle \text{nombre}, v, \text{entrada}, \text{salida}, \text{capacidad}, \text{ambito}, \text{riesgo}, \text{confirmacion}, \text{idempotencia}, \text{compensacion} \rangle. \quad (19.2)$$

El agente no recibe credenciales de almacenamiento ni acceso general a Repositorio. Usa herramientas estrechas que llaman casos de uso. El contrato especifica esquemas, precondiciones, efectos, permisos, necesidad de confirmación y forma de compensar cuando sea posible.

Cuadro 19.1: Niveles de autonomía propuestos.

Nivel	Capacidad	Control requerido
A0	Responder sin herramientas	Evidencia y filtrado de salida.
A1	Consultar datos de bajo riesgo	Ámbito, mínimo privilegio y auditoría.
A2	Preparar una propuesta	Vista previa y validación previa.
A3	Ejecutar acción reversible	Confirmación, idempotencia y compensación.
A4	Acción de alto impacto	Aprobación humana explícita y segregación.

El nivel se asigna por herramienta y contexto, no por agente completo. Una misma sesión puede consultar un catálogo en A1 y requerir A4 para modificar una obligación financiera.

19.4. Memoria de agentes

La memoria de conversación, las preferencias inferidas y los resúmenes son proyecciones con retención y propósito. No deben convertirse silenciosamente en perfil canónico. Una preferencia confirmada puede persistirse mediante un caso de uso; una inferida conserva marca de procedencia y expiración.

La memoria se segmenta por persona, organización y contexto. Recuperar memoria de otro ámbito es una violación aunque el texto parezca inofensivo. Las instrucciones de borrado deben propagarse a índices y cachés.

19.5. Evidencia y verificabilidad

El Registro de evidencias registra solicitud, actor, ámbito, política, modelo, instrucción versionado, herramientas, entradas resumidas o protegidas, fuentes, salida, validaciones, decisión y efectos. No debe almacenar secretos ni duplicar indiscriminadamente datos personales. Los resúmenes criptográficos permiten verificar integridad sin revelar contenido.

Una respuesta con citas no es automáticamente correcta. Se evalúan relevancia de la evidencia, soporte de afirmaciones, cobertura, vigencia y fidelidad de atribución. Las pruebas incluyen preguntas sin respuesta para comprobar abstención.

Gobierno, seguridad y gestión del riesgo

20.1. Clasificación de artefactos

Cada artefacto se clasifica por autoridad, sensibilidad, propósito, retención, reconstruibilidad y criticidad. La combinación determina controles. Un embedding puede parecer opaco, pero puede revelar similitud o información sensible; hereda al menos la clasificación de su fuente salvo evaluación que justifique otra cosa.

Cuadro 20.1: Gobierno según tipo de artefacto.

Artefacto	Autoridad	Control destacado
Canónico	Primaria	Invariantes, autorización y revisión.
Conjunto de datos	Derivada	Propósito, linaje, calidad y retención.
Característica	Derivada	Corrección temporal y equivalencia.
Representación vectorial	Derivada	Compatibilidad, permisos y retiro.
Modelo	Inferencial	Evaluación, alcance y documentación.
Predicción	Inferencial	Evidencia, vigencia e incertidumbre.
Acción de agente	Propuesta o efecto	Capacidad, confirmación y auditoría.

20.2. Registro y documentación de modelos

El Registro de modelos conserva identidad, versión, tarea, propietario, código, dependencias, conjunto de datos, Conjunto de características, hiperparámetros, métricas, población evaluada, limitaciones, fecha, estado y política de retiro. Las fichas de modelo ofrecen una estructura para comunicar uso previsto, rendimiento y limitaciones (Mitchell et al., 2019).

Un modelo no pasa a producción solo por superar una métrica global. Se revisan segmentos relevantes, calibración, robustez, privacidad, seguridad, costo y compatibilidad con el caso de uso. La aprobación enlaza exactamente los artefactos evaluados.

20.3. Separación de funciones

COBRIA distingue productor de datos, propietario del contrato, desarrollador del modelo, revisor de riesgo, aprobador y operador. En contextos pequeños una persona puede asumir varias funciones, pero las decisiones quedan explícitas. Las acciones de alto impacto no se autoaprueban desde el mismo proceso que generó la propuesta.

20.4. Amenazas específicas

El análisis incluye contaminación de datos, fuga entre organizaciones, inyección de instrucciones, recuperación de contenido prohibido, extracción de información, dependencia de modelos, deriva, abuso de herramientas, resultados fabricados y automatización excesiva. Para IA generativa se consideran además confabulación, integridad de la información, propiedad intelectual y riesgos de la interacción humano-máquina, en consonancia con el perfil de IA generativa del AI RMF (National Institute of Standards and Technology, 2023, 2024).

20.5. Controles por frontera

1. **Ingesta:** validación, análisis de programas maliciosos, procedencia y clasificación.
2. **Transformación:** ejecución aislada, dependencias fijadas y resúmenes criptográficos.
3. **Almacenamiento:** cifrado, segregación, mínimo privilegio y retención.
4. **Recuperación:** filtros previos, límites y verificación de vigencia.
5. **Inferencia:** modelos aprobados, plantillas versionadas y cuotas.
6. **Acción:** herramientas tipadas, confirmación e idempotencia.
7. **Observación:** registros minimizados, alertas y revisión de incidentes.

20.6. Ciclo de vida

Los estados propuestos para conjuntos de datos y modelos son borrador, validación, aprobado, activo, degradado, retirándose y retirado. Una transición requiere evidencia y actor. Un artefacto degradado puede continuar disponible bajo restricciones o volver a una versión segura; la política define la respuesta.

Los sistemas de aprendizaje acumulan deuda más allá del código: dependencias de datos, configuración, consumidores ocultos y cambios de comportamiento pueden elevar el costo de mantenimiento (Sculley et al., 2015). El grafo de linaje y los contratos de COBRIA hacen visibles varias de estas dependencias, aunque no las eliminan.

20.7. Monitoreo y respuesta

El monitoreo separa salud del servicio, calidad de datos, desempeño del modelo, equidad, seguridad y costo. Una alerta debe enlazar versión, población y ventana. La respuesta puede detener publicación, volver a una versión, reducir autonomía, desactivar una herramienta, reconstruir un derivado o solicitar revisión humana.

Evaluación de adaptabilidad analítica e inteligente

21.1. Preguntas de evaluación

La evaluación de la Parte IV busca responder:

1. ¿Puede el mismo modelo de entidad, ámbito y proyección representar datos empresariales, eventos temporales y contenido multimodal?
2. ¿Qué proporción de un conjunto de datos y sus características posee linaje completo?
3. ¿Puede reproducirse una versión sin acceder a datos fuera del corte temporal?
4. ¿Cuánto difieren características fuera de línea y en línea?
5. ¿La recuperación respeta autorización y aporta evidencia suficiente?
6. ¿Los agentes ejecutan únicamente capacidades declaradas y confirmadas?
7. ¿Qué costo adicional introducen gobierno, versionado y reconstrucción?

21.2. Escenarios anonimizados

El Sistema A se representará mediante eventos, secuencias, métricas, texto y recursos multimedia sintéticos. Permitirá evaluar ventanas, extracción, embeddings, recuperación y cadenas de procesamiento de IA. El Sistema B utilizará entidades empresariales, catálogos, configuraciones, documentos y operaciones multiinquilino sintéticas. Permitirá evaluar Conjunto de características, aislamiento, RAG autorizado y herramientas de agentes.

La comparación no exige las mismas proyecciones en ambos sistemas. La adaptabilidad se apoya si los principios y contratos permanecen, mientras las políticas y transformaciones se especializan por dominio.

21.3. Experimentos propuestos

Cuadro 21.1: Experimentos para analítica e inteligencia artificial.

ID	Experimento	Indicadores
AI-1	Reconstruir conjunto de datos versionado	Resumen criptográfico, cobertura, tiempo, diferencias y linaje.
AI-2	Introducir eventos tardíos	Registros corregidos, marca de corte y estabilidad.
AI-3	Comparar fuera de línea y en línea	Error absoluto, tasa de divergencia y frescura.
AI-4	Migrar modelo de embedding	Cobertura, coexistencia, recuperación y costo.
AI-5	Evaluar RAG por ámbito	Precisión@k, evidencia, fugas y abstención.
AI-6	Injectar instrucciones en corpus	Ataques bloqueados y efectos no autorizados.
AI-7	Ejecutar herramientas de agente	Aceptación, rechazo, confirmación e idempotencia.
AI-8	Retirar una fuente	Dependencias detectadas y derivados invalidados.
AI-9	Cambiar distribución de entrada	Deriva detectado, tiempo de respuesta y degradación.
AI-10	Reproducir entrenamiento	Métricas, variación y artefactos faltantes.

21.4. Métricas de linaje y reproducibilidad

La cobertura de linaje se define como:

$$L_c = \frac{\text{campos o artefactos con ruta completa a fuente}}{\text{campos o artefactos evaluados}}. \quad (21.1)$$

La reproducibilidad exacta utiliza proporción de registros idénticos; la aproximada declara tolerancia por variable. También se mide completitud del manifiesto, dependencias resueltas y tiempo hasta reconstrucción. Un L_c alto no garantiza calidad de la fuente, pero permite localizarla y responsabilizar su transformación.

21.5. Métricas de recuperación

Se utilizarán Precisión@k, Exhaustividad@k cuando exista conjunto relevante, rango recíproco, proporción de afirmaciones respaldadas, corrección de citas, vigencia, abstención y tasa de violaciones de ámbito. Las evaluaciones automáticas se complementan con revisión humana ciega sobre una muestra y acuerdo entre evaluadores.

La respuesta final se descompone para evitar que fluidez o estilo oculten fallos de recuperación. Se comparan búsqueda estructurada, vectorial e híbrida con el mismo corpus y política.

21.6. Métricas de agentes

Se registran tareas completadas, pasos, herramientas llamadas, parámetros inválidos, acciones rechazadas, confirmaciones solicitadas, efectos duplicados, compensaciones, tiempo, costo y severidad del peor efecto. El éxito funcional no compensa una violación de autorización.

21.7. Criterios de adaptabilidad

COBRIA se considera adaptable a una modalidad cuando: la fuente se expresa con identidad y ámbito; la derivación posee contrato; el tiempo relevante puede representarse; existe una política de acceso; el artefacto puede reconstruirse o su límite se documenta; y la salida se integra mediante un caso de uso sin eludir autoridad. Si una modalidad exige romper estas condiciones, se registra como límite de la propuesta.

21.8. Resultados negativos previstos

La investigación buscará activamente escenarios adversos: conjuntos de datos pequeños donde el gobierno sea desproporcionado, embeddings cuya reconstrucción sea costosa, modelos no deterministas con variación relevante, información histórica insuficiente, recuperación que

no mejore una búsqueda estructurada y tareas donde un agente añada riesgo sin valor. Estos casos delimitan COBRIA y fortalecen la validez del manuscrito.

21.9. Síntesis de la Parte IV

Esta parte formalizó la adaptabilidad de COBRIA desde datos operacionales hasta minería e inteligencia artificial. Los conjuntos de datos y características se definieron como proyecciones versionadas; el linaje como un grafo navegable; los recursos multimodales, fragmentos y embeddings como artefactos derivados; y RAG como una composición gobernada de recuperación e inferencia.

También se estableció que un agente opera mediante herramientas tipadas y no mediante acceso directo al almacenamiento. Las salidas probabilísticas conservan condición de inferencia o propuesta hasta que un caso de uso las valida. Gobierno, seguridad, documentación y monitoreo acompañan el ciclo completo, con controles proporcionales a sensibilidad y autonomía.

Finalmente, se definieron diez experimentos que permiten evaluar linaje, corrección temporal, equivalencia fuera de línea–en línea, recuperación, resistencia a instrucciones maliciosas, herramientas de agentes y retiro de datos. Con ello queda preparada la siguiente parte del documento: implementar una referencia neutral y describir, sin identificarlos, cómo los dos sistemas materializan y adaptan estos principios.

Parte V

Implementación de referencia y casos de estudio

Método de análisis de las implementaciones

Esta parte estudia cómo los principios de COBRIA aparecen en dos sistemas existentes sin convertir el manuscrito en documentación de productos. Los casos se mantienen anonimizados como Sistema A y Sistema B. El primero está orientado a eventos, secuencias, contenido multimodal y componentes de inteligencia; el segundo es una plataforma empresarial multiinquilino con un dominio amplio, operaciones críticas y una cantidad elevada de entidades.

El análisis se realizó sobre una instantánea del código disponible el 20 de agosto de 2026. Los conteos representan archivos estructurales encontrados en esa fecha y no usuarios, transacciones, cobertura funcional ni calidad. Los nombres de clases, organizaciones, personas, rutas físicas, credenciales y reglas comerciales particulares no se publican.

22.1. Preguntas del estudio de implementación

La observación busca responder cinco preguntas:

1. ¿Cómo se distribuyen modelo, acceso a datos, servicios, casos de uso, esquemas y catálogos?
2. ¿Qué elementos corresponden a patrones conocidos y cuáles constituyen la integración COBRIA?
3. ¿Dónde aparecen entidades canónicas y proyecciones o índices derivados?
4. ¿Qué tan consistente es la aplicación de la estructura dentro de cada caso?
5. ¿Qué cambios serían necesarios para implementar COBRIA de manera verificable y no solo nominal?

22.2. Fuentes de evidencia

Se utilizaron archivos de objetos, modelos base, Daters, conectores, servicios, casos de uso, esquemas, registries, catálogos, documentación técnica y pruebas existentes. La presencia de un archivo indica intención estructural; solo el código ejecutable y las pruebas pueden apoyar una afirmación de comportamiento.

Cuadro 22.1: Niveles de evidencia utilizados.

Nivel	Evidencia	Interpretación permitida
E0	Nombre o documento	Intención; no prueba implementación.
E1	Clase o contrato	Capacidad modelada.
E2	Flujo conectado	Capacidad integrada en código.
E3	Prueba automatizada	Comportamiento verificado en entorno controlado.
E4	Telemetría reproducible	Comportamiento observado bajo carga declarada.

Este criterio evita atribuir funcionalidad completa a una clase porque su nombre contenga términos como inteligencia, sincronización o auditoría. También impide usar el número de archivos como sustituto de mantenibilidad.

22.3. Procedimiento de inventario

El procedimiento fue: identificar directorios de capas; clasificar archivos por función; examinar bases y ejemplos representativos; buscar relaciones entre objeto y Dater; revisar conectores y rutas; localizar índices, cachés, metadatos y servicios; y contrastar el hallazgo con la documentación. Los resultados se sintetizaron en categorías neutrales.

Los conteos se calcularon mediante un criterio reproducible: archivos JavaScript bajo los directorios de objetos, Daters, servicios, casos de uso, schemas y catálogos o datos de referencia. Archivos generados y manuales pueden incrementar estas cifras; por eso se reportan como tamaño estructural aproximado.

22.4. Neutralidad y ética

No se copiaron datos operacionales ni secretos. Los ejemplos de esta parte son pseudocódigo o estructuras reducidas. Las descripciones de dominio se mantienen al nivel necesario para

explicar variación arquitectónica. Un lector puede reproducir la implementación de referencia sin conocer la identidad de los sistemas originales.

Implementación neutral de referencia

23.1. Objetivo

La implementación neutral no busca reemplazar las aplicaciones estudiadas. Su función es mostrar la unidad mínima de COBRIA sin depender de un marco de trabajo o proveedor. Debe ser pequeña, ejecutable y suficientemente explícita para probar identidad, ámbito, mapeo, proyección, reconstrucción y autorización.

La versión ejecutable se distribuye en `replication/examples/ecosistema-app`. Modela observaciones de biodiversidad sin depender de los dominios originales ni de un proveedor. Puede ejecutarse con `npm test` y `npm start`; no requiere bibliotecas externas.

23.2. Comportamientos comprobados de la aplicación pedagógica

El ejemplo contiene seis pruebas automatizadas: registro de un documento canónico válido; rechazo de un ámbito ajeno antes de acceder al repositorio; repetición idempotente sin duplicación; rechazo de una clave reutilizada con otra entrada; rechazo de revisiones obsoletas; y equivalencia después de retirar y reconstruir la proyección. Esta suite demuestra la conexión entre capas y contratos en una unidad pequeña. No se usa como sustituto de las pruebas distribuidas ni como evidencia de rendimiento.

23.3. Estructura propuesta

```
src/  
  core/
```

entities/	entidades canónicas
value-objects/	valores inmutables
policies/	invariantes de dominio
application/	
use-cases/	operaciones autorizadas
ports/	contratos de persistencia y eventos
data/	
mappers/	dominio <-> representación física
repositories/	acceso por identidad y consultas declaradas
projections/	contratos, writers y rebuilders
connectors/	adaptadores del proveedor
metadata/	
schemas/	tipos y validación
catalogs/	vocabularios y referencias
relationships/	vínculos semánticos
analytics/	
conjuntos de datos/	contratos y manifiestos
características/	definiciones fuera de línea/en línea
lineage/	grafo de procedencia
intelligence/	
retrieval/	fragmentos, embeddings y búsqueda
models/	registro y evaluación
tools/	capacidades gobernadas para agentes
governance/	
policies/	acceso, retención y clasificación
evidence/	auditoría y evidencia

La estructura expresa responsabilidades, no una obligación de crear un archivo por cada elemento. Un sistema pequeño puede agrupar módulos siempre que mantenga las fronteras.

23.4. Entidad canónica

Una implementación mínima contiene identidad, ámbito, revisión, versión de esquema, estado y atributos. El constructor valida forma básica; las transiciones con significado de negocio se realizan mediante métodos o políticas.

```
class CanonicalEntity {
```

```
constructor({ id, ámbito, revision = 0, schemaVersion = 1,
             status = "active", attributes = {} }) {
  requireId(id); requireScope(ámbito);
  this.id = id;
  this.ámbito = ámbito;
  this.revision = revision;
  this.schemaVersion = schemaVersion;
  this.status = status;
  this.attributes = structuredClone(attributes);
}
}
```

El objeto no conoce URL, SDK, tabla, colección ni caché. Esta independencia corresponde al principio de separar dominio y persistencia y se relaciona con Mapeador de datos y Repositorio (Fowler, 2002).

23.5. Mapeador y Repositorio

El Mapeador define correspondencia reversible entre nombres de dominio y campos físicos. El Repositorio ofrece operaciones en lenguaje del dominio y exige ámbito en todas ellas.

```
class EntityMapper {
  toDomain(id, ámbito, crudo) { /* mapeo, valores predeterminados y versión */ }
  toRaw(entity)                { /* representación persistible */ }
}

class EntityRepositorio {
  get(ámbito, id)                { /* lectura por clave */ }
  save(ámbito, entity, revision) { /* control optimista */ }
  listBy(ámbito, query, cursor)  { /* consulta permitida */ }
}
```

El Repositorio no debe devolver el SDK del proveedor ni permitir que la presentación forme rutas. Un Conector traduce sus operaciones a la API concreta.

23.6. Dater como variante de acceso a datos

Los dos casos utilizan el término local *Dater* para clases que convierten objetos, ejecutan CRUD, manejan caché y, en algunos módulos, mantienen índices. El nombre no designa un patrón reconocido distinto: funcionalmente combina responsabilidades de Data Acceso Objeto, Repositorio, Mapeador de datos e Mapa de identidad. COBRIA puede conservar el término por compatibilidad, pero recomienda declarar cuál de esas responsabilidades cumple cada clase.

Cuadro 23.1: Correspondencia del Dater con patrones consolidados.

Responsabilidad observada	Patrón relacionado	Recomendación
CRUD del proveedor	Data Acceso Objeto	Mantener detrás de un contrato.
Obtener entidad por identidad	Repositorio	Usar vocabulario del dominio.
Convertir crudo y objeto	Mapeador de datos	Separar mapeo cuando crece.
Conservar instancia por ID	Mapa de identidad	Definir ámbito e invalidación.
Listas con TTL	Caché	Medir frescura y fallos.
Índices derivados	Vista materializada	Versionar y reconstruir.

23.7. Contrato de proyección ejecutable

```
const ProjectionContract = {
  id: "entity-by-status",
  version: 2,
  sourceType: "Entidad",
  dependencies: [],
  consistency: "eventual",
  transform(entity) { return [[entity.status, entity.id, true]]; },
  verify(source, projection) { /* cobertura y huérfanos */ },
  rebuild({ ámbito, cursor }) { /* lotes y checkpoints */ }
};
```

El contrato reúne propiedades que en las implementaciones aparecen distribuidas entre rutas, métodos de Dater, procesos de trabajo y convenciones. Su versión permite crear una proyección paralela y efectuar cambio de versión.

23.8. Caso de uso y transacción lógica

Un caso de uso coordina autorización, carga, invariantes, persistencia, cola de salida y resultado.

```
UpdateEntity(command, actor):  
  policy.require(actor, "entity.update", command.ámbito)  
  current <- repository.require(command.ámbito, command.id)  
  updated <- domain.apply(current, command.change)  
  repository.save(command.ámbito, updated, current.revision)  
  cola de salida.append(EntityChanged(updated.id, updated.revision))  
  evidence.record(command, actor, updated)  
  return presenter.summary(updated)
```

Si el motor permite actualización multipath atómica, canónico, índices críticos y cola de salida pueden compartir una frontera. Si no, se documenta la compensación y se emplean procesos de trabajo idempotentes.

23.9. Pruebas mínimas

La referencia requiere pruebas de ida y vuelta del Mapeador, aislamiento de ámbito, revisión optimista, idempotencia, reconstrucción desde cero, reconstrucción incremental, detección de huérfanos, cambio de versión y rechazo de escritura directa en una proyección. Para analítica agrega corte temporal, linaje y equivalencia fuera de línea—en línea.

Caso de estudio A: sistema orientado a eventos y contenido

24.1. Caracterización estructural

En la fecha de corte, el inventario del Sistema A encontró aproximadamente 88 archivos de objetos de dominio, 91 Daters, 182 servicios, siete casos de uso explícitos, dos schemas centrales y 102 archivos de catálogos o datos de referencia. Estas cantidades describen una base heterogénea: módulos antiguos conviven con subsistemas recientes más formalizados.

El dominio combina participantes, grupos, sesiones, eventos, acciones evaluadas, planificación, salud, inventario, archivos multimedia, facturación y artefactos de IA. La variedad permite observar si una estructura común se adapta a información temporal, conectada y multimodal.

Cuadro 24.1: Inventario neutral del Sistema A.

Categoría	Conteo	Observación
Objetos de dominio	88	Estilos de modelado de distintas generaciones.
Daters	91	CRUD, caché, serialización e índices.
Servicios	182	Operación, analítica, multimedia e IA.
Casos de uso explícitos	7	Concentrados en flujos recientes.
Esquemas centrales	2	La validación también aparece distribuida.
Catálogos y referencias	102	Vocabularios, configuración y datos compartidos.

24.2. Objetos y serialización

Los objetos observados usan clases JavaScript con constructores, getters, setters y, en varios módulos, métodos de serialización. Algunos emplean campos privados y copias defensivas para estructuras anidadas; otros exponen propiedades públicas y conservan alias para compatibilidad. Esta coexistencia demuestra adaptabilidad, pero también riesgo de semántica desigual.

En los módulos de inteligencia, los objetos representan versiones de modelos, métricas, experimentos, consentimientos, tareas, auditorías, conocimiento, retroalimentación y solicitudes de acción. Esto se alinea con la Parte IV porque el modelo y la decisión no se reducen a un valor suelto: poseen identidad, estado, procedencia y tiempos.

24.3. Daters e índices derivados

Los Daters del Sistema A acceden al conector de datos, convierten cargas útiles, mantienen Mapa locales y actualizan índices por modelo, tipo, estado u otras dimensiones. Algunos construyen una actualización multipath que persiste el registro principal y varias rutas derivadas. Este es un ejemplo concreto del intercambio de COBRIA: una escritura lógica produce varias escrituras físicas para acelerar lecturas.

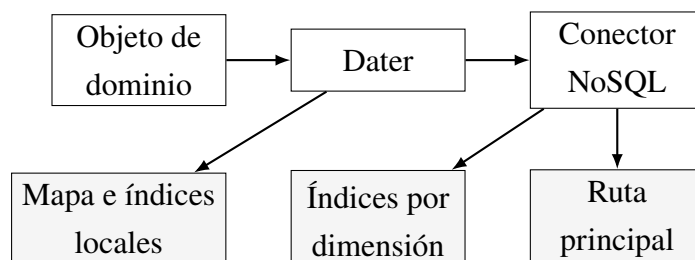


Figura 24.1: Flujo de datos observado y abstraído en el Sistema A.

La ruta principal se aproxima al canónico y los índices a proyecciones, pero esa clasificación debe verificarse por entidad. Un carga útil duplicado puede contener campos que otro módulo modifica directamente; en tal caso existe autoridad compartida y no una proyección COBRIA válida.

24.4. Servicios, casos de uso e IA

La capa de servicios incluye coordinación de eventos, análisis, reportes, sincronización, multimedia, salud, facturación e inteligencia. Los casos de uso explícitos aparecen en operaciones

que requieren transiciones y permisos. En otros módulos, un servicio o Dater todavía concentra la coordinación.

La capacidad de IA es estructuralmente relevante: existen artefactos para versiones, experimentos, métricas, preferencias, consentimiento, conocimiento, auditoría, retroalimentación y acciones propuestas. Esto apoya la adaptabilidad conceptual de COBRIA, pero no prueba exactitud de modelos ni seguridad de agentes. Esas capacidades deben alcanzar E3 o E4 mediante los experimentos de la Parte IV.

24.5. Fortalezas observadas

- separación visible entre presentación, lógica y datos;
- objetos especializados para eventos, multimedia e inteligencia;
- Daters con cachés e índices adecuados para consultas frecuentes;
- actualizaciones multipath en operaciones con derivados;
- servicios para analítica y ciclo de vida de artefactos;
- adopción progresiva de casos de uso en flujos con reglas.

24.6. Deuda y oportunidades

Los principales riesgos son estilos de objetos heterogéneos, validación distribuida, responsabilidades amplias en ciertos Daters, índices sin contrato uniforme, rutas conocidas por múltiples módulos y cobertura desigual de casos de uso. La optimización local mediante Mapa puede perder ámbito o frescura si la clave e invalidación no se declaran.

La migración recomendada no reescribe todos los módulos. Se priorizan entidades con datos sensibles, múltiples índices, IA o alto volumen. Para cada una se registra fuente canónica, contrato de proyección, versión, propietario, reconstrucción y pruebas. Los alias de compatibilidad pueden permanecer en el Mapeador mientras se estabiliza el modelo.

24.7. Evolución longitudinal posterior del Sistema A

Una segunda observación del repositorio, posterior al inventario inicial, permitió analizar cambio arquitectónico y no solo estructura estática. La evolución no se usa como un nuevo

prueba comparativa de rendimiento; constituye evidencia de aplicabilidad de los principios de COBRIA en una migración real y gradual.

Cuadro 24.2: Evidencia longitudinal neutral del Sistema A.

Propiedad observada	Evidencia	Interpretación COBRIA
Frontera Presentación– Data	0 importaciones directos	La dependencia se hace verificable por CI, no solo convencional.
Compatibilidad temporal	134 consumidores inventariados	La deuda se congela y solo puede disminuir.
Vocabulario físico central	36 nodos semánticos	Los nombres compactos quedan detrás de un registro de rutas.
Modelos de lectura explícitos	4 repositorios de vistas	Listados y resúmenes declaran almacén derivado separado.
Pruebas arquitectónicas y de escala	16 artefactos relevantes	Se verifican rutas, concurrencia, vistas, presupuestos y fronteras.

Las nuevas vistas de listas y resúmenes incluyen `complete`, `updatedAt`, TTL y operaciones de reemplazo, invalidación o reconstrucción. Una vista fría agrupa solicitudes concurrentes en una sola reconstrucción; las pruebas comprueban que veinte solicitudes comparten un único warmup. La autorización se ejecuta en cada solicitud contra la política canónica, aun cuando el resultado materializado esté fresco. Este detalle separa optimización de autoridad.

Las operaciones de dominio aplican actualizaciones multipath para canónico, índices, auditoría y notificación; utilizan versiones, transiciones explícitas, idempotencia y controles concurrentes para invariantes. Además, el contexto utilizado por telemetría y FinOps excluye valores no validados provenientes del cliente. Estas decisiones amplían COBRIA con dos reglas: *la optimización puede fallar sin falsificar el canónico y la atribución operacional debe proceder del ámbito autorizado*.

La migración del interfaz aporta otra lección. Una frontera temporal puede conservar 134 consumidores heredado sin permitir nuevos consumidores. El manifiesto y el límite automático convierten el patrón estrangulador en una transición medible. COBRIA debe, por tanto, exigir a cada puente: propietario, inventario, fecha o condición de retiro, presupuesto que no crezca y pruebas de delegación.

24.8. Implicación para la validez del caso

La observación longitudinal fortalece H8 porque el mismo sistema evolucionó hacia rutas semánticas, servicios de aplicación, políticas separadas, proyecciones materializadas y controles automatizados arquitectónicos sin una reescritura total. No demuestra causalidad: los cambios fueron realizados por participantes conocedores de la propuesta y no se compararon contra un equipo de control. Sí proporciona evidencia concreta de que COBRIA puede funcionar como arquitectura evolutiva y no únicamente como taxonomía retrospectiva.

Caso de estudio B: sistema empresarial multiinquilino

25.1. Caracterización estructural

El Sistema B posee una escala estructural considerablemente mayor. En la fecha de corte se encontraron aproximadamente 680 objetos de dominio, 680 Daters, 1,624 servicios, 18 casos de uso explícitos, 686 schemas y 514 archivos de catálogos o datos de referencia. La correspondencia cercana entre objetos, Daters y schemas revela una estrategia sistemática, parte de ella apoyada por generación o convenciones repetibles.

Cuadro 25.1: Inventario neutral del Sistema B.

Categoría	Conteo	Observación
Objetos de dominio	680	Cobertura de numerosos subdominios.
Daters	680	Correspondencia casi uno a uno con objetos.
Servicios	1,624	Políticas, procesos de trabajo, contratos y orquestadores.
Casos de uso explícitos	18	Operaciones críticas seleccionadas.
Esquemas	686	Validación estructural extensa.
Catálogos y referencias	514	Vocabularios y configuración por dominio.

Estos números no permiten concluir que todo módulo esté integrado o probado. Sí ofrecen evidencia de una arquitectura dirigida por estructura y metadatos, adecuada para estudiar

escalabilidad organizacional.

25.2. Modelo base y mapeo físico

Un modelo base representativo inicializa identidad y valores predeterminados, restringe campos desconocidos, serializa a JSON y mantiene un mapa explícito entre campo físico y campo de dominio. El método inverso reconstruye una instancia desde datos crudo. Esta forma materializa Mapeador de datos de manera directa.

La ventaja es que una entidad puede conservar vocabulario estable aunque cambien claves físicas. El riesgo aparece cuando valores predeterminados silenciosos ocultan ausencia o versiones antiguas. COBRIA recomienda distinguir campo ausente, nulo y vacío y registrar la versión del esquema utilizada en la reconstrucción.

25.3. Dater base

El Dater base observado recibe ruta, modelo, conector y TTL. Implementa lectura por ID, alta, actualización, borrado, lista, paginación por cursor, consulta por hijo, carga múltiple, escucha en tiempo real, caché de entidades y caché de listas. Convierte entre crudo y objeto mediante el modelo.

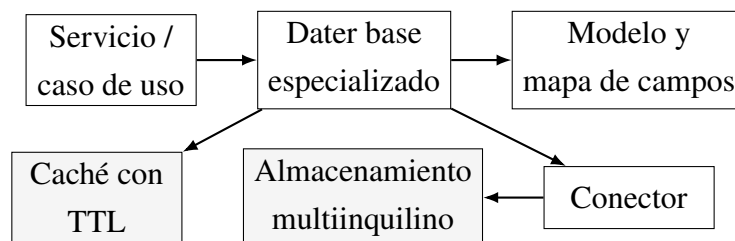


Figura 25.1: Abstracción del acceso común observado en el Sistema B.

Esta base reduce duplicación y ofrece una API uniforme. Sin embargo, el argumento de ruta debe incorporar ámbito de forma no opcional. Una ruta base global o una clave de caché basada solo en ID podría romper aislamiento aunque el resto de la aplicación conozca el organización.

25.4. Metadatos, relaciones y catálogos

El Sistema B contiene registries para políticas de campo, relaciones, clasificaciones de referencias y correspondencia con catálogos. Los campos pueden declararse configurables,

externos o asociados con vocabularios. Esta capa convierte reglas que normalmente quedarían dispersas en datos inspeccionables.

La arquitectura dirigida por metadatos favorece generación de objetos, schemas, Daters, documentación y controles de interfaz. Para que la generación sea confiable debe existir una fuente primaria del metamodelo, validación del generador y detección de divergencia. Editar artefactos generados manualmente sin retroalimentar el metamodelo crea deriva.

25.5. Servicios y operaciones críticas

Los servicios abarcan consulta, disponibilidad, inventario, finanzas, comunicaciones, integraciones, privacidad, sincronización, reportes y operaciones multiinquilino. Existen contratos de repositorio, procesos de trabajo, políticas, state machines y orquestadores. Los casos de uso explícitos se concentran en operaciones donde estados, permisos y efectos derivados requieren una frontera clara.

Los contratos operacionales muestran ámbito de organización y unidad, claves de idempotencia, autenticación, recálculo en servidor, rate limits y errores semánticos. Estos mecanismos coinciden con COBRIA Núcleo y Gobierno. La persistencia de cola de salida, operaciones idempotentes, versiones de esquema, bloqueos y trabajos programados ofrece puntos naturales para reconstrucción y observabilidad.

25.6. Fortalezas observadas

- correspondencia sistemática entre objeto, Dater y esquema;
- modelo base con mapeo entre dominio y representación física;
- Dater base con caché, paginación y tiempo real;
- metadatos explícitos para campos, relaciones y catálogos;
- contratos de repositorio y casos de uso en operaciones críticas;
- idempotencia, cola de salida, procesos de trabajo y controles multiinquilino modelados;
- suficiente variedad de dominios para evaluar extensibilidad.

25.7. Deuda y oportunidades

La escala produce riesgos de proliferación: una clase por estructura puede aumentar costo de navegación, generación y pruebas. Una correspondencia uno a uno no garantiza que cada estructura posea identidad o comportamiento suficiente para ser entidad. Los 1,624 servicios pueden incluir políticas pequeñas y artefactos generados; deben agruparse por responsabilidad y dependencia antes de inferir complejidad.

COBRIA recomienda clasificar cada tipo como entidad, value object, evento, comando, proyección o catálogo. También propone registrar responsabilidad de proyecciones, reconstrucción y presupuesto. La base común debe exigir ámbito, revisión y versión; los servicios deben depender de contratos, no de rutas ni SDKs.

Comparación entre casos y derivación de COBRIA

26.1. Similitudes estructurales

Ambos sistemas separan presentación, lógica y datos; representan el dominio mediante objetos; encapsulan persistencia en Daters; utilizan servicios; conservan catálogos y datos compartidos; y desnormalizan para responder consultas. Ambos poseen conectores a un almacenamiento NoSQL y cachés locales. Estas similitudes motivaron la búsqueda de un vocabulario común.

Cuadro 26.1: Comparación neutral de los casos.

Dimensión	Sistema A	Sistema B
Dominio	Eventos, secuencias, multimedia e inteligencia.	Operaciones empresariales multiinquilino de gran amplitud.
Objetos	Heterogéneos y especializados.	Sistemáticos, con modelo base frecuente.
Daters	Especializados; varios mantienen índices propios.	Base reutilizable y amplia correspondencia por entidad.
Esquemas	Parciales o distribuidos.	Cobertura estructural extensa.
Metadatos	Catálogos y configuraciones por módulo.	Registries de campos, relaciones y catálogos.
Casos de uso	Adopción focalizada en flujos recientes.	Adopción focalizada en operaciones críticas.

Dimensión	Sistema A	Sistema B
Proyecciones	Índices locales/remotos y artefactos analíticos.	Índices, reportes, cachés, cola de salida y snapshots.
IA	Componentes explícitos de versión, auditoría y retroalimentación.	Preparación estructural, automatización y datos empresariales.
Riesgo principal	Heterogeneidad y contratos implícitos.	Proliferación y divergencia entre artefactos.

26.2. Diferencias que prueban adaptabilidad

El Sistema A necesita representar tiempo fino, acciones, secuencias y archivos. El Sistema B necesita ámbito jerárquico, catálogos configurables, operaciones idempotentes y muchas relaciones. COBRIA no intenta igualar sus modelos. La adaptación ocurre porque ambos pueden expresar identidad, ámbito, versión, mapeo y proyección, pero emplean contratos diferentes.

En A, una proyección puede indexar eventos por participante, contexto o ventana y un pipeline puede generar clips o características. En B, una proyección puede resolver disponibilidad, saldos, reportes o búsquedas por organización. El mecanismo es común; la semántica pertenece al dominio.

26.3. Elementos heredados y contribución

Los objetos, DAO/Repositorio, Mapeador de datos, Capa de servicios, casos de uso, caché, catálogos, vistas materializadas, cola de salida y metadatos son conceptos existentes. El Dater es un nombre local para una combinación de esos patrones, no una nueva categoría científica.

La contribución COBRIA se ubica en la integración de estas ideas alrededor de:

1. una fuente canónica con identidad, ámbito, revisión y versión;
2. una separación explícita entre dominio y forma física;
3. proyecciones con contrato, responsabilidad, consistencia y reconstrucción;
4. metadatos que conectan modelo, esquema, relaciones y catálogos;
5. continuidad de linaje desde operación hasta conjunto de datos, modelo y decisión;

6. una frontera que impide a derivados o agentes adquirir autoridad por accidente.

26.4. Patrones de diseño identificados

Cuadro 26.2: Patrones observados y papel dentro de COBRIA.

Patrón	Evidencia típica	Papel COBRIA
Repositorio/DAO	Daters y contratos de repositorio	Aislar persistencia.
Mapeador de datos	fromRaw, toRaw y mapas de campos	Separar semántica y forma física.
Mapa de identidad	Mapas por identidad	Evitar instancias duplicadas en sesión.
Capa de servicios	Servicios de dominio y aplicación	Coordinar operaciones.
Caso de uso	Flujos explícitos y autorizados	Definir frontera transaccional.
Vista materializada	Índices, resúmenes y snapshots	Optimizar lectura con reconstrucción.
CQRS parcial	Modelos de consulta especializados	Separar necesidades de lectura.
Cola de salida	Eventos pendientes y procesos de trabajo	Reducir ventana de pérdida.
Registro	Metadatos y catálogos	Centralizar contratos inspeccionables.
Estrategia y política	Validadores y políticas intercambiables	Variar reglas sin rutas físicas.
Adaptador/Pasarela	Conectores y proveedores externos	Encapsular infraestructura.
Máquina de estados	Transiciones de operaciones críticas	Restringir ciclos de vida.

26.5. Modelo de madurez

Cuadro 26.3: Niveles de madurez COBRIA para una entidad.

Nivel	Nombre	Evidencia
M0	Acceso directo	interfaz de usuario o servicio conoce rutas y cargas útiles.
M1	Encapsulado	Objeto y Dater/Repositorio básico.
M2	Canónico	Identidad, ámbito, mapeo y esquema explícitos.
M3	Reconstruible	Proyecciones versionadas con rebuild y verificación.
M4	Gobernado	Linaje, políticas, observabilidad y recuperación.
M5	Adaptativo	Optimización basada en evidencia con aprobación.

La madurez se evalúa por entidad o flujo, no por sistema completo. Ambos casos pueden contener simultáneamente módulos M1 y M4. Esta granularidad permite una migración basada en riesgo.

Estrategia de adopción y migración

27.1. Adopción incremental

La estrategia recomendada sigue siete pasos:

1. seleccionar una entidad con dolor verificable;
2. declarar identidad, ámbito, autoridad e invariantes;
3. aislar Mapeador y Repositorio o documentar el Dater existente;
4. inventariar copias, índices, cachés y consumidores;
5. crear contratos de proyección y verificación;
6. instrumentar lecturas, escrituras, anomalías y costo;
7. reconstruir en paralelo, comparar y efectuar cambio de versión reversible.

No se recomienda renombrar masivamente los Daters. La migración comienza por contratos y propiedades verificables. El nombre puede cambiar cuando aporte claridad y no rompa consumidores.

27.2. estrangulador para datos

Una fachada compatible intercepta operaciones antiguas y las dirige al nuevo Repositorio. Durante la transición puede comparar lecturas o emitir eventos para nuevas proyecciones. La escritura dual sin verificación aumenta riesgo; se prefiere una fuente autoritativa y un derivado reconstruible.

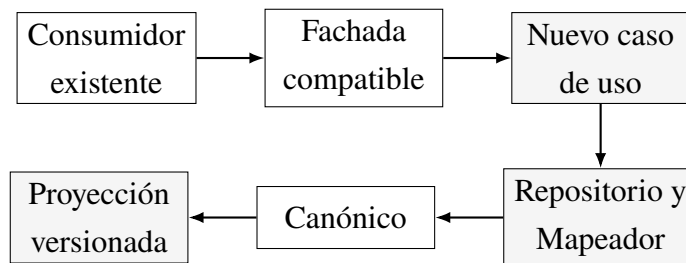


Figura 27.1: Migración incremental mediante una fachada compatible.

27.3. Priorización por riesgo y valor

Se asigna una puntuación basada en sensibilidad, frecuencia, amplificación, número de copias, incidentes, dificultad de reconstrucción y consumidores. Las entidades con alto riesgo y alto uso se atienden primero. Un catálogo estático y estable puede permanecer en M1 o M2 si no existe beneficio neto de mayor formalidad.

27.4. Generación dirigida por metadatos

En el Sistema B, la escala hace atractiva la generación. El metamodelo puede producir esqueleto de entidad, esquema, Mapeador, Dater/Repositorio, documentación y pruebas de contrato. La generación debe ser determinista; el repositorio rechaza divergencias entre salida generada y archivos versionados.

El generador no decide invariantes de negocio ni crea proyecciones por cada campo. Esas decisiones requieren demanda de consulta, responsable y presupuesto.

27.5. Compatibilidad y versionado

Los cambios se clasifican como aditivos, deprecatorios o incompatibles. Un Mapeador puede leer varias versiones y escribir solo la vigente. La migración se ejecuta por lotes y con checkpoint. Las proyecciones incluyen versión en ruta o clave para permitir coexistencia y reversión.

27.6. Presupuesto de deuda y retiro de puentes

Toda compatibilidad temporal se registra en un manifiesto reproducible. Sea L_t el conjunto de consumidores heredado permitidos en la versión t . La regla de integración continua exige:

$$L_{t+1} \subseteq L_t.$$

Un consumidor nuevo requiere migración al contrato vigente, no ampliación silenciosa del manifiesto. El puente solo reexporta o adapta; no recibe nuevas reglas de negocio. Su eliminación se realiza por dominio, con pruebas de comportamiento y reducción del presupuesto después de cada bloque estable.

27.7. Criterio de finalización

Una entidad se considera migrada cuando: toda escritura autorizada atraviesa casos de uso definidos; la fuente canónica es única; el ámbito es obligatorio; el ida y vuelta del Mapeador está probado; cada derivado posee contrato y responsable; una reconstrucción completa produce el resultado esperado; la telemetría permite medir costo y anomalías; y los controles automatizados impiden reintroducir rutas físicas, dependencias invertidas o consumidores heredado.

Reproducibilidad, limitaciones y síntesis de la Parte V

28.1. Paquete reproducible previsto

El manuscrito debe acompañarse de un paquete que no contenga los proyectos privados. El paquete incluirá implementación neutral, generadores sintéticos, contratos de ejemplo, scripts de inventario, pruebas, configuración del prueba comparativa y plantillas de resultados. Cada versión publicada fijará versiones y resúmenes criptográficos.

Cuadro 28.1: Contenido del paquete reproducible.

Artefacto	Contenido
Referencia	Núcleo, Repositorio, Mapeador, Gestor de proyecciones y Gobierno.
Datos sintéticos	Perfiles A y B con semilla y escalas configurables.
Contratos	Entidades, proyecciones, conjuntos de datos y herramientas.
Migraciones	Total, incremental, cambio de versión y reversión.
Pruebas	Aislamiento, idempotencia, ida y vuelta y reconstrucción.
Prueba comparativa	Cargas, instrumentación y análisis estadístico.
Documentación	Diccionario, decisiones y guía de réplica.

28.2. Limitaciones del análisis estático

El inventario no demuestra qué archivos se ejecutan en producción, qué rutas contienen datos, cuántos módulos tienen consumidores ni qué cobertura alcanzan las pruebas. Los conteos pueden incluir archivos generados, contratos pequeños o capacidades parciales. La

documentación puede estar desactualizada. Estas limitaciones se resolverán en la Parte VI mediante ejecución controlada, trazas y experimentos.

28.3. Riesgo de sesgo del investigador

El investigador es también autor o conocedor de los sistemas, lo que facilita acceso y comprensión, pero puede favorecer interpretaciones positivas. Se mitigará publicando criterios previos, scripts, resultados negativos, niveles de evidencia y una implementación neutral. Cuando sea posible, otro evaluador revisará la clasificación de una muestra de módulos.

28.4. Validez de la generalización

Los casos comparten lenguaje y almacenamiento NoSQL, por lo que no prueban adaptación a todos los ecosistemas. La contribución generalizable es el mecanismo y sus condiciones, no los nombres de carpetas ni las cantidades. Una réplica futura debe incluir otro lenguaje, otro motor documental y un equipo independiente.

28.5. Síntesis de la Parte V

Esta parte conectó la formalización de COBRIA con evidencia estructural de dos sistemas reales y distintos. El Sistema A mostró objetos heterogéneos, Daters con índices y una amplia superficie de eventos, multimedia e inteligencia. El Sistema B mostró una correspondencia masiva entre objetos, Daters y schemas, un modelo base con mapeo físico, metadatos y operaciones multiinquilino más sistemáticas.

El análisis confirmó que “Objeto–Dater–Servicio–Catálogo” no constituye por sí solo un patrón nuevo. Es una composición local de Modelo de dominio, DAO/Repositorio, Mapeador de datos, Capa de servicios, Registro, Caché y vistas materializadas. COBRIA adquiere interés de investigación al convertir esa composición en una arquitectura explícita, formal y evaluada alrededor de autoridad canónica, proyecciones reconstruibles, ámbito y linaje hacia IA.

También se definió una implementación neutral, un modelo de madurez y una estrategia de migración incremental. Los conteos estructurales sirven para caracterizar, no para celebrar tamaño. La siguiente parte ejecutará el protocolo: medirá rendimiento, reconstrucción, aislamiento, evolución, linaje y adaptabilidad, y presentará tanto resultados favorables como negativos.

Parte VI

Evaluación experimental, análisis y validez

Diseño de evaluación exclusivamente NoSQL

29.1. Delimitación tecnológica

Esta edición evalúa COBRIA únicamente dentro de sistemas NoSQL documentales. La unidad de persistencia es el documento u objeto agregado; las consultas se expresan mediante claves, filtros, índices del motor, colecciones derivadas y flujos de cambios. No se transfieren cifras obtenidas con motores de otro paradigma. Esta decisión reduce el volumen de evidencia cuantitativa disponible, pero alinea la inferencia con el objeto real de investigación.

La evaluación distingue tres niveles. El nivel estructural verifica contratos en código y configuración. El nivel ejecutable usa emuladores y entornos NoSQL aislados. El nivel empírico mide latencia, costo, frescura y recuperación en condiciones reproducibles. Una afirmación no asciende de nivel sin la evidencia correspondiente.

29.2. Casos documentales anonimizados

Se estudian tres aplicaciones con dominios distintos: eventos deportivos y multimedia, operación hotelera y operación comercial. Sus nombres se omiten. Las tres emplean documentos, colecciones, autenticación, funciones y almacenamiento administrado. Se buscan contratos repetidos sin revelar organizaciones, personas ni credenciales.

Cuadro 29.1: Perfiles NoSQL utilizados en la evaluación.

Perfil	Carga dominante	Riesgo arquitectónico observado
N1	eventos, resultados y multimedia	derivados obsoletos y reconstrucción costosa
N2	reservas, disponibilidad y catálogos	conurrencia, tiempo y aislamiento
N3	inventario, ventas y cierres	agregados operacionales y trazabilidad

Hipótesis, variables y protocolo

30.1. Hipótesis activas

- HN1. Una proyección declarada por contrato puede reconstruirse desde documentos canónicos sin convertirse en autoridad paralela.
- HN2. El ámbito obligatorio impide recuperar documentos o fragmentos de otra organización antes de calcular similitud o construir contexto.
- HN3. Revisión, idempotencia y publicación por versión reducen duplicados, regresiones y estados parcialmente visibles.
- HN4. Un conjunto analítico versionado puede reproducirse desde fuentes, catálogos y transformaciones declaradas.
- HN5. Una proyección solo se justifica si mejora un objetivo de lectura y su mantenimiento permanece dentro de un presupuesto explícito.

HN1–HN4 admiten verificación funcional. HN5 necesita mediciones específicas por motor, región, índice, escala y mezcla de operaciones. Esta edición no publica un umbral universal de lecturas por escritura.

30.2. Variables y unidades

Las variables independientes son motor documental, perfil, escala, número de ámbitos, forma del documento, política de consistencia, estrategia de reconstrucción y nivel de concurrencia. Las dependientes incluyen p50, p95 y p99; documentos y bytes leídos; operaciones facturables;

frescura; duplicados; divergencia; duración de reconstrucción; violaciones de ámbito; y exactitud del conjunto analítico.

La corrida completa es la unidad inferencial. Cada manifiesto registra semilla, versión de código, configuración, región, índices, calentamiento, orden de cargas y criterio de descarte. Las enmiendas se publican antes de observar el resultado que podrían favorecer.

Experimentos NoSQL

31.1. P1: lectura selectiva documental

P1 compara la consulta canónica indexada con una proyección de cobertura sobre el mismo conjunto lógico. Ambas configuraciones usan el mismo motor NoSQL y los mismos documentos de entrada. La proyección gana únicamente si reduce el percentil objetivo o el costo facturable sin alterar resultados, ámbito ni frescura más allá del límite acordado.

31.2. P2: amplificación de escritura

P2 actualiza documentos canónicos y registra operaciones adicionales, tamaño de eventos, invocaciones, reintentos y tiempo hasta convergencia. El resultado es un vector de costo, no una sola cifra. Una reducción de lectura no compensa automáticamente mayor complejidad.

31.3. R1: reconstrucción total e incremental

R1 elimina una versión derivada en un entorno controlado, reconstruye una candidata, verifica equivalencia y publica mediante un cambio de versión. La variante incremental procesa revisiones posteriores a un punto de control. Ambas rutas deben ser idempotentes, conservar procedencia y producir el mismo resultado lógico.

31.4. S1: aislamiento multiinquilino

S1 genera identidades, documentos y consultas para múltiples ámbitos. Incluye ausencia de ámbito, ámbito manipulado, claves de caché incompletas, búsqueda semántica y tareas

administrativas. La aceptación exige rechazo temprano y cero documentos cruzados. El filtro posterior a la recuperación no satisface el contrato.

31.5. A1: analítica e inteligencia artificial

A1 reconstruye un conjunto fechado, publica variables versionadas y ejecuta recuperación autorizada. Cada fragmento conserva fuente, ámbito, revisión, versión de transformación y política. Se miden precisión, exhaustividad, abstención, deriva y fugas. La evaluación de recuperación no se presenta como evaluación de un modelo generativo.

Evidencia y resultados responsables

32.1. Evidencia disponible

Los tres perfiles exhiben objetos canónicos, capas de acceso documental, servicios, catálogos y representaciones derivadas. La recurrencia apoya la relevancia práctica del problema, no la novedad de cada mecanismo. La contribución de COBRIA está en convertir esa recurrencia en contratos comparables: autoridad, ámbito, revisión, tiempo, versión, equivalencia y reconstrucción.

Las pruebas disponibles cubren reglas de acceso, matrices de roles, aislamiento, ciclos de vida, idempotencia y reconstrucción. También existe evidencia longitudinal de menor acceso directo desde presentación hacia la capa documental. Estos resultados prueban rutas concretas, pero no productividad humana ni superioridad general.

32.2. Resultados COBRIA Core 1.0

La primera réplica congelada se ejecutó sobre dos motores documentales embebidos, PouchDB y LokiJS, con escalas de 100, 1,000 y 5,000 documentos y treinta corridas por celda. En total se conservaron 180 corridas. Todas las celdas superaron conjuntamente P1, P2, R1, S1 y A1: equivalencia lógica completa, reconstrucción repetible, amplificación de dos escrituras por cambio proyectado, rechazo de ámbito ausente y cero documentos cruzados.

En PouchDB las medianas de lectura canónica fueron 1,186; 13,589 y 69,291 ms, frente a 0,440; 4,719 y 23,131 ms para la proyección. Las reconstrucciones medianas requirieron 8,376; 89,301 y 500,070 ms. En LokiJS las lecturas canónicas fueron 0,145; 1,263 y 7,026 ms, frente a 0,052; 0,409 y 2,220 ms; R1 requirió 0,663; 6,452 y 46,011 ms.

Estos resultados apoyan conformidad funcional y muestran una reducción de lectura en este arnés embebido. No representan redes, regiones, concurrencia administrada, cuotas ni

facturación. HN1–HN4 quedan apoyadas para esta implementación de referencia; HN5 recibe apoyo local condicionado y permanece abierta para servicios administrados.

Amenazas a la validez y síntesis

Los casos comparten ecosistema y autoría. Los emuladores no reproducen cuotas, particiones, latencia regional ni facturación administrada. Los datos anonimizados y sintéticos pueden subrepresentar deriva y errores históricos. La inspección no sustituye ejecución; las pruebas automáticas no sustituyen revisión humana; y una implementación correcta no demuestra comprensión por todos los equipos.

La evidencia permite afirmar que COBRIA Core 1.0 es formalizable, ejecutable y conforme en dos motores NoSQL documentales embebidos. No permite afirmar una mejora universal de rendimiento ni un umbral económico transferible. El siguiente nivel de evidencia debe provenir de servicios documentales administrados y una réplica externa.

Parte VII

Síntesis, adopción y conclusiones

Síntesis integral de COBRIA

COBRIA organiza decisiones que aparecen cuando un sistema NoSQL conserva documentos operacionales y necesita búsquedas, resúmenes, conjuntos analíticos y herramientas de inteligencia artificial. Existe una fuente canónica por hecho de dominio y toda representación derivada declara cómo se obtiene, verifica, delimita y reconstruye.

34.1. Respuesta a la investigación

La estructura Objeto–Dater–Servicio–Catálogo no constituye por sí sola una teoría nueva. COBRIA aporta una composición explícita para documentos NoSQL: identidad y revisión en el canónico; transformación y procedencia en el Dater; reglas y autorización en servicios; semántica versionada en catálogos; y reconstrucción en cada proyección.

La reducción de lectura depende de consulta, índice, forma documental y distribución. Una proyección se adopta después de compararla con la consulta canónica indexada. La amplificación se registra mediante operaciones, bytes, eventos, reintentos, almacenamiento y trabajo operacional.

La adaptabilidad hacia minería de datos e inteligencia artificial procede del linaje: conjuntos, características, fragmentos e inferencias conservan fuente, tiempo, versión y ámbito. Esto permite reconstruir un corte, detectar deriva y evitar que recuperación o agentes crucen fronteras de autorización.

Guía de adopción

Considérese COBRIA cuando existen múltiples consumidores, documentos heterogéneos, aislamiento organizacional, evolución frecuente, derivados costosos de reparar o productos de datos reproducibles. Simplifíquese cuando una colección indexada satisface el objetivo o el costo de gobierno supera el riesgo.

1. identificar la escritura autoritativa y cerrar rutas paralelas;
2. hacer obligatorio el ámbito en acceso, caché, eventos y recuperación;
3. añadir revisión, tiempo y versión de esquema;
4. medir la consulta antes de crear una proyección;
5. declarar transformación, equivalencia y reconstrucción;
6. publicar por versión y practicar reparación;
7. extender procedencia hacia analítica e inteligencia artificial;
8. retirar derivados sin beneficio o responsable.

Contribuciones y límites

La contribución conceptual es el dato reconstruible como contrato. La formal es la tupla que une fuente, transformación, ámbito, versión, equivalencia y reparación. La arquitectónica es una frontera coherente desde documentos hasta productos de IA. La práctica es un catálogo, listas de comprobación y protocolo reproducible.

La contribución empírica conserva dos estratos. El primero contiene 180 corridas locales históricas sobre PouchDB y LokiJS. El segundo contiene 270 corridas distribuidas históricas sobre MongoDB 8.0, CouchDB 3.4 y OpenSearch 2.19.3. Todas superaron P1, P2, R1, S1 y A1 bajo sus oráculos originales y no registraron fugas de ámbito. Como una auditoría posterior fortaleció el arnés, estas cifras describen evidencia histórica y no certificación vigente. La repetición preliminar corregida completó casos pequeños en MongoDB y CouchDB; OpenSearch 3.2.0 quedó pendiente por incompatibilidad de plataforma.

Falta repetir las celdas distribuidas completas con el arnés corregido; ejecutar OpenSearch 3.2.0 en ARM nativo o infraestructura compatible; realizar una réplica independiente; completar el doble cribado bibliográfico; y ejecutar el estudio humano de transferencia. P1, P2, R1, S1 y A1 forman el conjunto de criterios de conformidad, pero la certificación independiente permanece pendiente.

Conclusión

COBRIA hace visibles seis preguntas: qué documento manda, a qué ámbito pertenece, qué versión representa, qué copia puede perderse, cómo se reconstruye y cuánto cuesta conservarla. Al extenderlas a conjuntos de datos, recuperación e inferencias, ofrece un puente entre ingeniería NoSQL, minería de datos e inteligencia artificial.

La propuesta es suficientemente concreta para implementarse y refutarse. La réplica embebida apoya sus contratos nucleares, mientras la evaluación administrada y humana sigue abierta. Esa frontera define con honestidad qué se sabe y qué debe venir después.

Parte VIII

Manual de réplica y operación NoSQL

Propósito del manual de réplica

Esta parte traduce las hipótesis de COBRIA a un procedimiento ejecutable en motores NoSQL documentales. Su función no es imponer un proveedor, sino impedir que cada réplica redefina silenciosamente la unidad de datos, el índice, el ámbito o el criterio de equivalencia. Una comparación solo es interpretable cuando las configuraciones responden la misma pregunta lógica y registran las diferencias físicas relevantes.

38.1. Resultado esperado

Al terminar una réplica, un tercero debe poder reconstruir cuatro productos: el conjunto de documentos de entrada, las configuraciones comparadas, el registro de cada corrida y el informe que enlaza cifras con observaciones originales. También debe poder comprobar qué se excluyó, por qué se excluyó y si una enmienda ocurrió antes o después de conocer el resultado afectado.

38.2. Principios de neutralidad

1. la semántica del dominio permanece igual entre motores;
2. cada adaptador usa capacidades documentales normales del motor evaluado;
3. los índices se declaran y congelan antes de medir;
4. las proyecciones contienen únicamente los campos necesarios para la consulta;
5. costo, frescura y reparación se informan junto con la latencia;
6. una configuración que pierde permanece en el conjunto publicado.

Selección y descripción de motores documentales

39.1. Criterios de inclusión

Un motor es elegible cuando almacena documentos u objetos agregados, permite claves y filtros con ámbito, ofrece índices declarables, admite operaciones de actualización y posee una vía reproducible local o administrada. La réplica debe incluir al menos dos motores con modelos operacionales distintos para evitar que una capacidad particular se confunda con una propiedad general de COBRIA.

39.2. Ficha del motor

Cada adaptador publica versión, modalidad de despliegue, región, consistencia observable, límites de documento, estrategia de partición, índices, paginación, transacciones documentales disponibles, flujos de cambios, cuotas y unidad de facturación. Si una capacidad no existe, el protocolo registra la adaptación en vez de simular que todos los motores son idénticos.

Cuadro 39.1: Campos mínimos de la ficha del motor NoSQL.

Campo	Pregunta de control
Identidad de versión	¿Qué versión exacta o fecha del servicio se evaluó?
Distribución	¿Cómo se asignan documentos a particiones o regiones?
Consistencia	¿Qué garantía observa la consulta medida?
Índices	¿Qué campos, orden y ámbito cubre cada índice?
Costo	¿Qué operaciones, bytes y almacenamiento se facturan?
Recuperación	¿Cómo se exportan, reconstruyen y verifican documentos?

Modelo documental neutral

40.1. Documento canónico

El documento de referencia contiene identidad estable, ámbito, revisión, versión de esquema, tiempo de validez, tiempo de registro, estado y carga variable. Los campos opcionales se distribuyen según tasas congeladas. Las claves no incluyen información personal y se derivan de la semilla del generador.

40.2. Proyección de cobertura

La proyección conserva identidad, ámbito, revisión, campos de filtro, orden y salida de la consulta objetivo. No contiene reglas de negocio ni acepta escrituras autoritativas. Cada elemento enlaza documento fuente y versión de transformación. Una variante más ancha se permite como análisis de sensibilidad, pero se publica como configuración distinta.

40.3. Catálogos y evolución

Los datos incluyen versiones antiguas, valores desconocidos y sustituciones de catálogo. La transformación debe migrar o aislar cada caso según una política declarada. El generador conserva la verdad esperada para que la limpieza pueda evaluarse sin inspección manual de todas las filas lógicas.

Generación de cargas semirrealistas

41.1. Perfiles

El perfil N1 representa eventos y multimedia; N2, reservas y disponibilidad; N3, inventario, ventas y cierres. Cada perfil modifica selectividad, tamaño, frecuencia de actualización, eventos tardíos y campos opcionales. Las diferencias deben afectar la forma de la carga sin revelar nombres ni reglas de los sistemas que la inspiraron.

41.2. Anomalías controladas

Cuadro 41.1: Anomalías que debe producir el generador.

Anomalía	Parámetro	Resultado esperado
Campo ausente	tasa por perfil	valor predeterminado o exclusión declarada
Documento antiguo	versión de esquema	migración determinista
Evento tardío	retraso máximo	corte temporal reproducible
Entrega duplicada	identidad lógica	un solo efecto
Catálogo desconocido	tasa de novedad	cuarentena observable
Revisión obsoleta	distancia de revisión	rechazo sin regresión

La semilla gobierna identidades, orden y anomalías. Cambiarla produce otra muestra, no otra definición del experimento. Toda tasa se registra en el manifiesto y se incluye en el

identificador de configuración.

Protocolo P1: lectura documental

42.1. Configuraciones

N0 consulta documentos canónicos usando el índice nativo apropiado. N1 consulta una proyección de cobertura reconstruible. N2, opcional, evalúa una proyección más ancha para estimar el costo de duplicar campos. No se utiliza una consulta deliberadamente sin índice como adversario principal, porque la pregunta científica es si la proyección mejora una implementación NoSQL razonable.

42.2. Ejecución

Cada corrida prepara un conjunto nuevo o restaura una instantánea congelada, verifica conteos lógicos, calienta según protocolo y rota el orden de configuraciones. Se registran latencias por consulta, documentos examinados cuando el motor lo permite, bytes de respuesta, operaciones facturables y marca de frescura. La corrida se resume mediante p50, p95 y p99.

42.3. Interpretación

Una diferencia pequeña de latencia puede ser irrelevante si la proyección duplica datos sensibles, aumenta costo o incumple frescura. El informe presenta un vector de efectos. La recomendación se formula por región de carga y nunca como superioridad universal.

Protocolo P2: escritura y convergencia

43.1. Camino de actualización

P2 modifica el documento canónico con revisión esperada. El mecanismo produce o detecta el cambio, actualiza el derivado de forma idempotente y publica la nueva marca de frescura. Se miden tiempo de confirmación canónica y tiempo de convergencia por separado.

43.2. Amplificación

La amplificación incluye documentos escritos, eventos, reintentos, bytes, invocaciones, lecturas auxiliares, almacenamiento y tiempo operacional. Una implementación que oculta trabajo en una función o canal de cambios no posee amplificación cero: debe atribuir ese costo a la configuración.

43.3. Frontera de decisión

Sea ΔQ el ahorro por lectura y ΔW el costo adicional por escritura. La frontera temporal simple exige $Q\Delta Q > W\Delta W$. El informe añade costo monetario, almacenamiento, frescura y riesgo como dimensiones separadas. Si $\Delta Q \leq 0$, la proyección no se justifica por esa consulta.

Protocolos de reconstrucción y publicación

44.1. Reconstrucción total

R1 crea una versión candidata desde documentos canónicos, catálogos y transformación congelada. La candidata no es visible para consumidores hasta superar conteo, hash por partición, muestreo de contenido e invariantes de ámbito. La publicación cambia un alias o registro de versión y conserva la anterior durante una ventana reversible.

44.2. Reconstrucción incremental

La ruta incremental consume cambios posteriores a un punto de control. Cada lote registra inicio, fin, documentos procesados, duplicados y revisión máxima. Una reconstrucción total periódica actúa como verificador independiente. Si ambas rutas divergen, la incremental se considera defectuosa aunque la consulta parezca correcta.

44.3. Interrupción segura

El arnés interrumpe preparación, escritura parcial, verificación y cambio de versión. En ningún caso una candidata incompleta debe reemplazar la versión activa. La reanudación puede continuar o reiniciar, pero siempre produce el mismo resultado lógico.

Aislamiento, autorización y recuperación

45.1. Matriz adversarial

La prueba combina identidad válida, identidad ausente, ámbito manipulado, rol insuficiente, clave de caché incompleta, tarea administrativa y recuperación semántica. Los candidatos se restringen antes de ordenar o construir contexto. Un resultado ocultado después de recuperarlo ya constituye una violación del contrato.

45.2. Pruebas negativas

Cada ruta positiva exige una negativa equivalente. Se verifican identificadores de otro ámbito, referencias cruzadas, eventos reinyectados, paginación y búsqueda por campos opcionales. Los registros de auditoría evitan contenido sensible, pero conservan motivo, política, actor, ámbito y correlación.

Productos para minería de datos e inteligencia artificial

46.1. Conjunto reproducible

El conjunto declara población, corte temporal, fuente, versiones, limpieza, catálogos, exclusiones, semilla y división. Su identificador cambia cuando una dependencia cambia. La réplica reconstruye el conjunto desde cero y compara manifiesto, conteos, distribución y huellas de contenido.

46.2. Recuperación con evidencia

Cada fragmento enlaza documento, revisión, ámbito, transformación y autorización. Se miden precisión, exhaustividad, abstención, fugas y citas válidas. Las instrucciones contenidas en documentos se tratan como datos, no como órdenes. La calidad lingüística de una respuesta requiere un estudio separado.

46.3. Herramientas de agentes

Una herramienta declara capacidad, esquema de entrada, validación, idempotencia, efecto, confirmación y auditoría. Las pruebas incluyen repetición, entrada parcial, cambio de ámbito y cancelación. Una acción de alto riesgo sin confirmación falla el protocolo aun si el resultado funcional parece correcto.

Observabilidad y costos

47.1. Indicadores

El tablero mínimo contiene latencia, error, frescura, atraso, duplicados, rechazos de revisión, reconstrucciones, divergencia, operaciones por solicitud, almacenamiento y costo por ámbito. Los indicadores se segmentan por versión de proyección para detectar mezclas durante migración.

47.2. Presupuesto de proyecciones

Cada derivado consume un presupuesto compuesto por almacenamiento, escritura, eventos, monitoreo y carga cognitiva. El responsable revisa uso y beneficio en una fecha fija. Una proyección sin consultas, sin verificador o con reconstrucción rota entra en retiro.

47.3. Criterio de retiro

Retirar significa detener consumidores, verificar alternativas, eliminar el productor, esperar la ventana de seguridad y suprimir datos derivados. La fuente canónica permanece intacta. El proceso se prueba como cualquier migración y conserva un registro de decisión.

Análisis estadístico y resultados negativos

48.1. Unidad inferencial

La unidad es la corrida completa. Las consultas internas estiman un percentil de esa corrida y no se presentan como observaciones independientes. Las comparaciones usan pares generados con el mismo conjunto y semilla. Se informan mediana, intervalo remuestreado, diferencia, razón y dirección de cada par.

48.2. Multiplicidad y sensibilidad

Las familias de hipótesis se congelan antes de medir y se ajustan por comparaciones múltiples. El análisis de sensibilidad cambia una dimensión por vez: escala, tamaño de documento, selectividad, consistencia o ancho de proyección. Las exploraciones se marcan como tales y no reemplazan el análisis confirmatorio.

48.3. Publicación de derrotas

Una celda donde el documento canónico indexado iguala o supera a la proyección es un resultado útil. Delimita dónde COBRIA debe simplificarse. El paquete conserva corridas válidas aunque contradigan HN5 y explica cualquier descarte mediante una regla previa.

Paquete abierto y revisión independiente

49.1. Contenido del depósito

El depósito incluye protocolo, enmiendas, generador, adaptadores, manifiestos, eventos crudos, análisis, tablas, figuras, entorno, licencias y declaración de disponibilidad. Un comando de verificación comprueba huellas y vuelve a producir las salidas publicadas.

49.2. Revisión técnica

La persona revisora recibe una copia anonimizada, rúbrica y matriz de trazabilidad. Debe examinar equivalencia de configuraciones, unidad inferencial, exclusiones, afirmaciones y amenazas. Sus observaciones se responden una por una sin borrar desacuerdos sustantivos.

49.3. Criterio de cierre

La fase cuantitativa se considera completa cuando dos motores NoSQL terminan todas las celdas congeladas, el paquete reproduce cifras y una revisión independiente no detecta una falla que invalide la inferencia principal. Antes de ese punto, los resultados se describen como piloto o evidencia parcial.

Cuadernos de campo reproducibles

50.1. Bitácora de preparación

Antes de ejecutar una celda, la bitácora registra identificador, fecha, responsable, motor NoSQL, versión, configuración efectiva, semilla, conjunto, estado de índices y huellas de los artefactos. El registro no depende de memoria personal: se produce desde el mismo entorno que ejecuta la prueba y se conserva junto al resultado crudo. Si una configuración cambia, nace una celda nueva; nunca se sobrescribe la evidencia anterior.

La preparación verifica además espacio disponible, sincronización temporal, ausencia de carga ajena y aislamiento de credenciales. Una lectura breve de calentamiento confirma que el adaptador puede resolver documento, ámbito y revisión. Este control evita gastar una corrida completa para descubrir un error elemental de montaje.

50.2. Bitácora de ejecución

Cada fase deja marcas de inicio y final, conteos aceptados y rechazados, memoria máxima, operaciones por segundo, percentiles de latencia y códigos de error. Los mensajes libres se complementan con campos estructurados para permitir análisis automático. El operador puede anotar un incidente, pero no clasificarlo retroactivamente sin conservar la razón y la regla de exclusión utilizada.

50.3. Bitácora de cierre

Al terminar se verifica que los conteos coincidan con el manifiesto, que no existan ámbitos cruzados, que las colas estén drenadas y que las huellas puedan recalcularse. Una corrida válida

se sella como inmutable. Una corrida inválida también se conserva, etiquetada con su causa, porque la tasa y naturaleza de los fallos forman parte de la evaluación de operabilidad.

Migraciones y evolución del contrato

51.1. Cambio aditivo

El cambio preferido agrega un campo opcional con semántica, valor ausente explícito y versión de contrato. Lectores nuevos aceptan documentos antiguos; lectores antiguos ignoran el campo nuevo cuando esa conducta es segura. La prueba mínima cubre ambas direcciones y demuestra que la ausencia no se confunde con cero, cadena vacía o falso.

51.2. Cambio incompatible

Renombrar, dividir o reinterpretar un campo exige lectura doble, escritura controlada y una ventana de convivencia. La migración se expresa como función determinista e idempotente. Antes de retirar la versión previa se reconstruyen proyecciones, se comparan resultados por ámbito y se comprueba que ningún consumidor siga solicitando el contrato anterior.

51.3. Reversión

Toda migración declara el punto hasta el cual puede revertirse sin pérdida. Si el cambio destruye información, se conserva una copia temporal gobernada y con caducidad. Revertir no significa restaurar archivos a ciegas: significa volver a una combinación coherente de contrato, documentos, catálogos, índices, proyecciones y consumidores.

Rúbricas de conformidad COBRIA

52.1. Rúbrica del documento canónico

La conformidad se valora de cero a dos por criterio: identidad estable, ámbito explícito, revisión monotónica, procedencia, contrato versionado y validación en frontera. Cero indica ausencia; uno, presencia parcial o no verificable; dos, evidencia automática. Un promedio alto no compensa un cero en aislamiento o autoridad, que son condiciones de seguridad y no preferencias de estilo.

52.2. Rúbrica del derivado

Un derivado demuestra propietario, propósito de consulta, versión de proyección, idempotencia, frescura observable, reconstrucción total y procedimiento de retiro. Se evalúa también si puede confundirse con la fuente autorizada. La etiqueta “caché” no exime ningún criterio: si persiste y alimenta decisiones, debe poder explicarse y rehacerse.

52.3. Rúbrica del uso analítico

El conjunto analítico enlaza cada fila o fragmento con documento, revisión, ámbito y transformación. La partición de entrenamiento evita contaminación temporal y cruce de entidades. Una salida de inteligencia artificial conserva citas comprobables y se abstiene cuando no reúne evidencia autorizada suficiente. La métrica de calidad se publica junto con la de fuga, no en sustitución de ella.

Escenarios de aceptación integral

53.1. Escenario A: lectura y actualización concurrentes

Se inicia una lectura sobre una revisión conocida mientras llega una actualización del mismo documento. El sistema puede devolver la revisión anterior o la nueva según el contrato declarado, pero nunca una mezcla de campos. El evento posterior incluye la nueva revisión, la proyección converge y una repetición no duplica efectos.

53.2. Escenario B: reconstrucción bajo cambio de esquema

Se elimina un derivado, se activa una nueva versión de proyección y se recorre la fuente canónica. El resultado se compara con una construcción incremental limpia usando huellas, conteos y consultas de aceptación. Toda diferencia queda localizada por documento y transformación; una coincidencia meramente global no basta.

53.3. Escenario C: recuperación con inteligencia artificial

Una consulta solicita información disponible en dos ámbitos, pero la identidad solo autoriza uno. El recuperador filtra antes de clasificar, conserva procedencia y entrega únicamente fragmentos permitidos. Si esos fragmentos no sustentan una respuesta, el sistema se abstiene. La prueba falla ante una cita inexistente, una revisión obsoleta no declarada o cualquier exposición del ámbito prohibido.

53.4. Escenario D: degradación y recuperación

Se interrumpe temporalmente el consumidor que mantiene una proyección. La fuente acepta escrituras dentro de su contrato, el atraso se vuelve visible y las lecturas degradadas se etiquetan. Al reanudar, el consumidor procesa desde el último punto confirmado, converge sin duplicados y demuestra equivalencia con una reconstrucción independiente.

Parte IX

Anexos técnicos y reproducibilidad

Glosario operacional

Este glosario fija el uso de términos dentro del manuscrito. No pretende reemplazar definiciones generales de ingeniería de software.

Cuadro 54.1: Glosario COBRIA.

Término	Definición operacional
Autoridad	Capacidad reconocida para establecer un hecho de negocio.
Canónico	Representación autoritativa de una entidad dentro de un ámbito y versión.
Catálogo	Vocabulario o conjunto de referencias permitidas y gobernadas.
Conector	Adaptador que encapsula la API de almacenamiento o proveedor.
Contrato	Especificación versionada de entradas, salidas, políticas y verificaciones.
Dater	Nombre local de una clase que combina acceso, mapeo, Repositorio, caché o índices.
Conjunto de datos	Proyección analítica versionada con selección, transformación y manifiesto.
Derivado	Artefacto calculado desde una fuente que no adquiere autoridad automáticamente.
Entidad	Objeto con identidad, ámbito, atributos, ciclo de vida, versión y revisión.
Registro de evidencias	Registro minimizado que enlaza solicitud, evidencia, decisión y efecto.
Conjunto de características	Conjunto versionado de características con tiempo y procedencia.

Término	Definición operacional
Idempotencia	Propiedad por la cual repetir una operación no multiplica su efecto.
Inferencia	Salida probabilística de un modelo, distinta de un hecho confirmado.
Linaje	Grafo que conecta artefactos con fuentes, transformaciones y versiones.
Mapeador	Componente que traduce entre dominio y representación física.
Metadato	Dato que declara estructura, relación, política, versión o significado.
Cola de salida	Registro persistente de una intención de publicación asociada a un cambio.
Proyección	Representación derivada optimizada para una lectura, proceso o análisis.
Gestor de proyecciones	Componente que mantiene, verifica, reconstruye y versiona proyecciones.
Reconstruibilidad	Capacidad de regenerar un derivado desde fuentes y contratos suficientes.
Repositorio	Interfaz de acceso a entidades expresada en lenguaje del dominio.
Revisión	Contador o token utilizado para detectar actualización concurrente obsoleta.
Ámbito	Frontera de pertenencia y autorización, como organización, organización o contexto.
Servicio	Componente que expresa coordinación o capacidad de dominio/aplicación.
Caso de uso	Frontera autorizada que coordina una intención y sus efectos.
Marca de corte	Punto temporal o cursor declarado para procesar cambios o cortes.

Plantillas de contratos

55.1. Contrato de entidad

```
entity:
  name: ExampleEntity
  version: 1
  identity: id
  ámbito: [organizaciónId]
  revision: revision
  lifecycle: [draft, active, archived]
  fields:
    - name: title
      type: string
      required: true
      sensitivity: internal
  invariants:
    - "title must not be empty"
  retention: policy-entity-default
  responsable: domain-team
```

55.2. Contrato de Mapeador

```
mapper:
  entity: ExampleEntity
  version: 2
  readsPhysicalVersions: [1, 2]
  writesPhysicalVersion: 2
```

```
fields:
  id: key
  organizaciónId: organización_id
  title: display_name
  revision: rev
roundTrip:
  equality: semantic
  ignored: [storage_updated_at]
```

55.3. Contrato de proyección

```
projection:
  id: example-by-status
  version: 3
  sources: [ExampleEntity@1]
  key: [organizaciónId, status, id]
  consistency: eventual
  freshnessSloSeconds: 30
  deterministic: true
  idempotent: true
  authoritative: false
  rebuild:
    mode: [full, incremental]
    batchSize: 500
    checkpoint: required
    cambio de versión: version-alias
  verify:
    coverage: 1.0
    orphanCount: 0
  responsable: query-team
```

55.4. Contrato de conjunto de datos

```
conjunto de datos:
  id: example-training-set
  version: 4
  purpose: approved-task
```

```
ámbito: organización
sources: [ExampleEntity@1, ExampleEvent@2]
cutoff: required
transformationsCommit: abc123
esquema: example-conjunto de datos-esquema@4
quality:
  completenessMin: 0.98
  duplicateMax: 0
privacy:
  allowedFields: [category, eventTime, value]
  retentionDays: 90
lineage: required
responsable: equipo-analitica
```

55.5. Contrato de característica

```
feature:
  name: rolling_count_30d
  version: 2
  entity: ExampleEntity
  type: integer
  window: 30d
  availabilityTime: ingestionTime
  toleranciaFueraEnLinea: 0
  freshnessSeconds: 300
  sourceDataset: example-events@2
  responsable: ml-platform
```

55.6. Contrato de herramienta de agente

```
tool:
  name: propose_entity_change
  version: 1
  capability: entity.change.propose
  inputSchema: proposal-input@1
  outputSchema: proposal-output@1
  ámbito: inherited-and-verified
```

```
risk: medium
confirmation: required-before-effect
idempotencyKey: required
reversible: true
compensation: cancel-proposal
audit: evidence-ledger
```

Listas de comprobación de ingeniería y gobierno

56.1. Lista de comprobación de entidad canónica

Cuadro 56.1: Verificación de una entidad canónica.

OK	Criterio
☐	Posee identidad estable y definida.
☐	El ámbito es obligatorio y forma parte de toda clave de acceso.
☐	Distingue atributos, relaciones y derivados.
☐	Declara versión de esquema y estrategia de lectura histórica.
☐	Usa revisión o precondition para conflictos.
☐	Sus invariantes se ejecutan dentro de casos de uso.
☐	No conoce rutas ni SDK del proveedor.
☐	Tiene responsable, clasificación y retención.
☐	Su Mapeador pasa ida y vuelta semántico.
☐	Existe una sola autoridad para cada hecho.

56.2. Lista de comprobación de proyección

Cuadro 56.2: Verificación de una proyección reconstruible.

OK	Criterio
☐	Declara problema de lectura y consumidores.
☐	Identifica fuentes y versiones suficientes.
☐	No acepta escrituras autoritativas.
☐	Define ámbito, clave, función y versión.
☐	Declara consistencia y frescura.
☐	Es idempotente o documenta por qué no puede serlo.
☐	Posee rebuild total; incremental si está justificado.
☐	Verifica cobertura, huérfanos y diferencias.
☐	Tiene presupuesto, responsable, observabilidad y retiro.
☐	Puede hacer cambio de versión o reversión sin mezclar versiones.

56.3. Lista de comprobación multiinquilino

Cuadro 56.3: Verificación de aislamiento por ámbito.

OK	Criterio
☐	API, caso de uso y Repositorio requieren ámbito.
☐	Rutas e índices incluyen ámbito antes de identidad.
☐	Claves de caché y de idempotencia incluyen ámbito.
☐	Procesos de trabajo vuelven a autorizar mensajes.
☐	Exports, backups y conjuntos de datos conservan separación.
☐	Reconstrucciones requieren alcance explícito.
☐	Telemetría no expone cargas útiles de otras organizaciones.
☐	IDs iguales entre organizaciones forman parte de las pruebas.
☐	Las pruebas negativas abarcan lectura y escritura.
☐	Una fuga se trata como fallo crítico, no como tasa promedio.

56.4. Lista de comprobación de IA y agentes

Cuadro 56.4: Verificación de componentes inteligentes.

OK	Criterio
☐	Conjunto de datos y Conjunto de características poseen propósito, versión y linaje.
☐	El corte temporal evita utilizar información futura.
☐	Modelo y instrucción están versionados.
☐	Recuperación filtra ámbito y permisos antes de generar.
☐	La evidencia enlaza fragmento y fuente vigente.
☐	El contenido recuperado no puede redefinir políticas.
☐	Herramientas tienen capacidades mínimas y schemas.
☐	Acciones sensibles solicitan confirmación.
☐	Las operaciones mutantes son idempotentes.
☐	Una inferencia no se persiste como hecho sin caso de uso.
☐	Existen monitoreo, retiro y respuesta a incidentes.

Diccionario de métricas

Cuadro 57.1: Definición de métricas del estudio.

Métrica	Cálculo o unidad	Precaución
Latencia p50	Mediana en ms por corrida	No sustituir por media.
Latencia p95/p99	Percentil por corrida	Requiere muestra suficiente.
Rendimiento efectivo	Operaciones exitosas por segundo	Reportar errores y concurrencia.
A_r	Bytes leídos / bytes útiles	Definir respuesta vacía.
A_w	Escrituras físicas / escritura lógica	Separar réplicas internas.
A_s	Almacenamiento total / canónico	Incluir índices y staging.
Frescura	Confirmación a visibilidad correcta	Usar reloj y frontera declarados.
Cobertura Huérfanos	Esperados presentes / esperados	No detecta registros extra.
Exactitud de reconstrucción	Derivados sin fuente válida	Reportar conteo absoluto.
Cobertura de linaje	$1 - (faltantes + extras + diferentes) / esperados$	Desglosar errores.
Deriva	Artefactos con ruta completa / evaluados	No implica calidad.
Precisión@k	Distancia o prueba por variable	Cambio no siempre es defecto.
	Relevantes recuperados / k	Depende del juicio de relevancia.

Métrica	Cálculo o unidad	Precaución
Exhaustividad@k	Relevantes recuperados / relevantes	Requiere conjunto completo.
Abstención	Sin respuesta cuando no hay evidencia	Evaluar falsos rechazos.
Violaciones ámbito	Accesos o efectos cruzados	Objetivo obligatorio: cero.
Esfuerzo	Tiempo activo y artefactos modificados	Separar aprendizaje.
Tamaño de efecto	Diferencia estandarizada o razón	Informar intervalo.

Esquema del paquete reproducible

58.1. Estructura de directorios

```
cobria-replication/  
  README.md  
  LICENSE  
  CITATION.cff  
  environment/  
    versions.bloqueo  
    hardware-template.json  
  reference/  
    src/  
    tests/  
  contracts/  
    entities/  
    projections/  
    conjuntos de datos/  
    tools/  
  generators/  
    profile-r/  
    profile-a/  
    profile-b/  
  experiments/  
    performance/  
    reconstruction/  
    isolation/
```

```
    evolution/  
    intelligence/  
analysis/  
    notebooks/  
    scripts/  
results/  
    crudo/  
    processed/  
    figures/  
    manifests/
```

58.2. Manifiesto de corrida

```
{  
  "runId": "uuid",  
  "startedAt": "ISO-8601",  
  "revisión": "git-sha",  
  "profile": "R|A|B",  
  "configuration": "B0|B1|C1|C2|C3|C4",  
  "seed": 42001,  
  "N": 100000,  
  "organizaciones": 10,  
  "workload": "W2",  
  "concurrency": 32,  
  "projectionCount": 3,  
  "environmentHash": "sha256",  
  "conjunto de datosHash": "sha256",  
  "result": "completed|failed|excluded",  
  "exclusionReason": null  
}
```

58.3. Formato de evento de medición

```
{  
  "runId": "uuid",
```

```
"operationId": "uuid",
"operation": "query-by-status",
"organizaciónAlias": "t-03",
"startedNs": 0,
"durationNs": 0,
"reads": 0,
"writes": 0,
"bytesRead": 0,
"bytesWritten": 0,
"retries": 0,
"resultCount": 0,
"status": "ok|business-reject|error|timeout"
}
```

58.4. Reglas de publicación

Los resultados crudo son inmutables. El procesamiento escribe otra carpeta y conserva versión del guion. Las tablas del manuscrito se generan, no se editan manualmente. Una exclusión permanece en crudo y aparece en el reporte de flujo. Los secretos se suministran fuera del paquete y nunca se imprimen.

58.5. Lista de comprobación de réplica

1. verificar hash y licencia del paquete;
2. instalar versiones fijadas;
3. registrar hardware y límites;
4. generar perfiles con semillas publicadas;
5. ejecutar pruebas de corrección antes del prueba comparativa;
6. correr configuraciones en orden aleatorio;
7. conservar registros y manifiestos;
8. ejecutar análisis sin modificar crudo;

9. comparar tablas y documentar divergencias;
10. publicar entorno y resultados negativos.

Plantillas de registro y publicación

59.1. Ficha de experimento

Cuadro 59.1: Plantilla de ficha experimental.

Campo	Registro
ID y versión	
Hipótesis	
Variable independiente	
Variables dependientes	
Controles	
Conjunto de datos y hash	
Configuraciones	
Repeticiones	
Exclusiones previas	
Regla de decisión	
Revisión	

59.2. Ficha de resultado

Cuadro 59.2: Plantilla de resultado.

Campo	Registro
Corridas incluidas	_____
Corridas excluidas	_____
Resumen descriptivo	_____
Tamaño de efecto	_____
Intervalo	_____
Resultado de hipótesis	Apoyada / no apoyada / inconclusa
Costo adverso	_____
Amenaza principal	_____
Artefactos	_____

59.3. Ficha de incidente experimental

Un incidente registra identificador de corrida, instante, etapa, síntoma, impacto, datos afectados, causa conocida, decisión de exclusión, evidencia y acción preventiva. Un incidente de aislamiento se reporta aunque la corrida no sea válida para rendimiento.

59.4. Declaración de disponibilidad

La publicación deberá indicar qué artefactos son abiertos, cuáles se sustituyeron por datos sintéticos y cuáles no pueden compartirse. La imposibilidad de publicar código privado no impide compartir implementación neutral, generadores, contratos y resultados.

59.5. Declaración de autoría

El investigador y autor de la propuesta es Billy Jak Cordero Porras. Las contribuciones de revisores, colaboradores y herramientas se declararán según las políticas de la institución y de la sede de publicación. La autoría implica responsabilidad sobre afirmaciones, datos, análisis y correcciones.

Matriz de trazabilidad del manuscrito

Cuadro 60.1: Trazabilidad entre preguntas, artefactos y evaluación.

RQ	Artefacto COBRIA	Experimento	Resultado requerido
RQ1	Modelo de complejidad	P1–P5, R1–R2	Curvas y amplificación.
RQ2	Proyecciones de consulta	P1	Latencia, lecturas y bytes.
RQ3	Presupuesto de proyección	P2–P3	A_w , A_s y equilibrio.
RQ4	Gestor de proyecciones	R1–R5	Exactitud, tiempo y recuperación.
RQ5	Metadatos y generador	Evolución y generator test	Esfuerzo y divergencia.
RQ6	Ámbito y políticas	Pruebas negativas	Cero fugas y efectos.
RQ7	Perfiles A y B	Mapeo y ejecución comparada	Principios sin ruptura.
RQ8	Conjunto de datos/Feature contracts	AI-1–AI-2	Linaje y equivalencia.
RQ9	Retrieval, tools y ledger	AI-3–AI-7	Evidencia y acciones seguras.
RQ10	Lienzo y costo	Todos, incluidos negativos	Región de no adopción.

60.1. Trazabilidad de partes

Cuadro 60.2: Función de cada parte del documento maestro.

Parte	Función	Producto
I	Plantear problema y preguntas	Hipótesis y alcance.
II	Formalizar artefacto	Modelo, contratos y algoritmos.
III	Razonar sobre costo	Complejidad y optimización.
IV	Extender a datos e IA	Linaje, RAG, agentes y gobierno.
V	Conectar con implementaciones	Casos anónimos y referencia.
VI	Diseñar evaluación	Experimentos y análisis.
VII	Sintetizar y concluir	Guía, contribuciones y límites.
VIII	Facilitar réplica	Plantillas, checklists y diccionario.

60.2. Criterio de completitud para una versión publicable

El manuscrito estará listo para someterse como estudio empírico cuando: la referencia y los generadores sean públicos; P1, P2, R1, R2, aislamiento, evolución, AI-1 y AI-3 tengan corridas reproducibles; las hipótesis se actualicen sin eliminar resultados negativos; otro evaluador revise una muestra; y tablas y figuras provengan de scripts.

Si se publica antes de esos experimentos, debe presentarse como propuesta arquitectónica y protocolo de investigación, no como validación de rendimiento. Esta distinción debe aparecer en título, resumen, conclusiones y carta de presentación.

60.3. Cierre de los anexos

Los anexos transforman COBRIA de una descripción conceptual en un artefacto inspeccionable. Los contratos hacen explícitas sus propiedades; los checklists permiten auditar una implementación; el diccionario fija métricas; y el paquete reproducible define cómo convertir futuras ejecuciones en evidencia.

La utilidad de estas plantillas depende de su aplicación crítica. Marcar todas las casillas sin pruebas no aumenta madurez. Cada afirmación debe conservar una ruta hacia código, configuración, datos, ejecución o revisión que otra persona pueda examinar.

Referencias

- Armbrust, M., Ghodsi, A., Xin, R., & Zaharia, M. (2021). Lakehouse: A New Generation of Open Platforms that Unify Data Warehousing and Advanced Analytics. *11th Conference on Innovative Data Systems Research*. https://www.cidrdb.org/cidr2021/papers/cidr2021_paper17.pdf
- Aulbach, S., Grust, T., Jacobs, D., Kemper, A., & Rittinger, J. (2009). A Comparison of Flexible Schemas for Software as a Service. *Proceedings of the 2009 ACM SIGMOD International Conference on Management of Data*, 881-888. <https://doi.org/10.1145/1559845.1559937>
- Buneman, P., Khanna, S., & Tan, W.-C. (2001). Why and Where: A Characterization of Data Provenance. *Database Theory—ICDT 2001*, 316-330. https://doi.org/10.1007/3-540-44503-X_20
- Chang, F., Dean, J., Ghemawat, S., Hsieh, W. C., Wallach, D. A., Burrows, M., Chandra, T., Fikes, A., & Gruber, R. E. (2006). Bigtable: A Distributed Storage System for Structured Data. *7th USENIX Symposium on Operating Systems Design and Implementation*, 205-218. <https://www.usenix.org/conference/osdi-06/bigtable-distributed-storage-system-structured-data>
- Cooper, B. F., Silberstein, A., Tam, E., Ramakrishnan, R., & Sears, R. (2010). Benchmarking Cloud Serving Systems with YCSB. *Proceedings of the 1st ACM Symposium on Cloud Computing*, 143-154. <https://doi.org/10.1145/1807128.1807152>
- Corbett, J. C., Dean, J., Epstein, M., Fikes, A., Frost, C., Furman, J. J., Ghemawat, S., Gubarev, A., Heiser, C., Hochschild, P., Hsieh, W., Kanthak, S., Kogan, E., Li, H., Lloyd, A., Melnik, S., Mwaura, D., Nagle, D., Quinlan, S., . . . Woodford, D. (2012). Spanner: Google's Globally-Distributed Database. *10th USENIX Symposium on Operating Systems Design and Implementation*, 251-264. <https://research.google/pubs/spanner-googles-globally-distributed-database-2/>
- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to Algorithms* (4.^a ed.). MIT Press.

- DeCandia, G., Hastorun, D., Jampani, M., Kakulapati, G., Lakshman, A., Pilchin, A., Sivasubramanian, S., Voshall, P., & Vogels, W. (2007). Dynamo: Amazon's Highly Available Key-value Store. *Proceedings of Twenty-First ACM SIGOPS Symposium on Operating Systems Principles*, 205-220. <https://doi.org/10.1145/1294261.1294281>
- Fowler, M. (2002). *Patterns of Enterprise Application Architecture*. Addison-Wesley. <https://martinfowler.com/eaCatalog/>
- Fowler, M. (2005, 12 de diciembre). *Event Sourcing*. Consultado el 23 de agosto de 2026, desde <https://martinfowler.com/eaDev/EventSourcing.html>
- Gebru, T., Morgenstern, J., Vecchione, B., Vaughan, J. W., Wallach, H., Daumé III, H., & Crawford, K. (2021). Datasheets for Datasets. *Communications of the ACM*, 64(12), 86-92. <https://doi.org/10.1145/3458723>
- Gupta, A., & Mumick, I. S. (1995). Maintenance of Materialized Views: Problems, Techniques, and Applications. *IEEE Data Engineering Bulletin*, 18(2), 3-18.
- Gupta, A., Mumick, I. S., & Subrahmanian, V. S. (1993). Maintaining Views Incrementally. *Proceedings of the 1993 ACM SIGMOD International Conference on Management of Data*, 157-166. <https://doi.org/10.1145/170035.170066>
- Hevner, A. R., March, S. T., Park, J., & Ram, S. (2004). Design Science in Information Systems Research. *MIS Quarterly*, 28(1), 75-105. <https://doi.org/10.2307/25148625>
- Jain, R. (1991). *The Art of Computer Systems Performance Analysis*. Wiley.
- Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Kuttler, H., Lewis, M., Yih, W.-t., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *Advances in Neural Information Processing Systems*, 33, 9459-9474. <https://proceedings.neurips.cc/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html>
- Li, A., et al. (2023). Managed Geo-Distributed Feature Store: Architecture and System Design. <https://doi.org/10.48550/arXiv.2305.20077>
- Microsoft. (2025a). *Command and Query Responsibility Segregation (CQRS) Pattern*. Consultado el 20 de agosto de 2026, desde <https://learn.microsoft.com/en-us/azure/architecture/patterns/cqrs>
- Microsoft. (2025b). *Materialized View Pattern*. Consultado el 20 de agosto de 2026, desde <https://learn.microsoft.com/en-us/azure/architecture/patterns/materialized-view>
- Mitchell, M., Wu, S., Zaldivar, A., Barnes, P., Vasserman, L., Hutchinson, B., Spitzer, E., Raji, I. D., & Gebru, T. (2019). Model Cards for Model Reporting. *Proceedings of the Conference on Fairness, Accountability, and Transparency*, 220-229. <https://doi.org/10.1145/3287560.3287596>

- National Institute of Standards and Technology. (2023). *Artificial Intelligence Risk Management Framework (AI RMF 1.0)* (inf. téc. N.º NIST AI 100-1). U.S. Department of Commerce. <https://doi.org/10.6028/NIST.AI.100-1>
- National Institute of Standards and Technology. (2024). *Artificial Intelligence Risk Management Framework: Generative Artificial Intelligence Profile* (inf. téc. N.º NIST AI 600-1). U.S. Department of Commerce. <https://doi.org/10.6028/NIST.AI.600-1>
- Orr, L., Sanyal, A., Ling, X., Goel, K., & Leszczynski, M. (2021). Managing ML Pipelines: Feature Stores and the Coming Wave of Embedding Ecosystems. <https://doi.org/10.48550/arXiv.2108.05053>
- Pavlo, A., Angulo, G., Arulraj, J., Lin, H., Lin, J., Ma, L., Menon, P., Mowry, T. C., Perron, M., Quah, I., Santurkar, S., Tomasic, A., Toor, S., Van Aken, D., Wang, Z., Wu, Y., Xian, R., & Zhang, T. (2017). Self-Driving Database Management Systems. *8th Conference on Innovative Data Systems Research*. <https://www.cidrdb.org/cidr2017/papers/p42-pavlo-cidr17.pdf>
- Peppers, K., Tuunanen, T., Rothenberger, M. A., & Chatterjee, S. (2007). A Design Science Research Methodology for Information Systems Research. *Journal of Management Information Systems*, 24(3), 45-77. <https://doi.org/10.2753/MIS0742-1222240302>
- Rico, A., Noguera, M., Garrido, J. L., Benghazi, K., & Barjis, J. (2016). Extending Multi-Tenant Architectures: A Database Model for Multi-Target Support in SaaS Applications. *Enterprise Information Systems*, 10(4), 400-421. <https://doi.org/10.1080/17517575.2014.947636>
- Scherzinger, S., Klettke, M., & Störl, U. (2013). Managing Schema Evolution in NoSQL Data Stores. *arXiv preprint arXiv:1308.0514*. <https://doi.org/10.48550/arXiv.1308.0514>
- Sculley, D., Holt, G., Golovin, D., Davydov, E., Phillips, T., Ebner, D., Chaudhary, V., Young, M., Crespo, J.-F., & Dennison, D. (2015). Hidden Technical Debt in Machine Learning Systems. *Advances in Neural Information Processing Systems*, 28, 2503-2511. <https://papers.nips.cc/paper/5656-hidden-technical-debt-in-machine-learning-systems>
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention Is All You Need. *Advances in Neural Information Processing Systems*, 30, 5998-6008. <https://papers.nips.cc/paper/7181-attention-is-all-you-need>
- Yaish, H., Goyal, M., & Feuerlicht, G. (2014). Adaptive Database Schema Design for Multi-Tenant Data Management. *IEEE Transactions on Knowledge and Data Engineering*, 26(9), 2079-2093. <https://doi.org/10.1109/TKDE.2013.42>